



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ INFORMATYKI

Projekt dyplomowy

An application for generating videos based on text descriptions
Aplikacja do generowania wideo na podstawie opisów tekstowych

Autor: Ryszard Żmija, Maciej Grzybacz, Michał Proć
Kierunek: Informatyka
Opiekun pracy: dr hab. inż. Rafał Dreżewski, prof. AGH

Kraków, 2026

Contents

1	Project goals and vision	6
1.1	Introduction	6
1.2	The text-to-video generation task	9
1.2.1	Defining text-to-video generation	9
1.2.2	Key challenges	10
1.2.3	Architectural approaches to text-to-video generation	11
1.3	General Functional and Non-Functional Objectives of the Project	26
1.3.1	Motivation	26
1.3.2	Functional Objectives	27
1.3.3	Non-Functional Objectives	29
1.4	Prototype Implementation and Feasibility Analysis	30
1.5	Risk Analysis	30
1.5.1	Technical Risks	30
1.5.2	Content Safety Risks	30
1.6	Chapter Summary	31
2	Functional scope	32
2.1	Introduction	32
2.1.1	Purpose of the chapter	32
2.2	Project scope	32
2.2.1	Planned functionality	32
2.2.2	Excluded functionality	33
2.2.3	Optional functionality	33
2.2.4	Performance limitations	34
2.3	System users and external systems	34
2.3.1	System users	34
2.3.2	External systems	35
2.4	Functional requirements	35
2.4.1	Account management	35
2.4.2	Video generation workflow	36
2.4.3	Gallery and retrieval	36
2.4.4	Video generation job management	37
2.4.5	User interface capabilities	37
2.5	Non-functional requirements	37
2.5.1	Scalability requirements	38
2.5.2	Reliability requirements	38
2.5.3	Usability requirements	38
2.5.4	Maintainability requirements	38

2.5.5	Resource constraints	39
2.6	Usage and testing scenarios	39
2.6.1	Generating videos	39
2.6.2	Managing videos	41
2.6.3	Account management	42
2.7	Summary	45
3	Selected realization aspects	46
3.1	Introduction	46
3.2	Architecture Overview	46
3.3	Detailed Architecture	47
3.3.1	User Interface	47
3.3.2	API Server	48
3.3.3	Video Generation Pipeline	48
3.3.4	Data Persistence	51
3.4	Technological Stack	52
3.4.1	Backend Technologies	52
3.4.2	Frontend Technologies	53
3.4.3	Infrastructure Technologies	54
3.4.4	Machine Learning Libraries	56
3.4.5	Pre-trained Models	57
3.4.6	Supporting Tools and Libraries	58
3.5	Testing and Quality Assurance	61
3.5.1	Manual Testing and API Mocking	61
3.6	Performance Analysis	61
3.6.1	Test Environment	61
3.6.2	Performance Impact of Interpolation	62
3.6.3	Performance Impact of Upscaling	64
3.6.4	Summary	64
4	Work organization	66
4.1	Project Characteristics and Methodology	66
4.2	Development Timeline	67
4.2.1	Prototyping and technology exploration	67
4.2.2	System integration and deeper dive into deep learning research	67
4.2.3	Integration, Testing, and Finalization	68
4.3	Project Stakeholders	68
4.4	The Team and Division of Duties	69
4.5	Work Organization and Tools	70
4.5.1	Communication	70
4.5.2	Task Management	71
4.5.3	AI Tool Usage	71
4.6	Techniques and Practices Applied	72
4.6.1	Development Workflow and Version Control	72
4.6.2	Testing and Validation	72

5	Project results	74
5.1	Introduction	74
5.2	Overview of the Implemented System	74
5.2.1	Video Generation Pipeline	74
5.2.2	Backend Infrastructure	75
5.2.3	Web Application Frontend	76
5.3	Application Walkthrough	78
5.3.1	User Registration and Authentication	79
5.3.2	Video Generation Interface	81
5.3.3	Generation Progress Monitoring	83
5.3.4	Video Gallery and Management	84
5.3.5	Social Media	86
5.3.6	Explore Section	87
5.3.7	User settings	88
5.4	Performance Results Summary	89
5.5	Limitations and Known Issues	89
5.5.1	Hardware Limitations	90
5.5.2	Architectural Limitations	90
5.6	Future Improvements	90
5.6.1	Pipeline Improvements	90
5.6.2	Feature Extensions	91
5.6.3	Infrastructure Scaling	91
5.7	Conclusions and Reflections	92
5.7.1	Project Success Evaluation	92
5.7.2	Challenges Faced During Development	92
5.7.3	Satisfaction with Outcomes	93
	List of Figures	99
	List of Tables	101

Chapter 1

Project goals and vision

1.1. Introduction

The rise of artificial intelligence and deep learning has opened up a new era of rapid technological innovation and transformative research in computer science. Over the past 15 years, a paradigm shift toward deep learning has driven an explosion of breakthroughs, building on decades of theoretical work. As researchers strive to develop systems with human-like reasoning and adaptability, they continue to push the boundaries of what is computationally possible—motivated both by practical applications and a deeper curiosity about the nature of intelligence itself. Fundamental advances, such as the introduction of new neural network architectures and effective training methods, have enabled AI systems to tackle previously intractable tasks across language understanding, visual perception, and complex decision-making. Yet, several fundamental challenges remain unsolved. For example, current AI systems struggle with systematic compositional generalization—they don't reliably compose known concepts in novel ways like humans do. A model might excel at individual tasks but fail when asked to combine them in ways not seen during training. This suggests that we are missing something fundamental about how abstract reasoning works.

Even though for most people outside the field of computer science it may seem like artificial intelligence is a very recent invention, the ideas of constructing machines which exhibit intelligence date back to the 1940s and 1950s. In 1950 Alan Turing posed the foundational question in his paper "Computing Machinery and Intelligence" [1] - "Can machines think?" and tried to reason about its viability and implications. The core takeaway was that intelligence is a matter of observable behavior, not a mysterious inner essence, and there is no fundamental reason machines can't exhibit it. It argued that constructing intelligent machines is a legitimate scientific and engineering pursuit, not science fiction or philosophical impossibility. Turing devised a test in which a human interrogator has text-based conversations with two hidden participants - one human, one machine. If the interrogator cannot reliably determine which is which after questioning both, the machine is said to have passed the test. The Turing Test remains historically significant not as a practical evaluation method, but for establishing that intelligence should be assessed through observable behavior rather than metaphysical criteria. This behavioral perspective—that what matters is what a system does, not what it's "made of"—continues to underpin how we evaluate AI systems today, even though the specific test Turing proposed has been superseded by more targeted benchmarks.

Modern AI is dominated by an approach called deep learning which uses computational models called neural networks. Neural networks are computational systems organized in layers that learn to recognize patterns by adjusting connection strengths between artificial neurons.

The concept of neural networks can be traced back to the work of Warren McCulloch and Walter Pitts (1943). They combined basic insights into how brain neurons work with Russell and Whitehead's formal propositional logic and Turing's theory of computation to propose the first model of artificial neurons. They managed to show that networks of connected neurons could perform logical operations and simulate finite automata, laying groundwork for later proofs of computational universality. Later advances established methods for modifying the strengths of connections between neurons to train the neural networks to compute desired functions within some margin of error. For roughly sixty years, neural networks were not the dominant approach to building intelligent systems as they are today. They were perceived to have significant limitations, some theoretical and some stemming from the inability of researchers to experiment with large neural networks trained on large datasets. By the late 2000s computing technology such as the CUDA platform introduced by NVIDIA enabled general purpose programming of GPUs which by this point had become massively-parallel processors. Over the next couple of years research groups, most prominently Geoffrey Hinton's group at the University of Toronto, managed to implement algorithms for training neural networks on this new hardware. This culminated in the development of a deep convolutional neural network dubbed AlexNet in 2012 [2], which achieved dramatic improvement over state-of-the-art algorithms for image classification and initiated a flurry of research that has only accelerated since then.

A crucial aspect powering much of the progress in AI in recent years is the explosion of computing power available and neural network architectures that can effectively harness this compute. While fully-connected networks, where every neuron in one layer is connected to every neuron in the next layer, can theoretically approximate any continuous function on a compact set, they're impractical for most real-world problems because they lack inductive biases that match the structure of the data. For example, a fully-connected network processing images treats each pixel independently, requiring enormous amounts of data to learn basic facts like "nearby pixels are related" or "a cat looks the same whether it's on the left or right side of the image". Specialized architectures encode these assumptions directly: convolutional networks build in spatial locality and translation invariance for images, recurrent networks handle sequential dependencies for time-series data, and transformers capture long-range relationships through attention mechanisms. These architectural choices dramatically reduce the hypothesis space the network must search, making learning tractable with realistic amounts of data and computation.

In this thesis we focus on generative AI. Generative AI models are designed to create new content - text, images, video, or other data - rather than simply classifying or analyzing existing inputs. While discriminative models learn to map inputs to outputs, distinguishing between categories or making predictions, generative models learn to capture the statistical structure of the data itself, enabling them to synthesize novel samples. Modern generative systems are trained on massive datasets to capture statistical patterns at multiple levels allowing them to generate coherent text, images, videos or other complex outputs. To contextualize the text-to-video generation task that is the focus of this thesis, we first briefly survey the foundational architectural approaches that have shaped the field of generative AI.

The introduction of Generative Adversarial Networks (GANs) [3] marked a pivotal moment in the field. GANs comprise two neural networks—a generator and a discriminator—that compete against each other, enabling the generation of highly realistic images and data distributions previously considered extremely challenging to model. Around the same time, Variational Autoencoders (VAEs) [4] provided a complementary probabilistic framework, enabling researchers to generate new data points by sampling from learned latent spaces and allowing for more structured and interpretable representations.

Building on these foundations, Diffusion Probabilistic Models [5] [6] have more recently emerged as a powerful alternative, demonstrating exceptional capabilities particularly in image generation tasks. Diffusion models operate by gradually transforming random noise into coherent data through iterative refinement, achieving state-of-the-art fidelity and diversity in generated outputs. Their robust performance has rapidly positioned diffusion models at the forefront of generative research, owing to their stability during training and superior sample quality compared to earlier architectures.

Complementing these core generative architectures, the advent of attention mechanisms [7] and the Transformer model [8] has significantly enhanced generative capabilities, particularly for sequential data such as language. By efficiently capturing long-range dependencies, Transformers have enabled generative models to produce coherent and contextually relevant sequences of text—a development exemplified by Transformer-based large language models such as the GPT family, culminating in GPT-3.5 in 2022.

Collectively, these innovations—GANs, VAEs, Diffusion models, and Transformer architectures—have expanded the possibilities of generative AI across diverse data types and modalities. Beyond high-fidelity image and text generation, researchers have applied these models to tasks such as music composition [9], 3D object synthesis [10], speech generation [11], and cross-modal translation [12]. This trajectory has set the stage for exploration of even more complex generative tasks, including text-to-image and text-to-video generation. These multimodal tasks require the integration of multiple data modalities and demand both technical sophistication and computational scalability—topics addressed in depth in the following sections.

Multimodal generative models are designed to understand and generate content involving multiple types of data, such as text, images, audio, and video. These models typically learn joint representations that capture the complex relationships between different data types—for instance, associating textual descriptions with corresponding visual scenes or generating spoken language from written text. This capability to bridge distinct modalities enables a richer understanding of context and semantics than is possible with unimodal systems, and represents a step towards more comprehensive artificial intelligence.

Among the most ambitious areas within multimodal generative AI is text-to-video generation: the synthesis of video sequences from textual descriptions. This task requires not only the generation of visually coherent and realistic frames but also the creation of temporally consistent motion and narrative progression that accurately reflects the input text. Text-to-video generation extends the challenges of text-to-image generation by adding the dimension of time, demanding models that can understand and depict actions, interactions, and transformations over sequences of frames. The complexity of this task stems from the need to model intricate spatio-temporal dynamics, maintain long-range coherence, and align generated visual content precisely with nuanced textual prompts. While practical applications are still emerging as the technology matures, text-to-video generation holds promise for rapid prototyping in creative industries, synthetic data generation for training other AI systems, and lowering barriers to video content creation. The following section provides a foundational overview of text-to-video generation, outlining its main challenges and architectural approaches.

The goal of this thesis is to explore the current capabilities of text-to-video generation technology and to build a functional video generation system using open-weights models and open-source infrastructure. While state-of-the-art commercial systems such as OpenAI’s Sora and Google’s Veo demonstrate impressive results, they remain closed and inaccessible to independent researchers and developers. In contrast, a growing ecosystem of open-weights models provides an opportunity to study and combine these components into working systems. Our work takes advantage of this opportunity by constructing a video generation system from openly

available models, with a particular focus on making the system runnable on consumer-grade hardware rather than requiring data-center resources. Beyond the core generation functionality, we develop a web application with a user-friendly interface that allows users without specialized technical knowledge to generate videos and observe the capabilities of contemporary open-weights models firsthand. Subsequent chapters detail the specific components chosen for this system, the overall architecture, the implementation process, and the evaluation of the developed application.

1.2. The text-to-video generation task

1.2.1. Defining text-to-video generation

As introduced in the preceding overview of generative AI, text-to-video generation stands as a particularly ambitious frontier, extending the capabilities of text-to-image synthesis into the temporal domain. This section provides a foundational understanding of the text-to-video generation task, detailing its core definition, operational parameters, primary objectives, and inherent complexities that researchers and developers strive to overcome.

Text-to-video generation can be formally defined as the automated process of synthesizing a sequence of visual frames from a given natural language textual description. The fundamental aim is to produce a video that is not only semantically aligned with the input prompt but also visually coherent, realistic, and exhibits plausible temporal dynamics. The input to such a system is typically a textual prompt, which can range from a concise descriptive phrase or a single sentence (e.g., "a corgi playing fetch in a park on a sunny day") to more elaborate narratives or even script-like instructions, potentially specifying characters, actions, settings, and desired artistic styles. The output is a video sequence—a series of frames displayed in succession to create the illusion of motion—that ideally embodies the textual description in its entirety.

The successful generation of video from text hinges on achieving several related core objectives. Firstly, **semantic alignment** is paramount; the generated visual content must accurately interpret and reflect the meaning, objects, actions, and context conveyed in the input text. Secondly, the output must possess high **visual quality**, meaning frames should be clear, detailed, and free from distracting artifacts, approaching the fidelity of real-world footage or the desired artistic style. Finally, and critically differentiating this task from static image generation, is the need for **temporal coherence**. This implies that the video must present logical and smooth progression over time, with consistent appearance of objects and characters, natural-seeming motion, and believable transitions between states or scenes. High-quality text-to-video generation systems strive to meet these objectives, as illustrated by the example from Imagen Video [13] in Figure 1.1. Addressing these aspects simultaneously constitutes the central challenge of the text-to-video generation task.



Figure 1.1: An example of text-to-video generation by Imagen Video, depicting leaves falling onto a lake. This sequence demonstrates strong semantic alignment with the prompt "A bunch of autumn leaves falling on a calm lake to form the text Imagen Video. Smooth.". It also shows high visual quality in the rendering of the leaves and water, and crucial temporal coherence in the motion of the falling leaves along with the subtle ripples on the water surface. (Keyframes from Ho et al. [13]. Further examples and project details available at <https://imagen.research.google/video/>)

1.2.2. Key challenges

Generating high-quality video from textual descriptions involves particular technical challenges, distinct from those in other generative tasks such as text-to-image generation. These challenges stem from the essential requirements of correctly understanding detailed language, managing high-dimensional data inherent in videos, and crucially, modeling the complex nature of motion together with the development of a scene over time.

An essential part of text-to-video generation is an effective **semantic interpretation** of the provided prompts. Text descriptions can often be unclear or have multiple meanings, and they may require a good understanding of the context, especially when they involve many objects with specific features, complex actions, interactions, or abstract ideas like mood or artistic style. Effective models need to do more than just understand the sentence structure; they must connect the meaning of the text to believable visuals that change smoothly over time. If the model misunderstands small details in the prompt or fails to show the intended relationships, the resulting videos can be very different from what the user wanted. This could lead to, for instance, incorrect interactions between objects and completely nonsensical scenes.

The inherent nature of video as a data format contributes significantly to the complexity of the task. Video sequences are high-dimensional, typically comprising hundreds or even thousands of individual frames, each often a high-resolution image. Generating this extensive spatio-temporal data requires models with substantial expressive capacity and places a considerable computational load on both the training phase and subsequent inference stages. Consequently, the likelihood of introducing visual artifacts or inconsistencies in such complex data space is significantly higher than in the generation of static, individual images.

Perhaps one of the most defining and persistent challenges in text-to-video generation is achieving robust **temporal consistency** (also referred to as temporal coherence). While individual frames within a generated sequence might be plausible when viewed in isolation, ensuring that objects and characters maintain their identity, appearance, and attributes consistently across frames is a sophisticated problem. The objective is to prevent disruptive visual artifacts such as flickering, illogical transformations or morphing of objects, sudden disappearances, or unnatural, jerky motion. This requires the models to learn and effectively utilize long-range dependencies between frames, thereby establishing a smooth and believable frame-to-frame continuity. This aspect of maintaining coherence across time becomes progressively more difficult as the

target duration of the generated video increases.

Beyond achieving smooth continuity, a significant challenge lies in modeling **complex dynamics and motion with realism**. This involves more than just depicting simple object translations or rotations; it extends to generating believable interactions between multiple agents or characters ensuring that movements approximately respect the laws of physics. Furthermore, this includes the complex task of rendering the motion of non-rigid elements, such as the flowing of water. Capturing the subtleties of human gestures, facial expressions that convey emotion, or the coordinated but also individual motions of people in a crowd, also requires models to develop a robust internal understanding of how motion and behavior happen in the real world.

Finally, the advancement and overall capabilities of text-to-video models are closely tied to the **availability and quality of training data**. While large-scale datasets combining images and text have significantly propelled image generation, assembling text-to-video datasets of similar scope, diversity, and with accurate **annotations** is a more demanding task. Such datasets are crucial for teaching models the wide array of visual concepts, actions, temporal relationships between events, and stylistic variations they need to learn. It is important to note that major industry laboratories, such as OpenAI with its Sora model [14] and Google with its Veo model [15] (representing the state of the art as of early 2025), often leverage extensive resources and advanced data processing techniques (e.g., automated re-captioning of large unlabeled video corpuses) to amass vast, unique datasets. This can address certain aspects of data quantity and diversity for their specific models, though fundamental challenges related to data quality and inherent biases may still persist even at that scale. However, for the broader research community and in many specific domains, the difficulties in accessing and curating suitable public data remain considerable [16]. Consequently, factors such as the scarcity of data for specific types of content or domains, the potential for dataset bias to be learned and amplified by the models, and the substantial cost and effort involved in data annotation can significantly slow down overall advancement, limiting the fairness, robustness, and generalization capabilities of developed text-to-video systems.

1.2.3. Architectural approaches to text-to-video generation

The transformation of textual descriptions into dynamic video sequences presents a complex challenge that has spurred the development of diverse and sophisticated model architectures. These architectures are required not only to interpret natural language and synthesize high-fidelity visual frames but also to accurately model the temporal dimension, ensuring that motion, object identity, and generated scenes evolve coherently over time. Broadly, approaches to text-to-video generation fall into two main categories: **end-to-end models**, which are specifically designed and trained for direct text-to-video synthesis, and **pipeline-based systems**, which adapt and extend powerful pre-trained text-to-image models by incorporating additional mechanisms for temporal modeling. Both strategies ultimately seek to translate textual prompts into complete, coherent video sequences by addressing the distinct yet interconnected tasks of language understanding, visual synthesis, and temporal dynamics.

End-to-end text-to-video architectures

End-to-end text-to-video models are explicitly designed and trained to manage the complete video generation task directly from textual input. These architectures typically integrate three primary building blocks: a language interpreter, a vision processor, and a temporal handler. Each of these components is essential for translating text into a coherent video sequence, and their development has progressed rapidly in recent years [17] [18].

1. Language interpreter The process of converting text to video starts with the *language interpreter*. This component is responsible for taking the input text prompt and converting it into a numerical representation, often an *embedding*, that the vision model can effectively utilize. The ability of these models to transform sequences of words into visual objects and to connect the textual content with the dynamics present in video is foundational. The evolution of language interpreters is depicted in Figure 2 of Cho et al. (2024) [17] and includes:

- **Recurrent neural networks (RNNs):**

- **How it works:** RNNs were frequently used as text prompt encoders in early text-to-video models, particularly those employing GAN architectures. In these systems, the input text prompt would first be transformed into text embeddings using various vectorization techniques (such as GloVe [19], Skip-thought vectors [20], or outputs from a CNN/MLP). These embeddings were then processed sequentially by the recurrent network (which could be a simple RNN, LSTM, or GRU) to derive a contextual understanding of the text. This was particularly relevant for models attempting to generate sequential scenes based on an overarching topic sentence.
- **Benefits:** RNNs are inherently suited for processing sequential data like text, making them a natural choice for encoding prompts in early text-to-video systems. They provided an established method for capturing local context and temporal dependencies within the input sentences.
- **Challenges:** Standard RNNs can face difficulties in capturing very long-range dependencies within complex textual narratives, although LSTMs and GRUs were designed to mitigate this issue to some extent. Compared to more modern transformer-based architectures, RNNs may be less effective at understanding intricate semantic nuances and long-distance contextual relationships present in detailed prompts.
- **Examples:** Early text-to-video models often paired GAN-based vision processors with RNN language interpreters. Examples include TGANS-C [21], T2V [22], StoryGAN [23], and TivGan [24].

- **Transformer-based models:**

- **How it works:** More contemporary text-to-video architectures, including those based on autoregressive models, vector-quantized approaches, and some diffusion models, have adopted Transformer models to convert text prompts into language tokens or embeddings. BERT (an encoder-only transformer known for its bidirectional attention mechanism) [25] and T5 (an encoder-decoder transformer designed for text-to-text transfer tasks) [26] are two common language encoders integrated into these generation models. Both architectures fundamentally encode text by learning representations through objectives like denoising or masked token prediction.
- **Benefits:** Transformers excel at capturing long-range dependencies and complex contextual relationships within text due to their self-attention mechanisms. They are highly effective at performing sequence recognition from text descriptions and correlating these with fine-grained details in the generated video. T5-family models, in particular, are favored in many powerful text-to-video systems due to their scalability; increasing the parameter count of T5 has been shown to substantially boost performance.

-
- **Challenges:** While Transformer encoders like BERT and T5 are excellent for detailed textual understanding, they might be limited in their innate ability for global understanding of the multimodal context required for video (i.e., the direct mapping of text to overall visual scene semantics) when compared to models explicitly pre-trained for vision-language alignment, such as CLIP.
 - **Examples:** Several text-to-video models have incorporated general Transformer-based language interpreters. For instance, Phenaki [27] utilizes T5-family models. Other examples that employ Transformer encoders (distinct from or in addition to contrastive pre-training like CLIP) include GODIVA [28], CoGVideo [29], and VideoPoet [30].
- **Contrastive models:**
 - **How it works:** A large proportion of recent text-to-video generation models leverage text encoders from vision-language pre-trained (VLP) models like CLIP [12]. CLIP was originally trained on vast datasets of image-caption pairs using a contrastive learning objective, which teaches the model to align representations of images and their corresponding textual descriptions in a shared embedding space. The CLIP text encoder, itself a Transformer, is therefore adept at producing text embeddings that are highly correlated with visual semantics, particularly for matching a text description to an image as a whole. This alignment also enables CLIP to perform zero-shot classification on arbitrary categories, though its primary value for generative models lies in the quality of its learned vision-language embedding space.
 - **Benefits:** The primary advantage of using CLIP-like text encoders is their proficiency in achieving strong text-to-visual alignment due to their contrastive pre-training objective, which allows for efficient matching of text and image (and by extension, video frame) content. This often leads to a better global understanding of how the text prompt as a whole should map to the visual scene in the generated video. Their widespread adoption in various visual generation tasks underscores their effectiveness in bridging the gap between textual descriptions and visual outputs.
 - **Challenges:** While CLIP text encoders are excellent for capturing the overall semantic correspondence between text and visuals (general representation), there might be a trade-off concerning the level of highly detailed, fine-grained compositional understanding or nuanced relational reasoning compared to what very large, general-purpose language models might offer, unless they are specifically fine-tuned for such detailed generation tasks or augmented with other mechanisms.
 - **Examples:** The majority of recent diffusion-based text-to-video models utilize contrastive model text encoders, prominently CLIP. Examples include Make-A-Video [31], Follow Your Pose [32], Tune-A-Video [33], AnimateDiff [34], Video LDM [35], and Stable Video Diffusion [36].

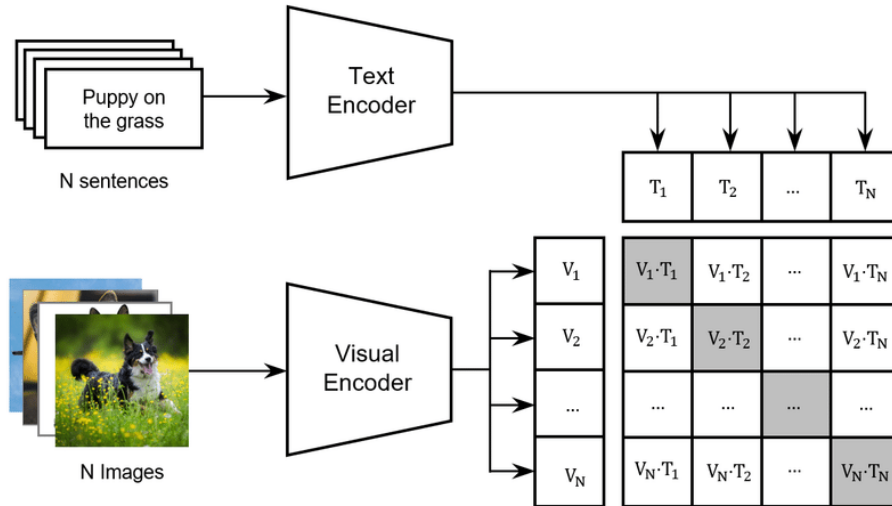


Figure 1.2: Diagram illustrating the contrastive learning approach in CLIP. The model processes N images and N text descriptions through dedicated encoders to produce visual (V_i) and textual (T_i) embeddings. A similarity matrix is then constructed, and the training objective is to maximize the similarity scores for correct image-text pairs (shaded diagonal entries) while minimizing them for incorrect pairs, thereby learning a shared representation space.

Image from Song et al. (2022) [37]. Licensed under CC BY-NC-ND 4.0 International.

2. Vision processor The vision processor is the core generative engine within text-to-video architectures. Its primary function is to synthesize the individual visual frames of the video, conditioned on the textual embeddings provided by the language interpreter. The choice and design of the vision processor have seen significant evolution, with various architectural families being explored. The evolution of these processors is depicted in Figure 3 of Cho et al. (2024) [17]:

- **VQ-VAE (Vector Quantized-Variational Autoencoder):**

- **How it works:** VQ-VAE models [38] operate by first encoding the input video into a set of discrete latent variables. This is achieved by compressing the video into a discrete latent space using a learned codebook through a process of vector quantization, which involves a nearest neighbor look-up in an embedding space. The generation then happens in this quantized latent space, based on the principles of variational inference similar to standard VAEs, which aim to reconstruct new data by sampling from an approximation of the posterior distribution of these latent variables.
- **Benefits:** Performing generation in a compressed latent space makes the training process less computationally costly compared to direct pixel-space generation. The discretization of the latent space also makes VQ-VAE adaptable to other data modalities like language and speech, and suitable for tasks involving reasoning and prediction. This approach has proven effective for the joint learning of text and video data.
- **Challenges:** While efficient, ensuring high-fidelity visual output that matches the quality of leading diffusion models can be a challenge. Moreover, the discrete nature, while beneficial for some aspects, may require sophisticated temporal handling to ensure smooth transitions if not inherently part of the autoregressive generation over discrete tokens.

- **Examples:** GODIVA [28] was a pioneering text-to-video model developed using VQ-VAE. Other models that have utilized VQ-VAE as their vision processor include CogVideo [29], StoryDALL-E [39], and Text2Performer [40].

- **Generative Adversarial Networks (GANs):**

- **How it works:** GANs [3] consist of two neural networks, a generator and a discriminator, which are trained in a competitive process. The generator creates video frames, while the discriminator attempts to distinguish these generated frames from real video frames. The generator’s objective is to produce frames that are realistic enough to deceive the discriminator, effectively maximizing the likelihood of the discriminator making an error.
- **Benefits:** GANs are primarily employed in text-to-video generation to produce video frames that exhibit both high visual quality and significant diversity. The adversarial training process, often without explicit regularization penalties on the generator’s output complexity, encourages the model to focus on fine-grained details, leading to visually sharp outputs. This makes them particularly useful for tasks like story visualization, where diverse scene generation is key.
- **Challenges:** Vanilla GANs often struggle with generating high-resolution video frames efficiently. This is due to the high computational cost of operating directly in pixel space for large frames and the potential limitations of standard CNN backbones in capturing complex relational compositions among visual elements. Training stability can also be a concern. More recent diffusion models have generally surpassed GANs in achieving a better balance between output diversity and fidelity.
- **Examples:** StoryGAN [23] was an early model that applied GANs to story visualization. TGANs-C [21] and T2V [22] were among the initial works using GANs for text-to-video tasks involving motion.

- **Autoregressive Transformers:**

- **How it works:** These models apply the transformer architecture directly to the task of video synthesis, typically operating on a sequence of discrete latent tokens representing video content. The video is generated token by token in an autoregressive manner, where each new token is predicted based on the previously generated tokens and the input text prompt. Phenaki, for example, introduced the C-ViViT architecture, which modifies the Video Vision Transformer (ViViT) [41] by incorporating causal attention to enable autoregressive generation along the time dimension. ViViT itself is designed to fuse both spatial and temporal information during the tokenization phase.
- **Benefits:** Synthesizing video in a discrete latent space using transformers has been shown to be effective for the joint learning of textual and visual data. This approach, leveraging discretization, can offer advantages such as support for multiple communication modalities, faster compression and decompression, and improved contextual understanding compared to some other methods.
- **Challenges:** The primary challenge is the sequential nature of generation, which can be computationally intensive and slow, especially for generating long videos with many tokens. Ensuring long-range temporal consistency purely through autoregressive prediction can also be demanding.

- **Examples:** Phenaki [27] was a pioneering model using this autoregressive transformer approach for variable-length video generation. Other models include for example VideoPoet [30].

- **Diffusion Models (DDPMs) and Latent Diffusion Models (LDMs):**

- **How it works:** Diffusion models [5] [6] operate by learning to reverse a gradual noising process. A forward process systematically adds Gaussian noise to input data (e.g., video frames or their latent representations) over a series of timesteps until the data becomes indistinguishable from pure noise. The core of the model is a neural network trained to denoise this data at each step; that is, given a noisy input z_t at timestep t , the network $e_\theta(z_t, t)$ predicts the noise that was added, or equivalently, the less noisy data z_{t-1} . Generation starts from random noise, which is then iteratively refined by the denoising network, conditioned on input prompts (like text), to produce a clean sample. **Latent Diffusion Models (LDMs)** [42], such as Stable Diffusion, perform this diffusion and denoising process in a lower-dimensional continuous latent space, which is learned by an autoencoder (VAE), making the process more computationally efficient. A recent advancement is the **Diffusion Transformer (DiT)** [43], which replaces the commonly used U-Net backbone in the denoising network with a transformer architecture, offering benefits in scalability and flexibility. OpenAI’s Sora model is based on such a DiT architecture. DiTs often incorporate conditioning information, such as text embeddings and timestep embeddings, through mechanisms like adaptive layer normalization (AdaLN).
- **Benefits:** Diffusion models have rapidly become a leading approach in text-to-video generation due to their ability to produce high-quality, diverse outputs, often outperforming GANs in balancing fidelity and diversity. The transformer-based variants like DiT are highly scalable, capable of benefiting from larger datasets and increased model parameters. Models like U-ViT, which also use transformers for diffusion, have achieved state-of-the-art results in image generation by treating all inputs (including time and conditioning) as tokens and employing effective architectural designs like long skip connections.
- **Challenges:** The iterative nature of the denoising process, which can involve hundreds or thousands of steps, can make sampling computationally expensive, particularly for high-resolution video data. While LDMs mitigate this by operating in latent space, the sampling time can still be considerable. Research into consistency models, which aim to distill the knowledge of a pre-trained diffusion model into a network that can generate samples in a single or very few steps, addresses this challenge. For video, effectively adapting T2I diffusion models requires careful architectural modifications to handle temporal consistency across frames, often addressed by the temporal handler component.
- **Examples:** A significant number of contemporary text-to-video models are built upon the foundation of Latent Diffusion Models, particularly Stable Diffusion. Numerous diffusion-based models, such as Make-a-video [31], Follow your pose [32], GPT4Motion [44], Dysen-VDM [45], Nuwa-XL [46] are documented in recent surveys of the field (an extensive list can be found in Table A1 of Cho et al., 2024). OpenAI’s Sora [14] is also a prominent example based on a Diffusion Transformer (DiT) architecture. Google’s Veo 3 [15] is another system that uses latent diffusion, where the diffusion process is applied jointly to temporal audio latents and

spatio-temporal video latents. In Veo, video and audio are encoded by respective autoencoders into compressed latent representations, and a transformer-based denoising network is optimized to remove noise from these noisy latent vectors during training. Furthermore, Imagen Video and Video LDM are notable video diffusion transformer systems that have advanced the application of diffusion techniques from text-to-image foundations to video generation.

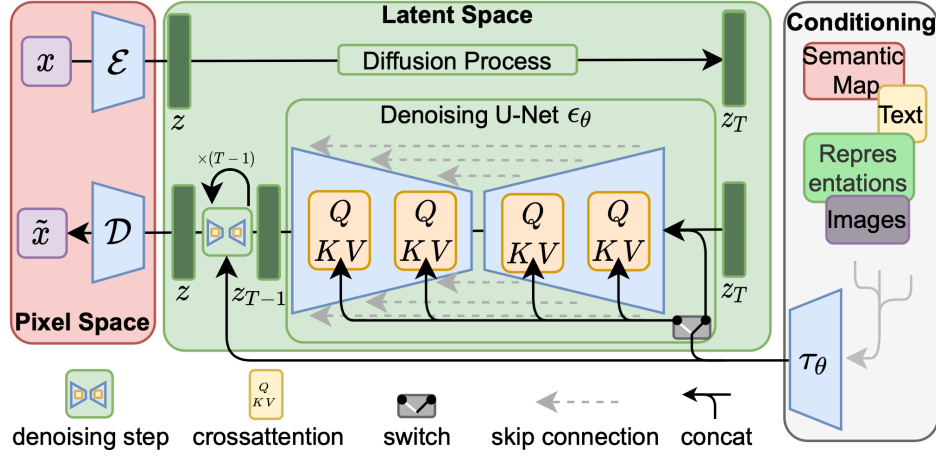


Figure 1.3: Architecture of the Latent Diffusion Model (LDM). The input image x from pixel space is first compressed into a lower-dimensional latent representation z by an encoder $\mathcal{E}$. The forward diffusion process (shown simplified at top) progressively adds noise to z yielding z_T . The core of the LDM is a time-conditional U-Net ϵ_θ operating in this latent space, trained to denoise z_t at timestep t back to z_{t-1} (or predict the added noise). This U-Net leverages skip connections and incorporates various conditioning modalities (like text or semantic maps) through a cross-attention mechanism, guided by embeddings from τ_θ . During inference, starting from random noise z_T , the U-Net iteratively denoises it, conditioned on the input prompt, to produce a clean latent representation z . Finally, a decoder $\mathcal{D}$ transforms z back into pixel space to generate the high-resolution image $\tilde{x}$. Image from Rombach et al. (2022) [42]

3. Temporal handler The temporal handler is a critical and unique component of text-to-video generation architectures, designed to complement the vision processor. While the vision processor focuses on learning and generating the visual content within each individual frame, the temporal handler is responsible for learning and modeling the dynamics of this content as it progresses from one frame to the next, ensuring coherent motion and consistent object appearance over time. The common mechanisms employed for temporal handling are illustrated in Figure 4 of Cho et al. (2024) [17] and are discussed below:

- **Temporal attention:**

- **How it works:** This approach directly incorporates the temporal dimension by allowing the model to selectively attend to different frames or temporal segments when generating the current frame or predicting future ones. It can be explicitly integrated into the architecture of generative transformers, for instance, through the temporal dimension of an axial transformer [47], or via spatiotemporal attention [48]

layers that jointly consider spatial and temporal contexts. More subtle implementations include using neural Ordinary Differential Equations (ODEs) [49] to approximate temporal dynamics or employing bidirectional masked attention transformers [50] that convert input frames into a temporal sequence by patchifying them.

- **Benefits:** Temporal attention is often considered the most straightforward method for incorporating temporality, especially in models with inherently autoregressive architectures such as those based on VQ-VAE or autoregressive transformers. This mechanism can ensure both the consistency and diversity of generated frames by relying on the tokenization of input frames during training and facilitating the injection of additional conditions like context memory or motion anchors.
- **Challenges:** Standard attention mechanisms can become computationally intensive with very long video sequences due to their quadratic complexity with respect to sequence length. Maintaining very long-range coherence solely through attention might still require substantial model capacity and data.
- **Examples:** Models like GODIVA [28] (VQ-VAE vision processor), Phenaki [27] (Autoregressive Transformer vision processor), and AnimateDiff [34] (Diffusion vision processor) are listed as utilizing temporal attention as their temporal handler. Other examples include CoGVideo [29] and VideoPoet [30].

- **Recurrent neural networks (RNNs):**

- **How it works:** For text-to-video models that do not possess a naturally autoregressive architecture, such as many GAN-based systems, attaching an RNN is a common solution for handling the temporal dimension. Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs) are the two most customary RNN types used for generating temporal sequencing from the input text prompt. Typically, the RNN takes the encoded text representation, a noise vector, and its previous hidden state as input to produce the current hidden state, which is then passed to the frame generator to output the video frame at the current timestep.
- **Benefits:** RNNs provide a well-established method for processing sequential data and introducing temporal dependencies in models where such capabilities are not inherent in the main vision processor.
- **Challenges:** While LSTMs and GRUs are designed to mitigate the vanishing/exploding gradient problems of simple RNNs, they can still face difficulties in capturing very long-range temporal dependencies effectively in complex video sequences. To ensure both temporal consistency and content diversity across frames, GAN-based models using RNNs often need to incorporate additional specialized modules, such as auxiliary frame discriminators [51], "gist" layers [23] that blend local and global context, or memory-augmented recurrent transformers (MART) [52].
- **Examples:** Early text-to-video models like TGANs-C [21] and StoryGAN [23] (both with GAN vision processors) employed RNNs for temporal handling. Other examples include Sync-DRAW [53], ObamaNet [54], T2V [22], and TivGan [24].

- **Pseudo-3D convolutions and attention:**

- **How it works:** Network inflation, specifically using pseudo-3D operations, is a widely adopted technique for incorporating temporal dimensionality into diffusion

models that are often built upon 2D U-Net architectures. Instead of directly replacing 2D layers with computationally heavier full 3D layers, this approach typically involves inserting a 1D temporal convolution layer after each 2D spatial convolution layer. This creates a "pseudo-3D" convolutional block. Similarly, a 1D temporal attention layer can be added after a 2D spatial attention layer to form a pseudo-3D attention block. During processing, the spatial layers typically handle each frame (often by temporarily merging batch and frame dimensions), while the temporal layers operate across the frame dimension to mix information and model temporal dynamics. Sinusoidal positional embeddings are also commonly used to provide frame index information to the input tensor.

- **Benefits:** This separable convolution [55] (and attention) technique significantly reduces the computational burden compared to using full 3D operations. A key advantage is that it allows for the preservation and leveraging of knowledge from powerful pre-trained 2D image generation models (whose weights in spatial layers can be kept frozen or fine-tuned), while the new temporal parameters can be trained from scratch on video data. This method, pioneered by models such as Make-A-Video [31], has become a prevalent, almost standard, adaptation technique for DDPMs to accommodate video generation tasks.
- **Challenges:** While more efficient than full 3D networks, these pseudo-3D layers still introduce additional computational costs. Ensuring robust temporal consistency and generating diverse inter-frame dynamics often requires these techniques to be coupled with other strategies, such as sophisticated noise scheduling for diffusion [56], specialized alignment modules [57], decoupled learning approaches [58], or trajectory anchoring methods [59].
- **Examples:** Make-A-Video was a pioneering diffusion model utilizing this approach. A vast majority of recent diffusion-based text-to-video models, such as Follow your pose [32], Nuwa-XL [46], Tune-a-video [33], Video LDM [35], Stable Video Diffusion [60], and Lumiere [61], employ pseudo-3D convolutions and/or attention for temporal modeling.

- **Large language models (LLMs) for temporal handling**

- **How it works:** This is a very recent trend that aims to borrow the extensive capabilities of LLMs in multimodal understanding and complex task planning for temporal modeling in video generation. One straightforward application involves using an LLM to take a simple user instruction and expand it into a comprehensive sequence of scene descriptions, which are then individually fed into a (often text-to-image) generation module to create a sequence of scenes [17]. A more subtle approach employs the LLM as a temporal encoder; here, the LLM generates temporal or scene evolution information that serves as an auxiliary condition (similar to a dynamic motion anchor or plan) to the main video generation module, alongside the primary text prompt.
- **Benefits:** LLMs excel at understanding complex narratives and can translate high-level textual goals into detailed, structured plans or descriptions that can guide video generation, potentially enabling more complex, story-driven, or logically consistent video sequences.
- **Challenges:** This is an emerging area, and the general robustness, controllability, and scalability of using LLMs for fine-grained temporal control across diverse video

types are still active research topics. The integration complexity of ensuring the LLM’s output effectively and consistently guides the vision model without introducing errors or undesirable biases from the LLM itself (like hallucination) is a key consideration.

- **Examples:** Models like GPT4Motion [44], Dancing Avatar [62], FlowZero [63], and Free-bloom [64] leverage LLMs for their temporal handling capabilities, often in conjunction with diffusion-based vision processors.

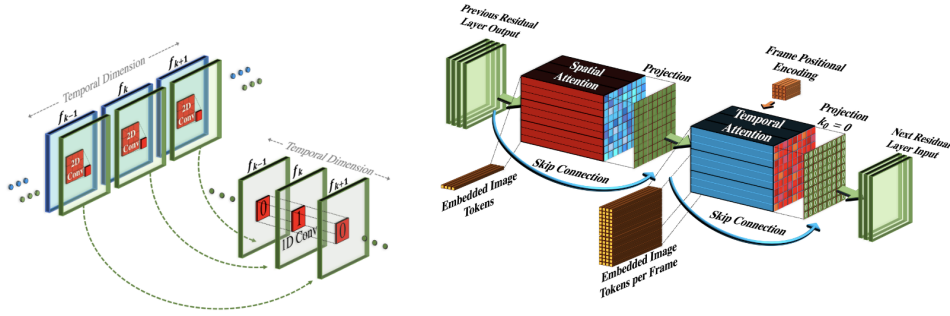


Figure 1.4: The architecture and initialization scheme of Pseudo-3D convolutional and attention layers, as proposed in Make-A-Video, designed to extend pre-trained Text-to-Image models for temporal processing.

Left panel (Pseudo-3D Convolutions): Illustrates the approach where each pre-trained spatial 2D convolutional layer is followed by a newly initialized 1D temporal convolutional layer. The temporal convolution is initialized as an identity function to facilitate a smooth transition from spatial-only to spatiotemporal processing, preserving learned spatial knowledge while enabling the learning of temporal dynamics.

Right panel (Pseudo-3D Attention): Shows temporal attention layers stacked after pre-trained spatial attention layers. Similar to the convolutional layers, the temporal attention blocks are initialized to behave as an identity function (e.g., by setting the temporal projection to zero initially). These factorized spatiotemporal mechanisms allow for the effective adaptation of T2I models to video generation by managing computational demands and leveraging existing knowledge. Image from Singer et al. (2022) [31].

Pipeline-based text-to-video generation: Extending text-to-image models

An alternative and widely adopted strategy for text-to-video generation involves constructing systems by extending powerful, pre-trained text-to-image (T2I) models [31] [34] [17]. This pipeline-based approach leverages the remarkable success of T2I models in synthesizing high-fidelity static images from text and augments them with specialized components or techniques to introduce temporal dynamics and motion. This methodology is particularly relevant as it aligns with the main approach taken in this thesis. The core idea is to utilize the strong visual and multimodal understanding learned by T2I models from vast image-text datasets, and then to specifically teach these systems "how the world moves", often using separate video data or purpose-built temporal modules [31] [34]. This can accelerate the training of the text-to-video model as it does not need to learn visual representations from scratch [31].

1. Foundational text-to-image (T2I) models as a base The cornerstone of pipeline systems is typically a state-of-the-art T2I model, often a Latent Diffusion Model (LDM) such as Stable Diffusion [42].

- **How it works:** As detailed previously, T2I diffusion models generally operate by iteratively denoising a random vector in a compressed latent space (learned by an autoencoder like a VAE), guided by text embeddings, with a U-Net architecture commonly serving as the denoiser to generate static images. For a visual explanation see figure 1.3.
- **Benefits as a foundation:** These models provide an exceptionally strong starting point due to their ability to generate diverse, high-resolution images with impressive spatial detail and strong semantic alignment to input text prompts [42]. This proficiency is a result of their pre-training on massive image-text datasets, endowing them with rich visual priors.
- **Challenges as a foundation:** Standard T2I models are designed to produce individual, independent images and inherently lack mechanisms for understanding or generating temporal coherence, modeling motion, or ensuring the consistent evolution of objects and scenes over a sequence of frames [34]. Their static nature can make it difficult to introduce complex, long-range temporal dynamics without significant architectural modifications or additional specialized components.
- **Examples:** Stable Diffusion is a very common open-source foundation model for such pipelines. Many community projects and research initiatives begin by adapting available T2I models [34].

2. Incorporating temporal dynamics and motion This is the crucial adaptation stage, focused on transforming a T2I model, which fundamentally generates static frames, into a system capable of producing video sequences with coherent motion.

- **Motion modules / Temporal layers:**
 - **How it works:** This prominent strategy involves designing and integrating specialized neural network modules—such as temporal attention layers, 1D temporal convolutions, or more complex pseudo-3D layers—into the architecture of a pre-trained T2I model, often within its U-Net denoiser [31] [34] [35]. These newly added modules are specifically trained on video data to learn motion patterns and temporal transitions between frames [34] [31]. Critically, the pre-trained weights of the T2I model responsible for spatial feature extraction and image synthesis are often kept frozen or only fine-tuned minimally to preserve their powerful generative capabilities [34] [17]. For instance, Make-A-Video extends its base T2I model’s U-Net and attention layers by incorporating new spatiotemporal convolutional and attention components that are initialized to preserve the original T2I function, ensuring a smooth transition to video fine-tuning [31]. Similarly, AnimateDiff is a well-known open-source example that injects "motion modules," primarily based on temporal attention, into various parts of the Stable Diffusion U-Net architecture, allowing it to generate animated sequences from text prompts [34].
 - **Benefits:** This approach enables the powerful T2I model to generate sequences of frames that exhibit learned motion. It is often parameter-efficient, especially if

only the new temporal modules require extensive training or fine-tuning on video data, while the bulk of the T2I model remains unchanged [34] [17]. This allows researchers and developers to leverage a wide variety of existing T2I models and their vast ecosystems of community-trained checkpoints and styles [34].

- **Challenges:** A key challenge is ensuring that the added temporal modules can effectively and consistently control the T2I backbone to produce coherent motion without degrading the original image quality or introducing visual artifacts. The diversity and complexity of motion that can be generated might be limited if the motion modules are too simplistic or if they are trained on video datasets with limited motion variety. Achieving highly complex camera movements or intricate, long-range object interactions can remain difficult.
- **Examples:** AnimateDiff [34] (see figure 1.5) is a widely used example for adapting Stable Diffusion. Make-A-Video by Singer et al. (2022) [31] explicitly details this extension of T2I models. Tune-A-Video demonstrates one-shot tuning of T2I models for specific video edits/motions. Many other research works and open-source projects demonstrate adding various forms of temporal layers or attention mechanisms to Stable Diffusion.

- **Frame-to-frame generation with explicit temporal conditioning:**

- **How it works:** Some pipeline approaches might involve generating frames more sequentially, where the generation of the current frame is explicitly conditioned on one or more previously generated frames. This can be achieved by feeding the latent representation of the previous frame(s) as an additional input to the T2I model when generating the current frame, sometimes in conjunction with estimated optical flow or motion vectors to guide the transformation.
- **Benefits:** This can create a very direct sense of temporal progression and allows for motion to be explicitly guided between frames if motion vectors are provided or estimated.
- **Challenges:** This method is highly susceptible to error accumulation over long sequences [65]. Small inconsistencies or artifacts in one frame can be propagated and amplified in subsequent frames, leading to a degradation of quality and coherence over time (often referred to as temporal drift). Maintaining long-term consistency, especially for object appearance and global scene structure, is a significant hurdle.
- **Examples:** While generating entirely new, long videos from text using purely frame-by-frame autoregressive conditioning is challenging due to error accumulation, this principle is foundational and evident in several contexts and specific models, for example Phenaki [27] or CogVideo [29].

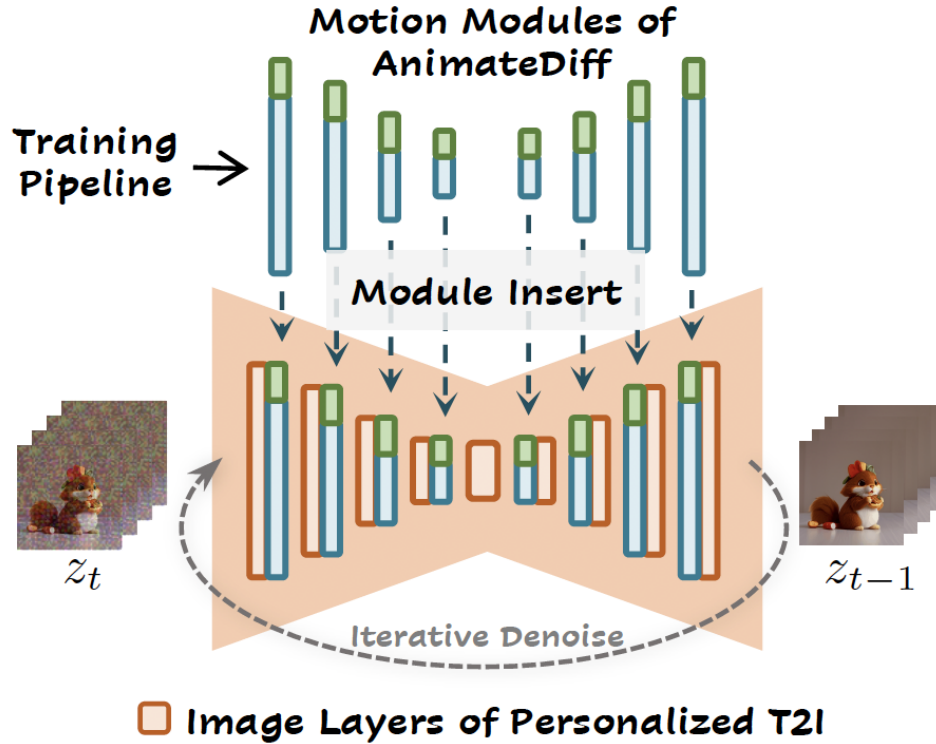


Figure 1.5: AnimateDiff’s inference architecture, illustrating how Motion Modules (pre-trained separately on video data, indicated by ‘Training Pipeline’ in the figure) are integrated with a personalized Text-to-Image (T2I) model to generate animations. The orange-outlined “Image Layers” belong to the existing personalized T2I model (e.g., a custom Stable Diffusion variant). These layers are responsible for rendering the spatial details and appearance of each video frame, processing each frame’s visual content independently as they would for a static image. The blue “Motion Modules of AnimateDiff” (optionally with green MotionLoRA enhancements) are specialized temporal networks, typically Transformer-based. Crucially, these modules have been separately pre-trained on large video datasets to learn diverse and transferable motion patterns (motion priors). During inference, these fully trained motion modules are inserted as “plug-and-play” components within the frozen architecture of the personalized T2I model. They operate across the sequence of frame latents, actively applying the learned temporal dynamics to ensure coherent motion throughout the generated animation. The combined system produces an animated sequence via an iterative denoising process (from z_t towards a clean z_0), effectively equipping static personalized T2I models with animation capabilities without needing to modify their original weights for motion. (It’s noteworthy that during the initial training phase of the motion modules themselves, their output layers are typically zero-initialized so they start as identity mappings. This ensures stable training when they are first introduced to the T2I architecture for learning motion, before they acquire their motion-generating capabilities). Image from Guo et al. (2024) [34].

3. Training strategies for pipeline models

- **How it works:** The strategies often involve training or fine-tuning the temporal modules (or sometimes the entire adapted model with a lower learning rate) on datasets of video clips. A notable approach, used by Make-A-Video [31], is to leverage unsupervised learning on unlabeled video footage purely to learn motion dynamics, effectively decoupling the learning of visual appearance (from text-image pairs) from the learning of motion

(from videos without text). Other approaches might involve image-video joint training, where the model is exposed to both static images (to reinforce visual quality and text alignment) and video clips (to learn temporal aspects) during its fine-tuning phase.

- **Benefits:** The ability to learn motion from unlabeled videos, as demonstrated by Make-A-Video, circumvents the need for large-scale paired text-video datasets, which are significantly more challenging and expensive to curate than text-image datasets or collections of unlabeled videos. This allows the system to learn general motion patterns from diverse and abundant video sources.
- **Challenges:** If motion is learned purely from unsupervised videos, effectively aligning these general learned motions with specific and varied textual prompts (which dictate the content, actors, and specific actions) can be non-trivial and relies heavily on the conditioning capabilities of the base T2I model [31]. Biases present in the video training data (e.g., common camera movements, types of actions) can influence the types of motion the model learns and prefers to generate. For example, When a model learns motion patterns (e.g., "walking," "rotating," "flowing") from videos without text labels, these motions are learned in a general, decontextualized way. The challenge then arises when a text prompt requests a specific character or object to perform a specific action that requires a nuanced version of that learned motion, or a combination of motions. For example, the model might have learned a generic "walking" motion, but making "an elderly astronaut walking slowly on the moon" requires precise adaptation of that general motion to the specified actor, action details, and environment.
- **Examples:** The training strategy of Make-A-Video [31], which combines T2I pre-training with learning motion from unlabeled videos, is a key example. Many community-driven efforts involve fine-tuning models like AnimateDiff on specific datasets to achieve particular motion styles or effects [34].

4. Post-processing for enhanced output quality A common and often essential characteristic of pipeline-based text-to-video generation systems is the inclusion of dedicated post-processing stages designed to substantially refine and enhance the initially generated raw sequence of frames [31]. The core generative models, while powerful, may not always produce outputs that are immediately production-ready or meet desired quality standards. This thesis, for instance, incorporates frame interpolation in a pipeline to increase the framerate of videos.

- **How it works:** Post-processing is a crucial stage in many text-to-video pipelines, designed to significantly enhance the raw video initially produced by core generative models. These core models often generate video at a lower frame rate or resolution to manage the immense computational demands of video synthesis, creating a sparser initial output. This output is then passed to specialized modules for refinement. Key among these is frame interpolation, which intelligently synthesizes new intermediate frames to create substantially smoother and more fluid motion. Concurrently, video super-resolution techniques increase the pixel dimensions and visual detail of each frame, transforming moderately-resolved inputs into crisp, high-definition visuals by learning to infer and reconstruct fine details. Additional steps might include color correction for aesthetic consistency and tools for removing visual artifacts like noise or flickering. For example, systems like Make-A-Video [31] explicitly implement such a pipeline, starting with a low-resolution, low-framerate video and then using dedicated networks to interpolate frames

and upscale resolution in stages. This layered approach allows for the efficient creation of detailed, high-frame-rate videos that would otherwise be challenging to generate directly. Ultimately, these post-processing techniques are vital for transforming a basic generated sequence into a polished, high-quality final product ready for viewing.

- **Benefits:** A significant benefit of employing a modular post-processing approach is that it allows the primary text-to-video generation model to dedicate its computational resources to the computationally intensive tasks of synthesizing coherent core content and fundamental motion from textual prompts. By offloading demanding operations like achieving high frame rates and resolutions to subsequent, specialized modules, the initial, intensive generation phase becomes more computationally manageable and efficient. This strategic division of labor not only streamlines the process but also typically results in a higher perceived quality in the final video, as each specialized module is highly optimized for its specific enhancement task, such as upscaling or frame insertion. Furthermore, such modularity offers considerable flexibility, enabling developers to select, update, or meticulously tune different post-processing tools to achieve desired aesthetic outcomes or performance benchmarks. This adaptability can even empower end-users with greater control over the final product. For instance, features like user-selectable output frame rates can often be directly controlled by adjusting parameters within the frame interpolation module, tailoring the video's fluidity to specific needs.
- **Challenges:** While post-processing significantly enhances generated videos, it presents several challenges that require careful consideration. Post-processing stages, themselves often sophisticated AI models, can inadvertently introduce their own specific visual artifacts—like unrealistic textures from super-resolution or motion distortions from frame interpolation—if not perfectly calibrated or if they encounter inputs outside their training distribution. These stages might also subtly alter the intended artistic style or crucial details of the content originally synthesized by the primary model, necessitating meticulous tuning to preserve creative intent.

A paramount and pervasive challenge also encountered is maintaining strict temporal consistency throughout all post-processing steps. For example, video super-resolution models must "hallucinate" or reconstruct fine details in a way that is perfectly coherent from one frame to the next; otherwise, the newly added details can appear to shimmer or flicker distractingly. Similarly, frame interpolation techniques face the complex task of generating entirely new in-between frames that seamlessly align with the motion, appearance, and identity of objects in the surrounding keyframes, avoiding any jarring discontinuities.

Finally, each additional module integrated into the post-processing pipeline inherently increases the system's overall complexity and adds to the total inference time. This cumulative computational cost can impact the speed and efficiency of video production, creating a trade-off between the level of polish and the practicality of the generation process.

- **Examples:** The use of dedicated post-processing modules is evident in both specially developed research systems and broader community applications. For instance, Meta AI's Make-A-Video [31] model clearly outlines how its custom-designed frame interpolation and super-resolution modules are essential to its full generation pipeline, playing a vital role in transforming initial low-resolution, low-framerate video drafts into polished, high-quality final outputs. Beyond such integrated systems, various readily available video enhancement tools are frequently adopted. For video frame interpolation, which aims to

create smoother motion by increasing frame rates, models like FILM (Frame Interpolation for Large Motion) [66] are well-regarded; FILM is known for its ability to handle significant motion effectively, often by using optical flow methods to accurately synthesize new frames. When it comes to video super-resolution for upscaling frames to higher resolutions and boosting visual detail, techniques adapted from advanced image super-resolution are commonly used. A prominent example here is Real-ESRGAN [67], an improvement over the original ESRGAN [68], which is widely chosen for its robust performance in upscaling diverse real-world images and video frames, making it a popular tool for enhancing AI-generated videos in many community projects.

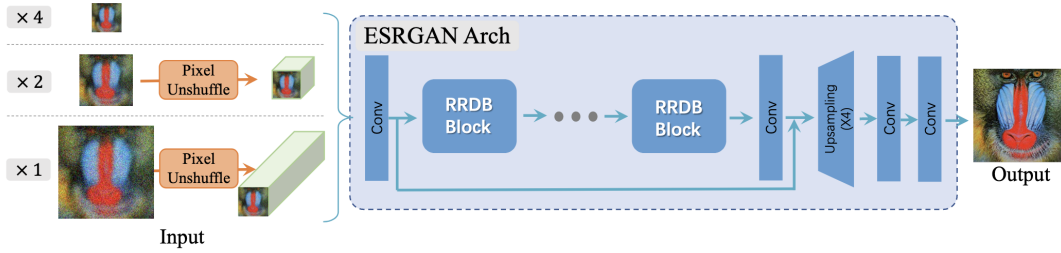


Figure 1.6: The generator architecture of the Real-ESRGAN model, adapted from the ESRGAN [68] design, for performing image super-resolution at various scaling factors. The central component, labeled "ESRGAN Arch," forms the core upscaling engine, primarily designed for a $\times 4$ magnification. An input low-resolution image (or its feature representation derived from pre-processing) initially passes through a convolutional layer. It then transits a deep sequence of Residual-in-Residual Dense Blocks (RRDBs), which are specialized for extracting and learning intricate hierarchical image features critical for high-quality restoration. Following the RRDBs, features undergo further convolution before being passed to an Upsampling (X4) module that increases the spatial resolution. Final convolutional layers then render the high-resolution output image. A prominent long skip connection bypasses the deep RRDB stack, facilitating the flow of low-frequency information and aiding in stable training. To accommodate different effective upscaling requirements, such as $\times 2$ or $\times 1$ (often for denoising/slight enhancement at near original size, or less magnification from a larger input), Real-ESRGAN employs a "Pixel Unshuffle" operation as an initial pre-processing step for these inputs. As illustrated for the $\times 2$ and $\times 1$ pathways, Pixel Unshuffle rearranges pixel data from an input image into a feature map with reduced spatial dimensions but an increased number of channels (depth). This intelligent transformation ensures that the main ESRGAN architecture always receives feature maps of a consistent internal spatial size, allowing the same powerful deep network to be efficiently utilized for various tasks while leveraging its deep feature learning capacity. Image from Wang et al. (2021) [67].

1.3. General Functional and Non-Functional Objectives of the Project

1.3.1. Motivation

Our primary objective for this project is to explore the current landscape of text-to-video generation by building a complete, functional video generation system using open-weights models and open-source infrastructure.

Text-to-video generation has advanced rapidly, with state-of-the-art commercial systems such as OpenAI’s Sora and Google’s Veo demonstrating remarkable results. However, these systems remain closed, offering limited insight into their architectures and training methodologies. In contrast, a growing ecosystem of open-weights models—including Stable Diffusion, AnimateDiff, and various frame interpolation and super-resolution networks—provides an opportunity to study and combine these components into working systems. Our project takes advantage of this opportunity by constructing a pipeline from openly available models and examining how they interact in practice.

Building such a system also requires robust software infrastructure. Open-source technologies such as Ray for distributed computing, FastAPI for backend services, and established machine learning frameworks enable us to create a scalable deployment that can handle the computational demands of video generation. A significant part of our work involves integrating these tools effectively—managing asynchronous processing, providing real-time progress feedback, and supporting concurrent users without degradation.

A secondary objective is to make this technology accessible beyond the research community. Current open-weights models frequently demand technical expertise, including familiarity with command-line interfaces and model configuration. Commercial alternatives, while more user-friendly, often impose significant costs and restrict customization. We address this gap by wrapping our pipeline in a web application with an intuitive interface, enabling users without specialized knowledge to generate videos and observe the capabilities of contemporary open-weights models firsthand.

1.3.2. Functional Objectives

Text-to-video Generation Pipeline

Our core objective is to build a text-to-video generation pipeline using a modular, pipeline-based approach. The system will leverage open-weights text-to-image models extended with temporal modeling capabilities—such as those provided by AnimateDiff—to generate base video sequences from textual prompts. To balance generation quality with computational efficiency, we plan to incorporate frame interpolation and upscaling stages into the pipeline. This approach allows us to generate fewer initial frames at lower resolution and then enhance the output through dedicated post-processing models, reducing the overall computation time while maintaining visual quality.

A guiding principle for our pipeline design is flexibility. Rather than targeting fixed output specifications, we aim to support a range of resolutions and frame rates, allowing users to adjust these parameters based on their needs and available hardware. This flexibility also enables us to explore different trade-offs between generation speed and output quality throughout development.

Crucially, we intend the entire system to remain runnable on consumer-grade GPUs rather than requiring data-center hardware. This constraint shapes our model selection and pipeline architecture decisions, prioritizing efficient models and processing strategies that can deliver acceptable results within the memory and compute limits of mid-range consumer hardware.

Configurable Generation Parameters

We aim to provide users with a wide array of configurable parameters to tailor video generation to their specific needs. At the most basic level, users will be able to enter a textual prompt

describing the desired video content and select from a range of output specifications including resolution, frame rate, and video duration.

Beyond these core settings, we plan to offer an advanced configuration interface for users who wish to have finer control over the generation process. This will include parameters such as guidance scale, which influences how closely the generated output adheres to the input prompt, as well as the ability to select from models pretrained with different visual styles. By providing an interface with both basic and advanced options, we aim to accommodate users with different levels of expertise—providing simplicity for casual users while preserving flexibility for those who wish to experiment with the underlying generation parameters.

Generation Tracking

We intend to build a robust infrastructure for tracking video generation tasks throughout their lifecycle. The system will persistently store generation parameters, status information, and results using a combination of a database for structured data and object storage for generated video files. This separation allows for efficient querying of generation metadata while handling large binary artifacts appropriately, and positions the system to scale as the number of users and generated videos grows.

The internal tracking infrastructure will also serve as the foundation for user-facing progress feedback. By maintaining up-to-date status information for each generation task, we can expose this data to the web frontend, enabling real-time progress updates that keep users informed as their videos are being generated.

User Management and Security

We aim to provide a secure web application that allows users to register, log in, and access the videos they have generated. Authentication will be implemented using JSON Web Tokens (JWT), enabling a stateless architecture where the backend does not need to maintain session state between requests. This approach simplifies horizontal scaling and aligns with modern best practices for web application design.

Authorization is a central concern. The system must ensure that users can only access their own generated videos and generation history—preventing unauthorized access to other users' content. We intend to enforce these access controls consistently across all relevant endpoints.

User Interface and Video Gallery

We aim to create a modern, intuitive user interface that guides users through the video generation process. The interface will provide a straightforward way to enter prompts, configure generation parameters, and submit generation requests. Once a generation is in progress, users will receive visual feedback through progress bars and status indicators, keeping them informed about how their request is advancing through the pipeline.

Each user will have access to a personal gallery displaying the videos they have generated. The gallery will present each video alongside the prompt that was used to generate it, allowing users to review their past generations and the inputs that produced them. Users will also be able to download their videos for use outside the application and delete videos they no longer want to see.

By combining accessible generation controls with a clear gallery view, we aim to provide a friendly experience that makes text-to-video generation approachable for users without technical backgrounds.

1.3.3. Non-Functional Objectives

Performance

Video generation is a computationally intensive process, and a significant part of our development effort will focus on ensuring effective utilization of available system resources. This includes careful selection of models and pipeline components that balance output quality with generation speed, as well as optimizing how these components are orchestrated to minimize idle time and redundant computation.

A key constraint guiding our performance objectives is GPU memory. We aim to build a system capable of generating good-quality videos using no more than 12 GB of GPU memory in its baseline configuration, scaling resource usage appropriately when users select more demanding generation parameters. This target reflects our goal of keeping the system runnable on consumer-grade hardware while still providing good output quality.

Scalability

We aim to design the system with horizontal scalability in mind, allowing it to grow from a single GPU to a cluster of multiple GPUs as demand increases. The architecture should support distributing generation tasks across available hardware, enabling concurrent processing of multiple user requests without degradation in response times.

A related objective is to clearly separate computations that require GPU resources from those that can be handled by the CPU. By distinguishing between these workloads, the system can schedule tasks more efficiently—ensuring that expensive GPU resources are reserved for model inference while lighter tasks such as request handling and data management are processed independently.

Usability

We aim to provide a modern and friendly user interface that makes video generation approachable for users regardless of their technical background. The interface should guide users naturally through the generation process, presenting options clearly and avoiding unnecessary complexity. Where advanced parameters are available, they should be organized in a way that does not overwhelm users who simply want to enter a prompt and generate a video.

Clear feedback is also essential. Users should always understand what the system is doing—whether their request is queued, in progress, or complete—and be able to navigate between generation and their personal gallery without confusion. Our goal is to minimize the learning curve so that new users can produce their first video with minimal friction.

The application will support both Polish and English languages, allowing users to interact with the interface in their preferred language.

Maintainability

We aim to build a modular application that clearly decomposes the complexity of the system into separate, well-defined components. Each module should have a focused responsibility and interact with other modules through clear interfaces. This separation of concerns makes the codebase easier to understand, test, and extend over time.

A modular architecture also facilitates future development. Individual components—such as specific models in the generation pipeline or elements of the backend infrastructure—can be updated, replaced, or improved independently without requiring changes throughout the entire

system. This flexibility is particularly valuable in a rapidly evolving field like generative AI, where new models and techniques emerge frequently.

1.4. Prototype Implementation and Feasibility Analysis

To verify the feasibility of the proposed approach, we developed and tested a prototype pipeline implementing the core video generation functionality. The prototype was run on an NVIDIA RTX 4070 Super, representative of the consumer-grade hardware we are targeting.

The pipeline follows a sequential processing approach consisting of four stages: initial frame generation from a textual prompt, temporal interpolation to increase smoothness, resolution enhancement to improve visual quality, and final video encoding. For frame generation we used AnimateDiff, for interpolation FILM (Frame Interpolation for Large Motion), and for upscaling Real-ESRGAN.

Prototype testing confirmed that our approach is viable within the constraints we have set. AnimateDiff demonstrated acceptable computational efficiency on our target hardware through memory optimization techniques that keep usage within the limits of consumer GPUs. FILM interpolation effectively enhanced animation fluidity, allowing us to generate fewer initial frames while still achieving smooth motion. Real-ESRGAN upscaling enabled us to produce high-resolution output without generating at full resolution from the start, further reducing computational requirements.

The feasibility study validates the technical viability of our video generation system concept and establishes a foundation for the full implementation described in the following chapters.

1.5. Risk Analysis

1.5.1. Technical Risks

Our approach relies on open-weights models designed to run on consumer-grade hardware, which introduces inherent limitations. These models will not match the output quality of state-of-the-art commercial systems that leverage significantly larger architectures and data-center-grade hardware. Users may encounter inconsistent results, particularly with complex prompts or when pushing the boundaries of what the chosen models can reliably produce.

The complexity of our system—spanning video generation, frame interpolation, upscaling, distributed task management, and a full web application—presents a risk of scope creep. Balancing the breadth of functionality we aim to deliver against the time and resources available will require careful prioritization throughout development.

1.5.2. Content Safety Risks

As with any generative AI system, there is an inherent risk that the models may produce inappropriate or biased content. These issues can stem from limitations or biases present in the training data used to develop the underlying models. The importance of AI safety is increasingly recognized across the industry—organizations such as Anthropic have placed safety research at the core of their mission, reflecting a broader understanding that responsible AI development requires proactive attention to potential harms.

We do not neglect this concern. However, implementing comprehensive content moderation mechanisms is beyond the scope of this project, which focuses primarily on exploring the tech-

nical feasibility of text-to-video generation on consumer hardware. We acknowledge that any production deployment of such a system would need to incorporate appropriate safeguards to prevent the generation of harmful or inappropriate content.

1.6. Chapter Summary

This chapter has provided the foundation for understanding the text-to-video generation task and the objectives of our project. We began by situating text-to-video generation within the broader context of generative AI, tracing the evolution from early neural network architectures through GANs, VAEs, and diffusion models to the sophisticated multimodal systems emerging today.

We then examined the text-to-video generation task in detail, identifying its core objectives—semantic alignment, visual quality, and temporal coherence— and the key challenges that make this task particularly demanding: semantic interpretation of prompts, the high dimensionality of video data, maintaining temporal consistency, modeling realistic motion, and the scarcity of suitable training data for research groups outside of frontier labs. We surveyed the architectural approaches to addressing these challenges, distinguishing between end-to-end models and pipeline-based systems that extend pre-trained text-to-image models with temporal modeling capabilities.

With this technical background established, we presented the motivation and objectives for our project: to explore the current landscape of text-to-video generation by building a functional system from open-weights models and open-source infrastructure, and to make this technology accessible through a user-friendly web application. We outlined both functional objectives—including the generation pipeline, configurable parameters, progress tracking, user management, and the video gallery—and non-functional objectives concerning performance, scalability, usability, and maintainability.

Our prototype implementation validated the feasibility of the proposed approach on consumer-grade hardware, demonstrating that a pipeline combining AnimateDiff, FILM, and Real-ESRGAN can produce acceptable results within our target constraints. We also acknowledged the risks inherent in our approach, including the limitations of open-weights models compared to commercial systems and the broader challenges of content safety in generative AI.

The following chapters will detail the design and implementation of the complete system, describing the architecture, the technologies employed, and the evaluation of our results.

Chapter 2

Functional scope

2.1. Introduction

2.1.1. Purpose of the chapter

The purpose of this chapter is to present a clear description of the scope of the project. We include the requirements that define the functionality and performance of the text-to-video generation system and analyze the feasibility of the solutions in the context of computational complexity of the problem. The chapter outlines what the system is expected to do, how it should behave under different conditions, and the quality standards it should meet. These requirements provide a foundation for the design of the system, its implementation, and evaluation. We specify both functional and non-functional requirements to provide a clear set of expectations for the users of the application.

2.2. Project scope

This section defines the boundaries of the system implemented in this thesis. Here, we aim to specify the functionalities meant to be included in the prototype and clarify the limitations under which it will operate. We also describe the possible extensions of the system that are outside of the scope of this thesis.

2.2.1. Planned functionality

The text-to-video generation system is meant to be a software application capable of automatically producing short videos from user-provided text prompts. The system should be designed to interpret natural language input, extract key elements such as subjects, actions, and contexts, and generate a video that visually represents the described scene.

Within the scope of the project we intend to develop a web application which will be separated into a SPA web interface and backend servers providing a scalable way to generate videos. To achieve this we plan to implement several groups of functionalities:

- *User registration, authentication, and account management*: These are crucial to distinguish between individual users of the application and provide personalized experience.
- *Submission of text prompts and generation parameters*: Users need a clear and friendly way to submit their text prompts and customize basic generation parameters to suit their needs.

- *Execution of video generation and storage of the results*: A text prompt along with a set of parameters needs to be transformed into a video and saved in the storage.
- *Gallery and management of generated videos*: Users need an intuitive way to view their videos and download them.
- *Web interface integrating backend functionalities*: Implements a user interface for the application delegating the video generation and the rest of the functionalities to the application backend.

The precise description of these planned functionalities is included in the Section 2.4.

2.2.2. Excluded functionality

Certain features and capabilities have been intentionally excluded from the scope of the project to make its development manageable given its time constraints, number of project participants, and hardware limitations. These exclusions ensure that the focus of the project is on demonstrating the feasibility of video generation in practical software implementations rather than on building a system that includes the maximum amount of features possible.

We have identified the following functionality that is available in commercial products providing text-to-video generation, but that we do not intend to include in the scope of our project:

- *Uploading of users' own videos and their management*: The storage and gallery functionality is meant to only respectively store and display the videos that were generated by the application. We do not intend to accept videos uploaded by the users of the application.
- *Organization of videos into folders*: Our intention is to show the user a timeline of their generated videos and we do not want to complicate the application by reimplementing functionality similar to personal cloud storage functionality.
- *Generation of images*: The research problem that we are trying to tackle in this thesis is designing an application for solving the text-to-video generative AI problem. Some commercial text-to-video generation applications bundle a text-to-image generation functionality to be more versatile. We do not wish to pursue this path since it is not the goal of our thesis.
- *Audio generation for videos*: State-of-the-art text-to-video models like Veo 3 or Sora 2 are able to add audio relevant to the videos they generate. This is still very much a research frontier and we have neither the expertise nor enough resources to pursue it.

2.2.3. Optional functionality

While certain features have been excluded from the initial scope, the modular architecture of the system is designed to accommodate future enhancements if additional development time becomes available. These potential extensions would add value to the application without compromising its core functionality:

- *Light social networking features*: A community gallery where users can optionally publish their generated videos for others to view and rate. This would include basic privacy controls allowing users to choose which videos to share publicly and features for browsing and discovering content created by other users.

- *Prompt templates and suggestions*: A library of pre-designed prompt templates categorized by themes such as nature, abstract art, or specific visual styles. This would help users who are unfamiliar with prompt engineering to achieve better generation results.
- *Video editing capabilities*: Basic post-generation editing tools allowing users to trim, crop, or apply simple filters to generated videos without leaving the application.
- *Batch generation*: The ability to queue multiple prompts for sequential generation, enabling users to explore variations of a concept or generate multiple videos overnight.
- *Generation history and analytics*: Detailed statistics about generation parameters and their outcomes, helping users understand which configurations produce their preferred results.
- *Collaborative features*: Functionality allowing multiple users to work together on video generation projects, sharing prompts and parameter configurations within teams.

These extensions represent natural evolution paths for the system that align with user needs while maintaining the focus on the core capability.

2.2.4. Performance limitations

In practical aspects of deep learning one of the central issues is the vast computational resources required to get the model weights (training) and use the model (inference). The generative problem that we are tackling is one of the most demanding since it not only requires for individual frames to be generated (as is done in text-to-image models) but the model also needs to handle temporal dimension. Because of the extra dimension, generating a video requires higher memory bandwidth, more VRAM for storing activations, more storage for saving weights/intermediate data, and possibly multiple GPUs to distribute the workload and get the result in acceptable time. Memory requirements for generating videos using open source, end-to-end architectures currently push the limits of VRAM capacity of consumer GPUs at reasonable prices. This is significantly more severe for GPUs that are not from the RTX xx90 series. Our development hardware includes one GPU with 12 GB of VRAM and another with 8 GB. Accordingly our initial options for testing text-to-video models are limited and we decided to focus our attention on pipeline-based text-to-video models that repurpose text-to-image diffusion models for video generation. If we get access to higher-end GPUs that include much more computing power and VRAM we intend to experiment with open source end-to-end models and compare their outputs to our pipeline-based approach.

2.3. System users and external systems

In this section we will describe different types of users who will interact with the system and other systems that it will depend on.

2.3.1. System users

Our system aims to provide video generation capabilities through an easy-to-use and elegant web-based user interface. It means to be a self-contained system that does not require any administration by providing users with control of the videos they generated and giving them

full control over any modifications or deletions of data stored in their accounts, including the account itself.

Accordingly, the only type of user that we have identified for the system is the *end user*, who interacts directly with the application through a web interface. After registration and authentication, the end user gains complete access to the functionality offered by the system. This includes submitting text prompts for video generation, adjusting generation parameters, downloading the generated video, and viewing the results of previous generations.

The planned design assumes that users have no specialized knowledge of AI models or video processing tools, therefore emphasis will be placed on usability and accessibility of the application for non-technical users.

2.3.2. External systems

The planned system will operate as a self-contained application with all processing tasks and storage managed within the system infrastructure. It will not depend on external systems for user interaction, authentication, or video generation.

The only interactions that will occur with external systems will involve downloading external resources such as weights used for various models used in the video generation process. Specifically, the system will retrieve pre-trained model weights from repositories such as *Hugging Face* and *GitHub*. These requests will be limited to downloading publicly accessible assets using secure HTTP requests. No user data or content generated by the system will be transmitted to these external sources.

Apart from these one-directional data transfers the system will not depend on any third-party APIs or external computational services. In particular, the storage of user data and generated videos will be handled on the servers which host the application.

2.4. Functional requirements

This section specifies the functional requirements of the system. Functional requirements define the expected behavior of the system and describe the services it provides to its users. The requirements are organized around the main functionalities of the application such as user account management, video generation workflow, job management, distributed execution, storage and retrieval of user data and generated content. For each functional area the following subsections describe the specific features and their purpose.

2.4.1. Account management

- FR-1 The system shall allow users to create a new account by providing a unique email address and username, as well as a password.
- FR-2 The system shall prevent the creation of new accounts that use an email address or username already associated with an existing account.
- FR-3 The system shall allow users to recover access to their account by sending a password reset link to the email address registered with the account.
- FR-4 The system shall allow users to log in using their registered credentials.

- FR-5 The system shall allow users to temporarily deactivate their account without deleting its associated data from the system.
- FR-6 The system shall allow users to reactivate their account and return the account status to the state from before the deactivation.
- FR-7 The system shall allow users to permanently delete their account, removing all related data from the system.

These requirements ensure that users can securely manage the entire lifecycle of their accounts, from registration and authentication to temporary deactivation and permanent deletion.

2.4.2. Video generation workflow

- FR-8 The system shall allow users to submit a text prompt describing the content of the desired video.
- FR-9 The system shall allow users to configure generation parameters such as video length, resolution, aspect ratio, and frame rate.
- FR-10 The system shall validate the submitted prompt and parameters before starting video generation.
- FR-11 The system shall initiate video generation process based on the validated parameters provided by the user.
- FR-12 The system shall track the progress of the video generation process and update its status in the database as the job progresses.
- FR-13 The system shall handle failures in the video generation process by reporting errors and preventing corrupted or incomplete outputs from being stored.
- FR-14 The system shall notify the user when video generation is complete and the resulting video is available.
- FR-15 The system shall store the generated video and its metadata for later retrieval.

These requirements define the complete workflow for transforming a user-provided text prompt into a video. They ensure that the system can validate and process user input, perform the generation process reliably, and provide the user with access to the result once it is available.

2.4.3. Gallery and retrieval

- FR-16 The system shall provide users with a gallery view of the videos they generated.
- FR-17 The system shall present the gallery of generated videos showing them in chronological order.
- FR-18 The system shall allow users to view the prompt that generated the video.
- FR-19 The system shall allow users to preview and play the generated videos within the application.

FR-20 The system shall allow users to download generated videos.

FR-21 The system shall allow users to share their generated videos on social media platforms.

FR-22 The system shall allow users to delete generated videos.

These requirements ensure that users can access and manage their generated videos as well as view the text prompt that was used to create them.

2.4.4. Video generation job management

FR-23 The system shall monitor the execution status of each video generation job.

FR-24 The system shall update the stored status of a video generation job when its state changes.

FR-25 The system shall notify the user when a video generation job succeeds or fails.

FR-26 The system shall prevent incomplete or failed jobs from producing user visible outputs except for informing the user that the video generation failed.

These requirements ensure that the system can reliably monitor the video generation process, inform the user about the generation status, and maintain consistency between the video generation job status and videos displayed to the user.

2.4.5. User interface capabilities

FR-27 The system shall provide a web-based graphical user interface that allows users to access all application functions after authentication.

FR-28 The system shall organize the main functions of the application into clearly identifiable sections, including areas for prompt submission, video generation progress, and the gallery of generated videos.

FR-29 The system shall display the current state of user actions, including progress indicators for video generation, and status notifications when the generation is completed.

FR-30 The system shall present error messages in a clear manner, indicating when user input is invalid or when generation fails.

FR-31 The system shall provide users with consistent navigation elements that allow users to move between the major sections of the application.

These requirements specify how the system presents and communicates its functionality to users. They ensure that the application interface is clear, informative, and intuitive to use.

2.5. Non-functional requirements

This section specifies the non-functional requirements of the system. Non-functional requirements define the quality attributes and constraints that govern how the system performs its functions, rather than what it does.

2.5.1. Scalability requirements

- NFR-1 The system architecture shall support horizontal and vertical scaling to accommodate increasing numbers of users and video generation requests.
- NFR-2 The system shall distribute video generation tasks across multiple GPU workers (if available) to optimize resource utilization.

These requirements ensure that the system can grow to meet increasing demand by adding computational resources and distributing workload effectively across available infrastructure.

2.5.2. Reliability requirements

- NFR-3 The system shall implement health checks with automatic restart of failed components within 60 seconds.
- NFR-4 The system shall preserve generation tasks state in case of unexpected shutdowns or restarts.

These requirements ensure that the system remains available and operational, can recover gracefully from failures, and maintains data consistency even during unexpected disruptions.

2.5.3. Usability requirements

- NFR-5 The system shall provide an intuitive interface requiring no more than 5 clicks to initiate video generation.
- NFR-6 Error messages shall be displayed in clear, non-technical language with actionable guidance.
- NFR-7 The system shall provide visual progress indicators during video generation processes.

These requirements ensure that the system is easy to use and communicates issues in an understandable manner.

2.5.4. Maintainability requirements

- NFR-8 The system shall use modular architecture allowing individual components to be updated independently without affecting other parts of the system.
- NFR-9 The system shall provide structured logging with severity levels for troubleshooting and monitoring.
- NFR-10 The system shall be deployable on Docker containers.
- NFR-11 The system shall maintain comprehensive API documentation for all endpoints.

These requirements ensure that the system can be easily updated, debugged, and maintained over time without disrupting service availability.

2.5.5. Resource constraints

NFR-12 Individual video generation tasks shall operate within 12GB VRAM limits for GPU processing.

NFR-13 Individual video generation tasks shall operate within 16GB RAM limits for CPU processing.

These requirements ensure that the system operates within defined hardware and storage limitations, preventing resource exhaustion and maintaining system stability.

2.6. Usage and testing scenarios

This section presents the main user interaction scenarios that describe how the system is expected to be used in real conditions. Each scenario defines the sequence of steps performed by the user and the system's corresponding behavior, forming a foundation for acceptance and integration testing. They also define the basis for the testing phase, as each scenario corresponds to one or more test cases verifying its successful execution.

The structure of these scenarios follows the *use case template* defined in the UML (Unified Modeling Language) methodology and compliant with the IEEE 830 and ISO/IEC/IEEE 29148 standards for software requirement documentation. This approach provides a clear link between system functionality, user interactions and validation procedures.

Every scenario includes:

- **Preconditions** - assumptions that must hold before the scenario begins,
- **Scenario** - step-by-step interaction between the user and the system,
- **Postconditions** - the resulting state of the system or user data,
- **Testing goal** - the expected outcome used as a reference during validation.

2.6.1. Generating videos

U-1: User views generation parameters

Preconditions: The user has opened the video generation window.

Scenario:

1. The user opens the video generation interface.
2. The system displays all available parameters for the generation process.
3. The user can hover over or select any parameter to view an explanatory tooltip or text description.
4. Each parameter is described in a clear and understandable way, allowing users unfamiliar with the generation process to configure them correctly.

Postconditions: Parameters and their descriptions are visible and accessible to the user within the interface.

Testing goal: Verify that all generation parameters have visible, understandable descriptions and users can access this information easily before starting the generation process.

U-2: The user generates and manages a new video

Preconditions: The user is logged into the system.

Scenario:

1. The user navigates to the “Create Video“ page.
2. The user enters a text prompt and other generation parameters.
3. The system validates the input and starts the generation process.
4. A progress indicator is displayed, showing each stage of generation.
5. Once the process is completed, the system notifies the user and displays the generated video in a preview window.
6. The user can view or download the generated video.
7. The user can share the video on social media platforms.
8. The generated video becomes available in the user’s personal gallery and remains visible after re-logging into the system.

Postconditions: The generated video is stored in the user’s account and remains available for future access, management and sharing.

Testing goal: Verify that a logged-in user can generate, preview, download and share a video on social media platforms and that the video persists in their personal gallery.

U-3: User generates multiple videos simultaneously

Preconditions: The user is logged in and has access to the video generation interface and can initiate multiple generation tasks.

Scenario:

1. The user starts generating a video using a selected prompt and parameters.
2. While the first generation is in progress, the system displays a badge or indicator showing the active generation status.
3. The user initiates one or more additional video generations in parallel.
4. The system queues or processes multiple generation tasks concurrently without reducing performance or responsiveness of the interface.
5. Once the processes complete, each generated video becomes available for preview or download.

Postconditions: Multiple videos are successfully generated in parallel without affecting the application’s responsiveness or causing backend issues.

Testing goal: Verify that users can start and manage multiple video generations at once and that the system handles concurrent requests without performance degradation or errors.

U-4: System handles concurrent generation from multiple users

Preconditions: Multiple users are simultaneously generating videos through the application interface.

Scenario:

1. Several users initiate independent video generation processes at the same time.
2. The backend architecture distributes tasks efficiently among available resources.
3. The system maintains stable performance and ensures isolation between user sessions.
4. Each user receives progress feedback for their individual generation tasks.
5. All requested videos are generated successfully without failures or slowdowns.

Postconditions: The system remains stable and responsive under concurrent load from multiple users.

Testing goal: Verify that the application's architecture can handle simultaneous generation requests from many users without crashes, bottlenecks, or performance loss.

2.6.2. Managing videos**U-5: User shares a video**

Preconditions: The user is logged in and has at least one generated video in their gallery.

Scenario:

1. The user opens their gallery and selects a generated video.
2. The system displays sharing options available for the selected video.
3. The user selects a desired social media platform for sharing.
4. The system initiates the sharing process through the platform's integration interface.
5. The system publishes the video on the selected social media platform.

Postconditions: The selected video is successfully published on the chosen social media platform and visible on the user's profile. **Testing goal:** Verify that a logged-in user can share a generated video directly from the application to supported social media platforms and that the video appears correctly on the selected profile.

U-6: User deletes a video

Preconditions: The user is logged in and has at least one video available in their gallery.

Scenario:

1. The user navigates to their video gallery.
2. The system displays all existing videos.
3. The user selects a video to remove.
4. The system asks for confirmation before deletion.

5. After confirmation, the selected item is permanently deleted.
6. The gallery view refreshes and no longer displays the deleted item.

Postconditions: The deleted video is permanently removed from the user's account and is no longer visible or accessible in any part of the application.

Testing goal: Verify that users can permanently delete videos from their collection and that the removed items are no longer visible or recoverable within the application.

2.6.3. Account management

U-7: User registers a new account

Preconditions: The user is not logged in and has access to the registration form.

Scenario:

1. The user navigates to the registration page.
2. The user provides a valid e-mail address and password, then submits the form.
3. The system sends a verification e-mail containing an activation link.
4. The user opens the received e-mail and clicks the activation link.
5. The system verifies the link and activates the new account.
6. The user is notified that registration has been successfully completed and can now log in.

Postconditions: A verified account is created in the system, allowing the user to log in and access all authenticated functionalities.

Testing goal: Verify that a new user can register with valid credentials, receive a verification e-mail and activate the account successfully.

U-8: User logs into the system

Preconditions: The user has a verified account.

Scenario:

1. The user navigates to the login page.
2. The user enters valid login credentials (e-mail and password).
3. The user optionally selects the "Remember me" option to stay logged in across sessions.
4. The system validates the credentials and grants access to the application.
5. The user is redirected to the main interface with access to their personal gallery and profile options.

Postconditions: The user is authenticated and has access to all functionalities available to logged-in users.

Testing goal: Verify that users can log in successfully using valid credentials and remain logged in if the "Remember me" option is selected.

U-9: User resets the account password

Preconditions: The user has a registered account and access to their e-mail address.

Scenario:

1. The user opens the password reset page and enters their e-mail address.
2. The system sends a password reset e-mail containing a secure link.
3. The user opens the e-mail and follows the link to the password reset form.
4. The user provides a new password and submits the form.
5. The system updates the credentials and confirms that the password has been changed successfully.

Postconditions: The user's password is updated and they can log in using the new credentials.

Testing goal: Verify that users can reset their password through the e-mail-based recovery process and regain access to their account.

U-10: User permanently deletes account with all data

Preconditions: The user is logged in and has an active account containing videos and other personal data.

Scenario:

1. The user navigates to the account settings section.
2. The user selects the option to permanently delete the account.
3. The system displays a confirmation prompt informing that all data, including videos and settings, will be irreversibly removed.
4. The user confirms the action.
5. The system deletes the user account and all related data from the database.
6. The session is terminated and the user is redirected to the homepage with a message confirming successful deletion.

Postconditions: The user account and all associated data are permanently removed from the system. The user can no longer log in with the deleted credentials.

Testing goal: Verify that the system completely removes the user's account and all dependent data and that access is fully revoked after deletion.

U-11: User deactivates and reactivates account

Preconditions: The user is logged in and has an active account.

Scenario:

1. The user navigates to the account settings section.
2. The user selects the option to temporarily deactivate their account.

3. The system displays information about the consequences of deactivation, such as hidden videos and disabled sharing.
4. The user confirms deactivation.
5. The system updates the account status to “deactivated” and logs the user out.
6. The user later returns to the login page and provides their credentials.
7. The system detects the deactivated status and displays an option to reactivate the account.
8. The user confirms reactivation.
9. The system restores the account and all related data to the state before deactivation.

Postconditions: The account is successfully deactivated and later reactivated with all user data preserved.

Testing goal: Verify that the system correctly handles account deactivation and reactivation, preserving user data and restoring access after reactivation.

The presented testing scenarios cover all essential functionalities of the system, ensuring that each user interaction path can be verified. They provide a structured basis for assessing the correctness of system behavior and allow for effective evaluation of its overall performance and responsiveness.

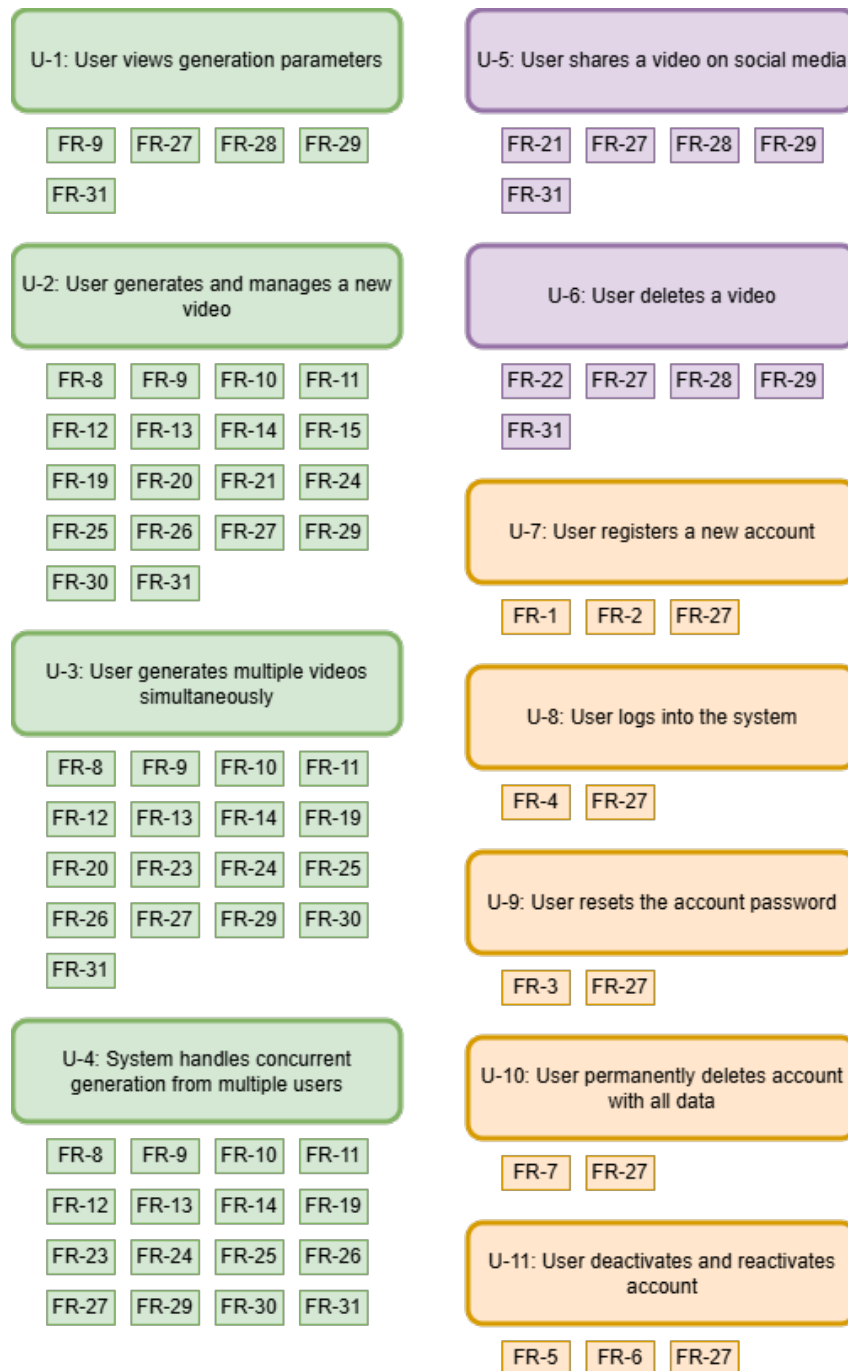


Figure 2.1: Mapping between user test scenarios and functional requirements

2.7. Summary

This chapter provided a comprehensive specification of the system requirements and usage scenarios for the text-to-video generation application. It established the foundation for the subsequent design, implementation and testing phases by defining both the functional and non-functional aspects of the system. The presented requirements describe what the system should do, how it should perform and under what constraints it will operate. The usage scenarios illustrated typical user interactions, forming a basis for acceptance testing and validation of the system's behavior in real-world conditions.

Chapter 3

Selected realization aspects

3.1. Introduction

Video generation systems must address unique challenges such as managing computationally expensive AI models and scaling to handle multiple concurrent requests. These systems operate under strict GPU resource constraints and must remain responsive despite long-running computations. As a result, the overall architecture must balance performance, reliability and real-time communication between its components.

In this chapter we describe the main components, their responsibilities and the communication patterns between them. The system is composed of a web interface, an API layer coordinating requests, a distributed generation pipeline, a relational database and object storage.

The chapter also outlines how the architecture addresses key challenges such as managing long and expensive GPU computations, providing real-time feedback to users and maintaining system scalability under varying loads.

3.2. Architecture Overview

At the highest level, the architecture is organized into five components that handle different aspects of the video generation workflow:

- **User Interface (Next.js):** Web application for user interaction with the system
- **API Server (FastAPI):** Authentication, job submission, status queries
- **Generation Pipeline (Ray Serve):** System core - distributed video generation
- **Database (PostgreSQL):** User accounts, job metadata
- **Object Storage (MinIO):** Video files

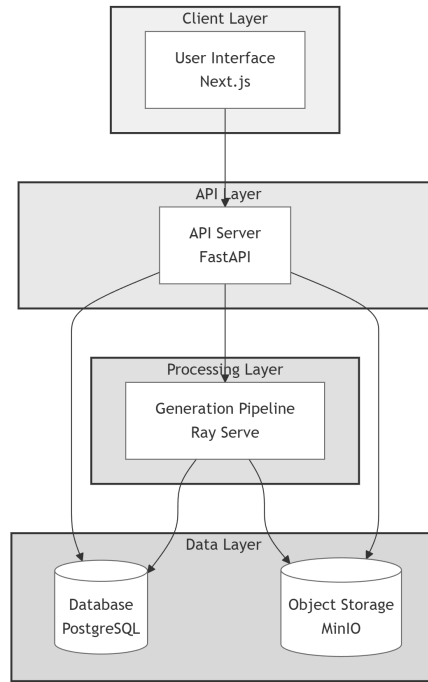


Figure 3.1: High-level system architecture.

3.3. Detailed Architecture

3.3.1. User Interface

User interface is implemented as a modern and futuristic web application built with Next.js and styled using Tailwind CSS. This choice provides a combination of performance-focused optimizations and a flexible component-based architecture. Access to the interface is restricted to authenticated users, with all API communication secured through Bearer tokens issued by the backend. Once authenticated, the user is presented with a clear and structured dashboard that provides an organized, paginated view of all previously generated videos.

In the dashboard, users can filter, sort and browse their generated content. Each generated video can be viewed or downloaded in a chosen output format.

The interface includes a dedicated view where users can create new video generation jobs. Users can choose between two modes of operation: an advanced mode offering full control over model parameters and a simplified mode intended for users less familiar with diffusion models or the application itself. The advanced mode exposes a wide range of configuration options, allowing experienced users to fine-tune all aspects of the generation process and adjust the model's behaviour to their preferences. The simplified mode, in contrast, presents a compact and streamlined form that focuses solely on the key inputs required to start a generation task. Both modes provide clear, contextual instructions that help users understand the available options and make effective use of the system.

To enhance interactivity, the interface maintains a WebSocket connection with the backend. Channel provides up-to-date information about each running job, including its current state and progress. Users can monitor the generation with minimal delay and may cancel an active task directly from the interface.



Figure 3.2: Application logo displayed in the dashboard.

3.3.2. API Server

The FastAPI backend serves as the entry point for all client requests, implementing a RESTful interface that coordinates interactions between system layers.

Authentication is implemented using the JWT tokens. After successful login, the system creates access token used for API authentication. The user password is hashed and stored in the database, providing the high level of security. Authentication failures return an HTTP 401 status code.

The API Server performs request validation using Pydantic schemas - each incoming request is validated against type-safe data models that enforce constraints on required fields, types and value ranges. This early validation prevents malformed data from reaching business logic or the database. Invalid requests are rejected immediately with HTTP 4xx status codes and descriptive error messages indicating the specific validation failure.

The Server implements ownership-based access control, ensuring users can only access their own jobs and videos. Each authenticated request includes the user ID extracted from the JWT token, which is validated against the job owner ID stored in the database. Attempts to access other users' resources result in HTTP 403 Forbidden responses.

API Server is stateless, storing no session information in memory. All session state is maintained through JWT tokens and database records. This design enables horizontal scaling by deploying multiple instances behind a load balancer without requiring session affinity or shared session storage. The Server uses SQLAlchemy connection pooling to efficiently manage database connections across concurrent requests, preventing connection exhaustion under heavy load.

3.3.3. Video Generation Pipeline

The core pipeline is designed as a composition of multiple independent stages, each deployed as a separate Ray Serve deployment. This modular architecture addresses the varying computational demands of different processing stages, enables independent scaling, efficient resource utilisation and enables component-level replacements without system-wide code changes, as well as adding new pipeline stages with minimal system changes.

Ray Serve provides autoscaling capabilities that allow each component to scale independently based on load and resources available. Unlike monolithic approaches where the entire

pipeline must scale as one unit, this design allows CPU-intensive stages (preprocessing and postprocessing) to scale independently from GPU-intensive stages (generation, interpolation and upscaling). This separation is crucial because GPU resources are typically more constrained and more expensive than CPU resources.

The pipeline consists of five components:

- **Preprocessor:** Optimize parameters for downstream components
- **Video Generator:** Create base frames using AnimateDiff
- **Frame Interpolator:** Increase frame rate using FILM model
- **Frame Upscaler:** Upscale frames using ESRGAN
- **Postprocessor:** Trim, crop and encode final video

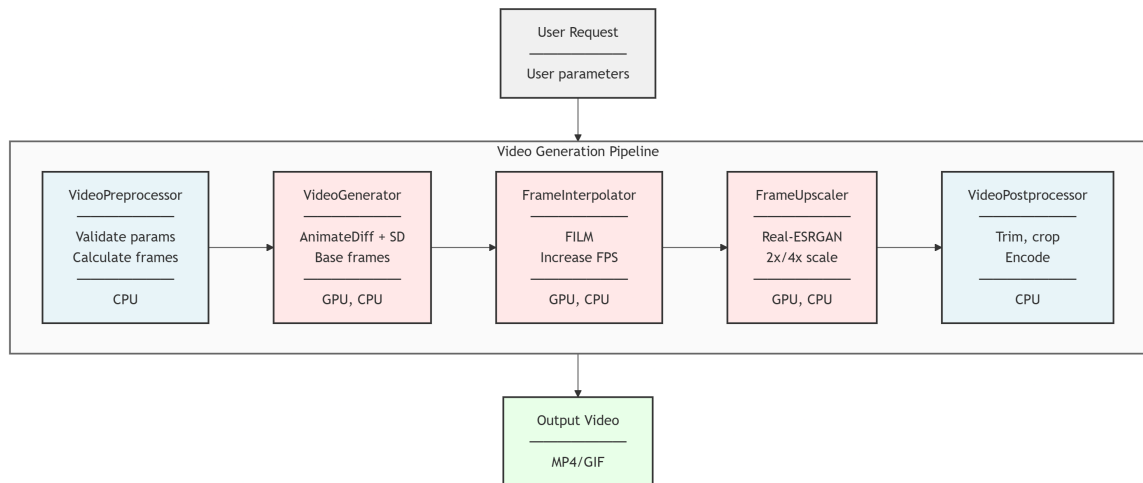


Figure 3.3: Video generation pipeline architecture.

Preprocessor

The **Preprocessor** serves as the pipeline's validation and configuration stage, ensuring all input parameters meet the technical constraints imposed by downstream components. It ensures that video dimensions are multiples of 8, adjusting them upward if necessary. This requirement stems from the Stable Diffusion VAE's architecture which performs downsampling in 8x8 pixel blocks - non-compliant dimensions would cause encoding failures in the diffusion model. Because AnimateDiff generates base frames at a fixed 8 FPS and reduced resolution to minimize VRAM usage and processing time - the Preprocessor determines how many base frames are needed for the requested duration, then calculates the interpolation factor required to reach the target FPS in later stages. Similarly, it computes the downscaling factor for base generation, ensuring the VideoGenerator operates within hardware constraints while the Frame Upscaler

later restores the desired resolution. For example, a request for 1024x1024 at 24 FPS might be configured as 512x512 base generation at 8 FPS, followed by 2x upscaling and 3x frame interpolation.

Video Generator

The **Video Generator** is the pipeline's most computationally intensive component, responsible for creating the base frame sequence from the text prompt. It implements AnimateDiff with Stable Diffusion as the underlying architecture, applying iterative denoising in the VAE latent space to generate temporally coherent frames.

The generator operates under strict VRAM constraints imposed by our development environment (NVIDIA RTX 4070 Super with 12GB VRAM).

Generating high-resolution videos directly is infeasible - memory usage exhibits asymptotically quadratic growth with frame resolution, as both the latent representation size and attention map dimensions scale with the square of pixel count. For example, doubling the resolution from 512x512 to 1024x1024 increases pixel count by 4x and VRAM requirements by approximately 3-4x due to attention mechanism overhead. Therefore, the generator produces frames at reduced resolution, delegating upscaling to the specialized Frame Upscaler component.

Similarly, generating at high frame rates would multiply VRAM requirements linearly. Therefore, generator operates at a fixed 8 FPS, producing smooth motion at this base rate, while the Frame Interpolator later synthesizes intermediate frames to reach target frame rates, multiples of 8.

Generation typically takes 60-240 seconds per job depending on resolution and frame count.

Frame Interpolator

The **Frame Interpolator** increases frame rate by synthesizing intermediate frames between AnimateDiff's base frames.

The implementation uses FILM (Frame Interpolation for Large Motion), chosen for its superior handling of complex motion compared to traditional optical flow methods like RIFE. FILM employs a multi-scale feature pyramid that jointly estimates optical flow and interpolation weights, producing smoother motion with fewer artifacts.

The interpolation factor is determined by the Preprocessor based on the ratio between target FPS and AnimateDiff's base rate of 8 FPS. For example, requesting 24 FPS requires 3x interpolation, inserting 2 synthetic frames between each pair of base frames. Supported target frame rates are limited to multiples of 8 (14, 24, 32 FPS) to ensure integer interpolation factors.

Interpolation is optional - if the target FPS matches the base 8 FPS, this stage is skipped entirely.

Processing typically takes 15-30 seconds per job, depending on the number of frames and interpolation factor.

Frame Upscaler

The **Frame Upscaler** enhances resolution using super-resolution neural network.

The implementation uses Real-ESRGAN (Enhanced Super-Resolution Generative Adversarial Network), chosen for its robustness to AI-generated image artifacts compared to alternatives like EDSR.

The upscaling factor is determined by the Preprocessor based on the ratio between target resolution and maximum base generation resolution. For example, requesting 1024x1024 output from 512x512 base frames requires 2x upscaling.

Upscaling is optional - if the target resolution matches the base generation resolution, this stage is skipped entirely.

Processing typically takes 20-40 seconds per job, depending on the number of frames and upscaling factor.

Postprocessor

The **Postprocessor** finalizes the video by performing trimming, cropping, and encoding operations to produce the output file.

After generation, interpolation and upscaling, the frame sequence may exceed the exact target duration or dimensions. The Postprocessor trims frames to achieve precise duration, and crops them to match the user's specified resolution. The implementation supports multiple output formats:

- MP4
- GIF
- WebM

The component processes the final frame sequence, encoding it to the target format and uploading the result to MinIO object storage.

Processing typically takes 10-30 seconds per job, depending on the number of frames, output format and encoding parameters.

3.3.4. Data Persistence

Database

The system uses PostgreSQL as the relational database for storing structured metadata. The schema consists of four main tables:

- **User**: account information
- **VideoGenerationJob**: job status and progress tracking
- **VideoGenerationJobParameters**: generation parameters
- **VideoGenerationJobResult**: output metadata

SQLAlchemy serves as the ORM layer, providing a Python interface to database operations. Connection pooling prevents connection exhaustion under heavy load.

Storage

The system uses MinIO for object storage. MinIO provides S3-compatible APIs for storing and retrieving large binary files, making it suitable for handling generated video content.

The dual-storage architecture separates concerns and optimizes performance. PostgreSQL excels at indexed queries and relational integrity, while MinIO handles large binary files efficiently. This separation also enables independent scaling - database capacity can grow through read replicas, while storage capacity scales through MinIO's distributed mode.

3.4. Technological Stack

The system is built using a modern technology stack selected to address the specific challenges of video generation workloads while maintaining developer productivity and system maintainability. The stack combines established frameworks with specialized libraries for machine learning and distributed computing. Technology choices prioritize three key criteria: efficient GPU resource utilization for computationally intensive video generation, robust support for asynchronous operations to handle long-running tasks, and proven scalability patterns for managing concurrent user requests. This section describes the core technologies employed across the system architecture, explaining their roles and the rationale behind their selection.

3.4.1. Backend Technologies

The backend infrastructure is built on Python and FastAPI, providing a robust foundation for implementing REST APIs and managing asynchronous operations.

Python

Python serves as the primary programming language for the backend, selected for its dominant position in the machine learning ecosystem and extensive library support for scientific computing. The language's first-class support for asynchronous programming through `async/await` syntax enables efficient handling of I/O-bound operations without blocking execution threads. Python's dynamic typing combined with type hints allows for rapid development while maintaining code clarity through static code analysis tools.

The rich ecosystem of Python libraries provides ready-made solutions for common tasks: SQLAlchemy for database operations, Pydantic for data validation, and PyJWT for token-based authentication. This extensive library support accelerates development by reducing the need for custom implementations of standard functionality.

FastAPI

FastAPI was chosen as the web framework for implementing the RESTful API server due to its modern design and performance characteristics. The framework provides automatic API documentation generation through OpenAPI (Swagger) and JSON Schema, creating interactive documentation that simplifies API exploration and testing during development.

FastAPI's native support for asynchronous request handling enables the server to efficiently manage concurrent connections without spawning multiple threads or processes. When handling long-running operations such as video generation job submissions, the framework can yield control while waiting for I/O operations, allowing other requests to be processed concurrently on the same thread.

The framework integrates tightly with Pydantic, leveraging type annotations for automatic request validation and serialization. API endpoints define expected request and response schemas using Python type hints, and FastAPI automatically validates incoming data against these schemas before the request handler executes. This approach eliminates boilerplate validation code and ensures type safety throughout the request lifecycle.

Dependency injection is a core feature of FastAPI, used extensively for managing cross-cutting concerns such as database sessions and user authentication. Dependencies are declared using function parameters, and the framework automatically resolves and injects them when handling requests. This pattern is particularly valuable for authentication, where a dependency extracts and validates JWT tokens, making the authenticated user object available to endpoint handlers.

FastAPI's performance characteristics align well with the system's requirements. Built on Starlette and using uvicorn as the ASGI server, the framework achieves request handling speeds comparable to Node.js and Go frameworks while maintaining Python's development productivity advantages.

3.4.2. Frontend Technologies

The user interface is implemented using Next.js and TypeScript, providing a modern, type-safe foundation for building a responsive web application.

Next.js

Next.js serves as the primary framework for the frontend, providing a robust foundation for building a responsive and highly optimized web application. Its architecture combines the flexibility of React with an advanced tooling ecosystem that simplifies development while ensuring strong performance characteristics. The framework offers server-side rendering (SSR) and static site generation (SSG), both of which contribute to reduced load times and improved perceived responsiveness. SSR enables the server to pre-render views before they are sent to the browser, improving initial load performance, while SSG allows pages with static content to be generated at build time and served instantly. Additionally, Next.js automatically optimizes assets by splitting bundles and delivering only the resources required for each view. Next.js also includes automatic asset optimization and bundle splitting to deliver only the resources required for each view. As part of this optimization toolchain, the `next/image` component provides built-in responsive image handling, automatically generating appropriate sizes and formats based on the user's device and improving loading performance.

Next.js includes a built-in router that maps the directory structure directly to application routes, removing the need for manual configuration and keeping navigation predictable as the project grows. The reactive nature of React enables the interface to update efficiently in response to user interactions and backend events, while Zustand provides lightweight global state management. For all asynchronous data operations, particularly those involving API communication, the application uses TanStack Query, which handles fetching, caching, and synchronization of data, ensuring consistent and up-to-date information across the interface.

Next.js also supports internationalization (i18n), which is used by the system to provide both Polish and English versions of the interface. Built-in routing with locale detection enables seamless transitions between languages without duplicating components or logic. Architecture allows new languages to be added easily by supplying an additional JSON language package.

TypeScript

TypeScript was adopted as the primary language for frontend development to provide static type checking and improved developer experience. Type annotations catch errors at compile time rather than runtime, reducing bugs related to incorrect data types, undefined values or accessing non-existent object properties.

The type system is particularly valuable when working with API responses from the FastAPI backend. TypeScript interfaces define the expected structure of data returned from each endpoint, and the compiler verifies that components handle this data correctly.

TypeScript's integration with modern IDEs provides intelligent code completion and inline documentation. When working with complex objects such as video generation parameters or job status responses, the editor can suggest available properties and their types, reducing the need to constantly reference API documentation.

The gradual typing system allows the codebase to balance strictness with pragmatism - critical business logic benefits from comprehensive type coverage, while less critical utility functions can use more permissive types when appropriate. This flexibility accelerates development without sacrificing the safety benefits that motivated TypeScript adoption. In addition, the presence of static types significantly simplifies long-term maintenance and enables the project to scale more easily as new features and modules are introduced.

Tailwind CSS

Tailwind CSS is employed as the styling framework for the user interface. Its utility-first approach enables rapid development of consistent and responsive layouts without the need to write custom CSS for each component. Tailwind accelerates the design process, reduces stylesheet complexity and ensures uniform visual structure across the entire application. Small portions of component-specific styling were implemented using SCSS to complement Tailwind's utilities.

3.4.3. Infrastructure Technologies

The infrastructure layer provides the foundational services for data persistence, object storage, and distributed computing, enabling the system to handle GPU-intensive workloads efficiently.

Ray Serve

Ray Serve serves as the distributed serving framework for the video generation pipeline, chosen specifically for its ability to manage GPU resources and scale machine learning workloads across multiple nodes. The framework builds on Ray's distributed computing capabilities, providing model serving abstractions optimized for deep learning inference.

Ray Serve's deployment model allows the generation pipeline to be deployed as a collection of independent replicas, each capable of handling video generation requests. The framework automatically distributes incoming requests across available replicas using configurable routing strategies, ensuring efficient utilization of GPU resources. When multiple users submit generation jobs simultaneously, Ray Serve's scheduler assigns each request to an available replica, preventing resource contention and maintaining predictable response times.

The framework provides built-in support for autoscaling based on request queue depth and resource utilization. As the number of pending generation jobs increases, Ray Serve can automatically spawn additional replicas on available GPU nodes, expanding capacity to meet de-

mand. Conversely, during periods of low activity, the framework can scale down replicas to conserve resources.

Ray Serve integrates seamlessly with PyTorch and other machine learning frameworks, allowing the generation pipeline to load models directly into GPU memory and maintain them across multiple requests. This persistent model loading can eliminate the overhead of repeatedly loading multi-gigabyte model weights for each generation request, significantly improving throughput.

The framework's fault tolerance mechanisms ensure system resilience in the face of hardware failures. If a GPU node becomes unavailable, Ray automatically detects the failure and redistributes pending requests to healthy replicas. Failed replicas are restarted on available nodes, restoring full system capacity without manual intervention.

PostgreSQL

PostgreSQL serves as the primary relational database, storing user accounts, job metadata, and authentication tokens. The database was selected for its proven reliability, ACID compliance, and rich feature set for handling complex queries and ensuring data integrity.

The database schema leverages PostgreSQL's support for foreign key constraints to maintain referential integrity between related entities. User accounts, generation jobs, and refresh tokens are linked through foreign key relationships, ensuring that orphaned records cannot exist in the database. When a user account is deleted, CASCADE constraints automatically remove associated jobs and tokens, maintaining database consistency.

PostgreSQL's transactional capabilities ensure that complex operations involving multiple tables complete atomically. When creating a new generation job, the system inserts records into both the jobs table and the job parameters table within a single transaction. If any part of the operation fails, the entire transaction rolls back, preventing partial state from persisting in the database.

The database provides indexing capabilities that optimize query performance for common access patterns. Indexes on user IDs and job status fields accelerate queries that retrieve a user's jobs or filter jobs by their current state.

PostgreSQL's connection pooling integration through SQLAlchemy ensures efficient management of database connections. Rather than opening a new connection for each API request, the application maintains a pool of persistent connections that are reused across requests. This approach reduces connection establishment overhead and prevents connection exhaustion under high load.

MinIO

MinIO provides S3-compatible object storage for generated video files, chosen for its simplicity, performance, and compatibility with industry-standard APIs. The storage service runs as a containerized application, offering a lightweight alternative to managed cloud storage services while maintaining the same programming interface.

The object storage architecture separates video file storage from the database, avoiding the performance and scalability issues associated with storing large binary objects in relational databases. When a video generation job completes, the pipeline writes the output file directly to MinIO and stores only the object key in the PostgreSQL database. This separation allows the database to remain optimized for metadata queries while MinIO handles the high-throughput demands of video file storage and retrieval.

The S3-compatible API enables the application to interact with MinIO using the minio Python client library. This compatibility provides flexibility for future migrations to managed cloud storage services like Amazon S3 or Google Cloud Storage without requiring significant code changes - primarily endpoint URL and credentials configuration. Additionally, the containerized nature of MinIO simplifies deployment across different cloud platforms, allowing the storage layer to be deployed on Azure Kubernetes Service (AKS) or Google Kubernetes Engine (GKE) alongside other system components while maintaining consistent S3-compatible interfaces.

3.4.4. Machine Learning Libraries

The video generation capabilities are built on PyTorch and specialized libraries from the Hugging Face ecosystem, leveraging pre-trained models for high-quality video generation.

PyTorch

PyTorch serves as the foundational deep learning framework for the video generation pipeline, chosen for its flexibility, strong GPU acceleration support, and widespread adoption in the research community. The framework provides tensor operations optimized for GPU execution, enabling efficient processing of the high-dimensional data involved in video generation.

PyTorch's dynamic computation graph allows the generation pipeline to adapt model behavior at runtime based on input parameters. Unlike static graph frameworks, PyTorch constructs the computation graph during forward passes, enabling conditional logic and variable-length sequences without pre-compilation. This flexibility is particularly valuable for video generation, where output dimensions and processing steps may vary based on user-specified parameters like frame count and resolution.

PyTorch's CUDA integration provides seamless access to GPU acceleration without requiring low-level CUDA programming. Tensors and models can be moved to GPU memory with simple method calls, and subsequent operations automatically execute on the GPU. This abstraction allows the pipeline to leverage GPU computational power while maintaining code readability.

Diffusers

The Diffusers library from Hugging Face provides high-level abstractions for working with diffusion-based generative models. The library handles the complex implementation details of the diffusion process, allowing the system to focus on pipeline configuration and parameter tuning rather than low-level model operations.

Diffusers offers pre-built pipeline implementations that encapsulate the entire generation workflow. The system loads these pipelines by specifying a model identifier from Hugging Face's model hub, and the library handles all aspects of model initialization, including downloading weights, configuring schedulers, and setting up the inference process. This abstraction eliminates the need to manually manage individual model components or implement the denoising algorithms.

The library's model hub integration enables direct loading of pre-trained models from Hugging Face repositories. Models are downloaded and cached locally on first use, then loaded from disk for subsequent generation requests. This caching mechanism eliminates redundant downloads and enables offline operation once models are cached.

Diffusers provides a consistent API across different model architectures and generation approaches. Whether using AnimateDiff for video generation or standard Stable Diffusion variants, the interaction pattern remains similar - instantiate a pipeline with a model identifier, configure generation parameters, and invoke the generation method. This consistency simplifies experimentation with different models and techniques.

The library supports various optimization techniques that improve generation speed and reduce memory requirements. Half-precision (FP16) computation reduces memory utilisation and accelerates tensor operations on compatible GPUs. These optimizations can be enabled through simple configuration parameters when initializing pipelines, without requiring manual tensor type conversions or custom CUDA kernels.

3.4.5. Pre-trained Models

The system employs a pipeline-based approach to video generation that leverages multiple pre-trained models working together, rather than relying on a single end-to-end video generation model. This modular architecture combines text-to-image diffusion models with motion modules and both interpolation and upscaling techniques to synthesize videos from text prompts.

Stable Diffusion Models

The pipeline utilizes **text-to-image diffusion models** as the foundation for visual content generation. These models, trained on large-scale image datasets, understand the relationship between textual descriptions and visual concepts. By applying these models iteratively across video frames while maintaining temporal coherence through AnimateDiff's motion modules, the system generates videos that align with the input text prompt while exhibiting smooth motion.

AnimateDiff

AnimateDiff serves as the core technique for introducing temporal consistency to still images generated by diffusion models. AnimateDiff extends pre-trained text-to-image models by injecting motion modules that learn temporal dependencies between frames. These motion modules are trained separately and can be combined with various base image generation models, providing flexibility in choosing the visual style and quality characteristics of the generated videos.

FILM Interpolator

FILM improves quality by increasing the frame rate of generated videos. After the diffusion process generates base frames, interpolation model synthesizes intermediate frames to achieve higher frame rates and smoother motion. This approach reduces the computational cost of generating every frame through the full diffusion process while maintaining visual quality.

Real-ESRGAN Upscaler

Real-ESRGAN enhances resolution of generated videos. Since generating high-resolution frames directly through diffusion is computationally- and memory-intensive, the system generates frames at lower resolution and then applies upscaling. Real-ESRGAN, trained on degraded

real-world images, performs resolution enhancement while preserving temporal consistency and preserves the details that would be lost in traditional interpolation methods.

All models are sourced from open repositories on Hugging Face and GitHub, eliminating the need for expensive model training infrastructure. Models are identified by their repository names and loaded dynamically based on configuration parameters, allowing the system to support different model combinations for various use cases:

- **Base image models:** Stable Diffusion variants and other text-to-image models that determine the visual style
- **Motion modules:** AnimateDiff motion adapters trained for different motion characteristics
- **LoRA adapters:** Low-Rank Adaptation modules that fine-tune the base models for specific visual styles or subjects

Model weights are stored separately from application code and cached locally after first download.

3.4.6. Supporting Tools and Libraries

Beyond the core technology stack, several supporting tools facilitate development, deployment, and system integration.

Docker and Docker Compose

Docker containerization provides consistent deployment environments across development and production systems. Each system component - Ray Serve cluster, PostgreSQL database, and MinIO storage - runs in isolated containers with explicitly defined dependencies and configurations.

Containerization addresses the "works on my machine" problem by packaging applications with their entire runtime environment. Python dependencies, system libraries, and configuration files are bundled into immutable container images, ensuring that the application behaves identically regardless of the host system. This consistency is particularly valuable for the video generation pipeline, which depends on specific versions of CUDA drivers, PyTorch, and diffusion model libraries that can be difficult to configure manually.

Docker Compose orchestrates multi-container deployments for local development and testing environments. A single `docker-compose.yml` file defines all system components, their network connections, volume mounts, and environment variables. Developers can launch the entire system stack with a single command, eliminating the need to manually start and configure each service.

The compose configuration establishes networks that enable inter-container communication while isolating the system from the host network. The FastAPI backend communicates with PostgreSQL and MinIO using service names as hostnames, with Docker's internal DNS resolving these names to container IP addresses. This approach simplifies configuration management - connection strings reference service names rather than hardcoded IP addresses that might change between deployments.

Volume mounts persist data across container restarts, ensuring that database contents, cached model weights, and generated videos survive container lifecycle events. Named volumes store PostgreSQL data and MinIO objects, while bind mounts during development allow code changes to be immediately reflected inside containers without rebuilding images.

GPU access within Docker containers is enabled through NVIDIA Container Toolkit, which provides the necessary runtime components to expose host GPU devices to containerized applications. The toolkit allows containers to access CUDA libraries and GPU hardware by mounting the required device files and driver libraries from the host system. Docker Compose configurations specify GPU resource requirements using the `deploy.resources.reservations.devices` syntax, declaring which GPUs should be accessible to each container. The Ray Serve container receives GPU access and internally manages the distribution of GPU resources across multiple pipeline deployment replicas, with Ray's scheduler handling the allocation of specific GPUs to individual generation requests. This architecture allows a single Ray Serve container to efficiently orchestrate multiple concurrent video generation jobs across available GPU resources.

SQLAlchemy

SQLAlchemy serves as the Object-Relational Mapping (ORM) layer between the Python application code and PostgreSQL database. The library provides a high-level abstraction that allows the application to interact with database records as Python objects rather than writing raw SQL queries.

Database tables are represented as Python classes, with table columns defined as class attributes. Relationships between tables - such as the one-to-many relationship between users and generation jobs - are expressed through SQLAlchemy relationship definitions that automatically handle foreign key constraints and join operations.

The ORM generates SQL queries automatically based on Python operations, reducing the likelihood of SQL injection vulnerabilities and syntax errors. When the application queries for a user's jobs, SQLAlchemy constructs the appropriate SELECT statement with WHERE clauses and JOIN operations, parameterizing user-supplied values to prevent injection attacks.

SQLAlchemy's session management provides transaction boundaries and change tracking. Operations on database objects are grouped into sessions, with changes automatically flushed to the database when the session commits. If an error occurs during a transaction, the session can be rolled back, ensuring that partial changes don't persist in the database.

Pydantic

Pydantic provides data validation and settings management throughout the application. The library defines data schemas using Python type annotations, automatically validating data against these schemas and providing detailed error messages when validation fails.

API request and response models are defined as Pydantic classes, specifying the expected structure, types, and constraints for each field. When the FastAPI backend receives a video generation request, Pydantic validates if all required parameters are present, numeric values fall within acceptable ranges, and string fields match expected patterns.

The library's integration with FastAPI enables automatic request validation at the framework level. Invalid requests are rejected before reaching endpoint handlers, with HTTP 422 responses containing detailed information about which fields failed validation and why. This early validation prevents malformed data from reaching business logic or the database.

Pydantic also manages application configuration, loading settings from environment variables with type conversion and validation. This approach centralizes configuration management and catches configuration errors at application startup rather than runtime.

Bcrypt

bcrypt provides password hashing functionality using the bcrypt algorithm, specifically designed for secure password storage. When a user registers or changes their password, bcrypt hashes the plaintext password with a randomly generated salt, producing a hash that can be safely stored in the database.

The adaptive nature of bcrypt allows the system to increase the work factor as computing power advances, maintaining security against increasingly powerful hardware. During authentication, the provided password is hashed using the same algorithm and compared to the stored hash - if they match, the password is correct. This one-way hashing ensures that even if the database is compromised, attackers cannot retrieve plaintext passwords.

PyJWT

PyJWT implements JSON Web Token (JWT) encoding and decoding for the authentication system. The library creates digitally signed tokens containing user claims such as user ID and expiration timestamp, ensuring that tokens cannot be tampered with without detection.

When a user successfully authenticates, PyJWT generates an access token signed with the application's secret key. The token includes the user ID and an expiration time, encoded as a compact JSON structure. Subsequent API requests include this token in the Authorization header, and PyJWT verifies the signature and checks expiration before extracting the user ID claim.

The library supports multiple signing algorithms, with the system using HMAC-SHA256 (HS256) for symmetric signing. This algorithm provides strong cryptographic guarantees while maintaining efficiency for the high-frequency token validation operations required by the API server.

PyJWT's automatic expiration handling simplifies token lifecycle management - expired tokens are automatically rejected during verification, eliminating the need for manual timestamp comparisons.

WebSocket

WebSockets provide a full-duplex communication channel between the frontend and backend, enabling real-time bidirectional data exchange. In contrast to traditional HTTP polling, where the client must continuously issue requests to check for new data, a WebSocket connection remains open and allows the server to push updates immediately when state changes. Eliminating polling removes unnecessary request overhead, reduces latency, and lowers both network and computational load on the system.

In this system, WebSockets are used to continuously stream job-related information, including the current state and progress of video generation tasks. The persistent connection ensures that users receive timely updates during long-running computations. This mechanism improves responsiveness and user experience compared to periodic polling, which would introduce unavoidable delays and unnecessary network traffic.

3.5. Testing and Quality Assurance

The system's reliability and correctness are verified through structured manual testing supported by API mocking tools.

3.5.1. Manual Testing and API Mocking

Mockoon is used to simulate API behaviour independently of the backend state, enabling frontend development to proceed without waiting for backend features to be completed. This setup allows both sides of the system to develop and test independently, eliminating implementation blockers and allowing UI components to be validated early in the development cycle.

3.6. Performance Analysis

This section presents performance measurements of the video generation pipeline, collected on hardware representative of consumer-grade workstations. Understanding these characteristics is essential for capacity planning and setting realistic user expectations.

3.6.1. Test Environment

All benchmarks were conducted on a system equipped with:

- **GPU:** NVIDIA RTX 4070 Super (12GB VRAM)
- **RAM:** 32GB DDR5
- **CPU:** AMD Ryzen 9 7900

The system ran Ubuntu 24.04 under WSL on Windows 11.

Generation Performance

Figure 3.4 presents end-to-end generation times for the complete pipeline across different target resolutions and video durations. All measurements were conducted at 8 FPS base frame rate. For resolutions above 512x512, the pipeline generates base frames at 512x512 and applies upscaling to reach target resolution.

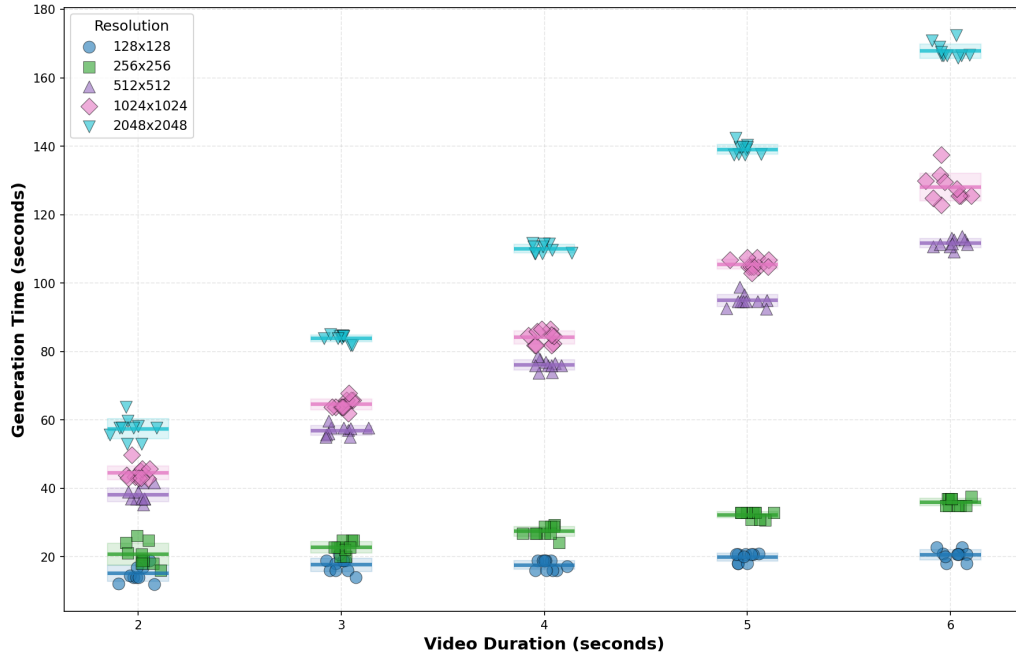


Figure 3.4: Total generation time across the complete pipeline for various video lengths and resolutions at 8 FPS.

The measurements show linear scaling with video duration for all resolutions. Low resolutions (128x128 and 256x256) complete in 12-22 seconds and 18-37 seconds respectively for the 2-6 second duration range.

The 512x512 resolution, serving as the base generation target, requires 38-110 seconds across the tested durations. This resolution provides the optimal balance between quality and processing time for the pipeline.

Higher resolutions demonstrate the efficiency of the upscaling approach. Generating 1024x1024 video takes 45-125 seconds, adding only 7-15 seconds of upscaling overhead to the 512x512 baseline. In contrast, direct 1024x1024 generation proved impractical: approximately 60 seconds for 2-second videos, approximately one hour for 5 seconds, and CUDA out-of-memory failures at 6 seconds. The 2048x2048 target requires 55-170 seconds, still completing in under 3 minutes for 6 seconds of video.

Notably, the variance in measurements (visible as scatter in the data points) remains relatively small across all resolutions, indicating stable performance characteristics.

3.6.2. Performance Impact of Interpolation

Figures 3.5 and 3.6 compare generation times for two approaches: baseline generation where all frames are produced through the diffusion process, and the pipeline's interpolation strategy where frames are generated at 8 FPS and intermediate frames synthesized using FILM.

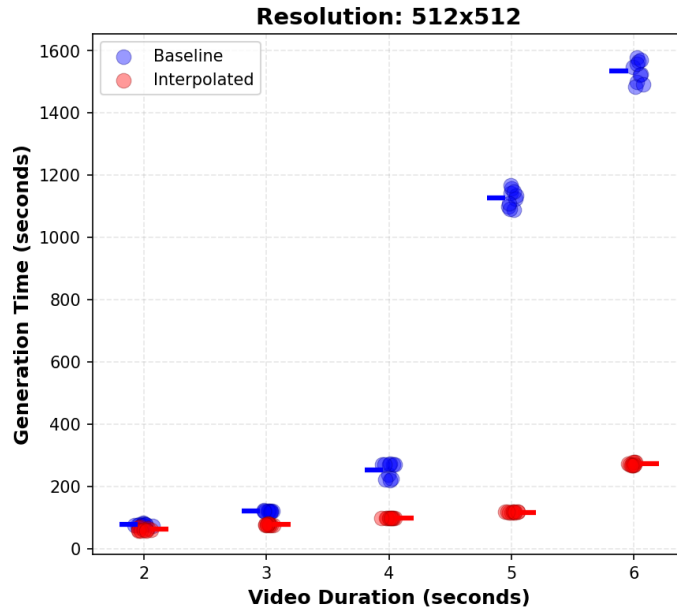


Figure 3.5: Generation time comparison for 16 FPS. The interpolation approach achieves nearly 10x speedup for longer videos.

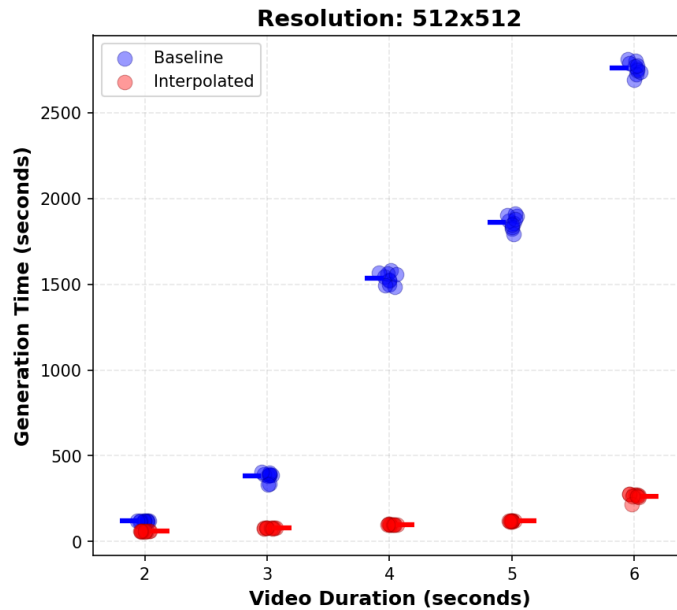


Figure 3.6: Generation time comparison for 24 FPS. The interpolation approach achieves nearly 20x speedup for longer videos.

The performance gains from interpolation are dramatic. For 16 FPS output, baseline generation requires 80-1550 seconds across the 2-6 second duration range. Notably, the baseline curve in both cases exhibits non-linear growth: while shorter videos scale approximately linearly with frame count, longer durations show accelerating slowdown. This occurs when VRAM pressure forces the system to offload data to RAM, adding substantial overhead from relatively slow PCIe bus transfers. In contrast, the interpolated approach completes approximately 10x faster for longer videos. This speedup comes from generating only half the frames through diffusion and synthesizing the remainder via FILM’s neural network.

The advantage becomes even more visible at 24 FPS. Baseline generation takes 120-2750 seconds, while the interpolated approach requires only 80-280 seconds - roughly 10x faster.

3.6.3. Performance Impact of Upscaling

Figure 3.7 compares two approaches for producing 1024x1024 videos: direct generation at target resolution versus generating frames at 512x512 and upscaling them with Real-ESRGAN neural network.

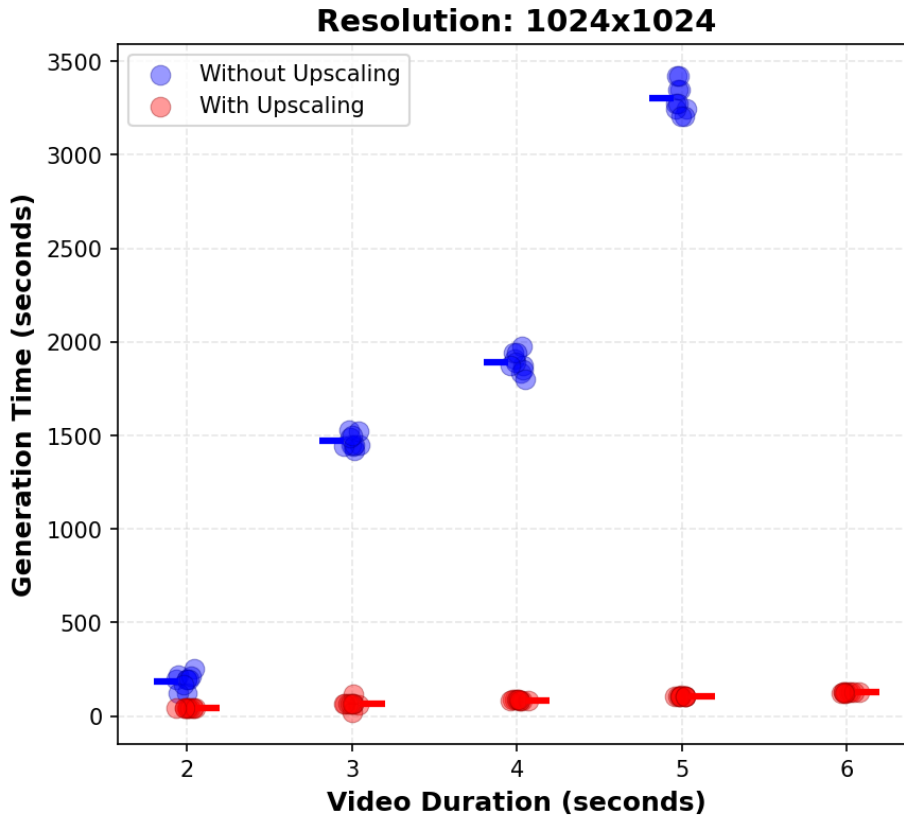


Figure 3.7: Generation time comparison for 1024x1024 output. The upscaling approach achieves 15-30x speedup for longer videos.

As we can see the performance difference is very noticeable. Direct 1024x1024 generation requires 180-3300 seconds for the 2-5 second duration range, with severe non-linear scaling due to VRAM pressure and CPU offload. Notably, 6-second videos could not be generated at all - every attempt resulted in CUDA out-of-memory errors despite CPU offload mechanisms. This hard limit demonstrates that direct high-resolution generation is not merely slow but fundamentally infeasible beyond certain durations on 12GB VRAM hardware.

In contrast, the upscaling approach completes in reasonable time across all tested durations - approximately 15-30x faster where direct generation succeeds. The upscaling overhead is minimal compared to baseline generation costs and scales linearly with frame count, avoiding the memory bottlenecks.

3.6.4. Summary

The results validate all three architectural decisions discussed in this chapter: generating at base 8 FPS with interpolation, producing frames at 512x512 with upscaling, and separating

concerns into independent pipeline stages. Each optimization compounds, transforming what would be either impossible or a 30-60 minute process into 1-3 minute workflow suitable for interactive video generation.

Chapter 4

Work organization

4.1. Project Characteristics and Methodology

The project described in this thesis was developed by combining elements of exploratory software development with a thoughtful approach to system design aiming to deliver a high-quality software system. While the core goal was to build a functional web application with a modern, distributed architecture, we have spent a significant portion of our efforts on exploring and evaluating existing deep learning models and their integrations to determine their viability for our project.

Unlike typical commercial projects with fixed requirements, the requirements for this system evolved dynamically. Initially, we knew that our goal was to approximate to whatever degree possible commercial solutions, such as Sora [14] by OpenAI or Veo [15] by Google DeepMind, given our computational and time constraints, but we did not know what best technologies to choose to pursue this goal. Eventually, the realization of the project proceeded roughly in three separate stages.

- **Exploration Phase:** The initial stage involved evaluating various open-source libraries (such as diffusers) and models (AnimateDiff [34], FILM [66], RealESRGAN [67]) to find a combination that produced acceptable results within hardware limits. Here we realized the true scope of the computational complexities and high-dimensionality of the problem that we were attempting to solve.
- **Prototyping Phase:** We designed the video generation pipeline and developed a module capable of executing the pipeline end-to-end. This allowed us to decide on the model composition that we would use to implement the video generation functionality of the application. Given this starting point, we were better able to make decisions regarding how to scale the generation process such that theoretically an arbitrary number of videos could be generated at the same time by the system.
- **System Integration and Finalization Phase:** The focus shifted to designing a system which enables an end user to generate and view the resulting videos through a friendly graphical user interface. While the previous two phases required us to dive deeply into generative models for images and videos, during this stage we needed to learn how to design a scalable and efficient inference service and integrate it with an intuitive and easy-to-use application frontend. This is the phase which took up the majority of our time and efforts that we have spent on this project since it proved quite complicated to make everything work correctly.

Large-scale generative models that unify natural language with visual and spatiotemporal scene modeling have emerged only recently and now represent one of the leading frontiers of AI research. Consequently, we approached this work primarily as a research effort, aiming to get a sense of the current progress in the field and learn how to deploy open weights models using contemporary approaches. After identifying the core technologies we decided to use, we transitioned from an iterative prototyping style of work to incremental/iterative development process. This has eventually led us to completing the video generation system and learning a great deal about real-world techniques for serving generative AI models.

4.2. Development Timeline

The project development spanned approximately 10 months, with distinct phases emerging organically as our understanding of the problem evolved.

4.2.1. Prototyping and technology exploration

The first four to five months were dedicated to research and exploration. During this period, we evaluated various approaches to model serving and system architecture. Initially, we considered building the system using a microservices architecture with gRPC for inter-service communication. However, after researching deployment strategies and examining the complexity this would introduce, we discovered Ray — a distributed computing framework that appeared well-suited for ML model serving. This discovery simplified our architecture significantly and became the foundation for our inference layer.

Simultaneously, we explored different text-to-video generation models and pipeline configurations. We experimented with AnimateDiff as the core generation model, evaluated FILM for frame interpolation, and tested RealESRGAN for upscaling. This exploration phase helped us understand the computational requirements, quality trade-offs, and practical constraints of running these models with our available hardware.

4.2.2. System integration and deeper dive into deep learning research

The following months (five to eighth) marked a transition toward integration and system infrastructure development. Having identified Ray as our deployment framework, we began the practical work of integrating the video generation pipeline with Ray Serve. This involved adapting our pipeline code to work within Ray’s distributed execution model, configuring resource allocation for GPU-accelerated inference, and implementing the API endpoints that would expose the generation capabilities to the rest of the system.

During this period, we also conducted a comprehensive literature review of text-to-video generation approaches. This deeper dive into the academic and industry research helped us understand the conceptual foundations of the models we were using, evaluate alternative architectures, and make informed decisions about pipeline composition. We examined various diffusion-based approaches and studied techniques for temporal consistency and frame interpolation. This research phase proved invaluable for understanding the trade-offs inherent in our chosen approach and identifying potential improvements to the generation quality.

Simultaneously, we focused on establishing the supporting infrastructure required for a production-ready system. We configured PostgreSQL as our primary database for storing user accounts, job metadata, and generation parameters. MinIO was set up to provide S3-compatible

object storage for the generated videos. A significant effort went into containerizing the entire application using Docker, which proved particularly challenging due to the need for custom images with NVIDIA CUDA dependencies to enable GPU access within the containerized Ray cluster. This containerization strategy ensured consistent testing across different development environments and made it possible to deploy the application in a cluster on a supercomputer or cloud infrastructure.

With the backend infrastructure taking shape, we began developing the user interface to provide an accessible way for users to interact with the video generation system. The frontend development proceeded in parallel with the backend work, with ongoing coordination to ensure the API contracts between the two layers remained consistent. At this stage, frontend used mock API responses to decouple development, allowing both frontend and backend work to proceed independently while each side focused on their respective priorities.

4.2.3. Integration, Testing, and Finalization

The final months (eighth to ten) brought all the components together and shifted our focus toward integration and feature prioritization. With the project deadline approaching, we needed to make pragmatic decisions about which features were essential for a functional system and which could be deferred or simplified.

Frontend development entered a more advanced stage as we transitioned from working with mock data to integrating with the actual backend APIs. This integration work revealed missing functionality and API contracts that needed to be adjusted. We refined the user interface based on real system behavior, adjusting loading states, error handling, and feedback mechanisms to provide a smooth user experience despite the inherently asynchronous nature of video generation.

On the backend side, we completed the inference endpoints and finalized the integration of the video generation pipeline with Ray Serve. This involved extensive testing to ensure that job submission, execution tracking, and result retrieval worked reliably under various conditions. We implemented the account management functionality, including user registration and profile management, along with authentication and authorization mechanisms to secure access to user resources. The API for retrieving generated videos was developed to support the frontend gallery interface, enabling users to browse and download their creations.

This period was characterized by intensive manual testing, where we validated the complete end-user registration and generation flow from account creation through video generation to result viewing. We identified and resolved numerous issues that only became apparent when all system components operated together, from authorization problems with object storage access to complexities in the job status tracking logic that needed to accurately reflect the multi-stage generation process. By the end of this phase, we had a functioning end-to-end system that met our core requirements and demonstrated the viability of serving open weights generative models through a modern web application architecture.

4.3. Project Stakeholders

The project involved several key stakeholders who played distinct roles in the realization and assessment of the system.

- **The Development Team (Authors):**

- **Michał Proć, Ryszard Żmija, Maciej Grzybac:** The students responsible for the design, implementation, and documentation of the system. In this project, the team also acted as the primary product owners, defining the system requirements based on the capabilities of the researched deep learning models and available open-source technologies.
- **Supervisor:**
 - **dr hab. inż. Rafał Dreżewski, prof. AGH:** Provided academic guidance, helped organize the workflow, oversaw the progress of the work, and ensured the project met the formal requirements of the thesis.
- **Diploma Project Class Instructor:**
 - **dr inż. Joanna Kosińska:** Supervised the progress of the project during diploma seminar and provided critical feedback.

4.4. The Team and Division of Duties

The project was realized by a team of three students. The division of duties was driven by both individual technical interests and hardware availability.

- **Michał Proć:** Primarily responsible for the **Frontend and User Experience**.
 - Designed the visual identity and user interface of the application, ensuring an intuitive and responsive user experience.
 - Implemented the web application frontend.
 - Helped with the design of the contract between the application frontend and backend, defining API contracts and data flow between the client and server.
 - Developed the client-side logic for job submission, gallery management and real-time status updates.
 - Created a user interface available in both Polish and English, together with a user guide explaining the application's key features.
 - Integrated the application with social media platforms, enabling users to easily share generated videos externally.
- **Ryszard Żmija:** Primarily responsible for **Infrastructure and Distributed Systems**.
 - Designed the overall system architecture, specifically the separation between the API server and the inference cluster.
 - Configured the Ray cluster to manage distributed model inference and resource allocation.
 - Set up a containerization strategy using Docker, including custom image with NVIDIA CUDA dependencies to enable GPU access within the containerized Ray cluster.
 - Configured the PostgreSQL database and MinIO object storage infrastructure, ensuring reliable persistence for user data and generated videos.

- Integrated the video generation pipeline with Ray Serve, connecting it to the business logic layer and exposing it through RESTful API endpoints.
 - Implemented the account management system, including user registration and profile management.
 - Developed the authentication and authorization mechanisms, ensuring secure access to the application's resources and API endpoints.
 - Led the effort to explore different approaches to text-to-video generation and conduct a thorough review of the existing literature.
- **Maciej Grzybacz:** Primarily responsible for **Pipeline Design and Model Integration**.
 - Designed and implemented the video generation pipeline, evaluating different combinations of components to optimize inference time and output quality.
 - Fine-tuned generation parameters to balance quality and performance.
 - Helped integrate the pipeline into the backend infrastructure and participated in the development of business logic.
 - Acted as the primary tester of the video generation pipeline, as he was the only team member with a GPU possessing sufficient VRAM to locally test its full capabilities.

4.5. Work Organization and Tools

We strived to develop an effective system for communication and collaboration that allows everyone on the team to understand our ongoing work and contribute to its implementation.

4.5.1. Communication

We used Discord as the primary communication platform and created a dedicated server to organize our efforts. While many auxiliary channels were created during development, the list below highlights only the most important ones that played a central role in collaboration.

- `#model`: For discussing the design and implementation of the video generation pipeline, exploring open weights models, and evaluating deployment strategies.
- `#backend`: For coordinating API development, business logic implementation, and the integration of the Ray cluster with the rest of the application.
- `#frontend`: For discussing UI design decisions, evaluating different layout approaches, and coordinating the integration between the client application and the backend services.
- `#important`: For critical project information, such as deadlines, milestone updates, major architectural decisions and summaries of key agreements reached during meetings.
- `#sources`: For providing links to relevant and interesting resources that we could use to better understand industry practices in deploying and serving machine learning models.

We also met in Discord voice channels to discuss the direction of the project and plan our next steps. Their frequency varied depending on the stage of the project: during the exploration and prototyping phase, meetings occurred sporadically as needed, often triggered by new research findings or experimental results. In the integration and finalization stages, voice meetings became more regular to synchronize implementation efforts.

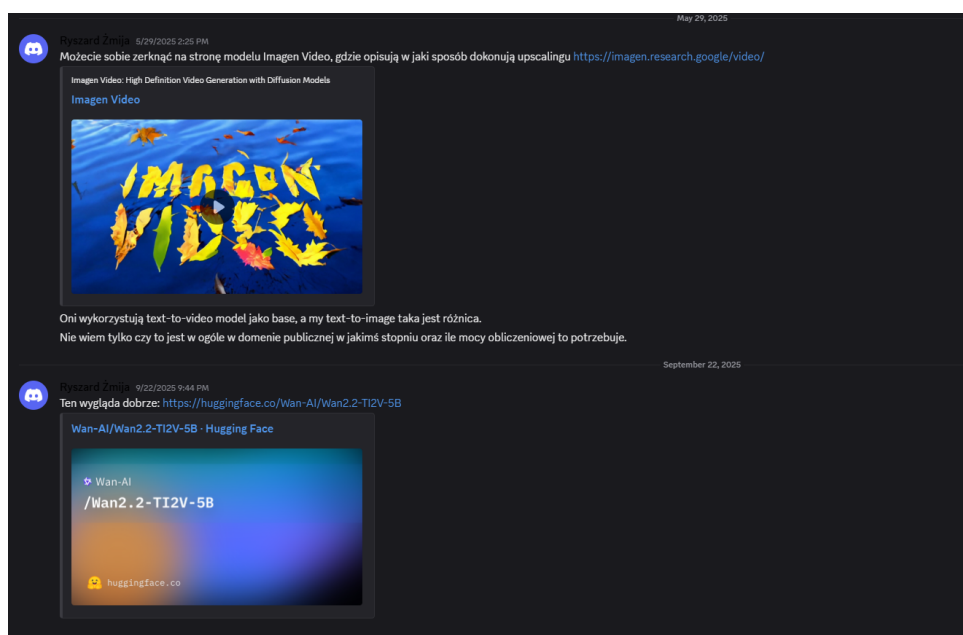


Figure 4.1: Discord was a great place for sharing information about deep learning research and existing open weights models.

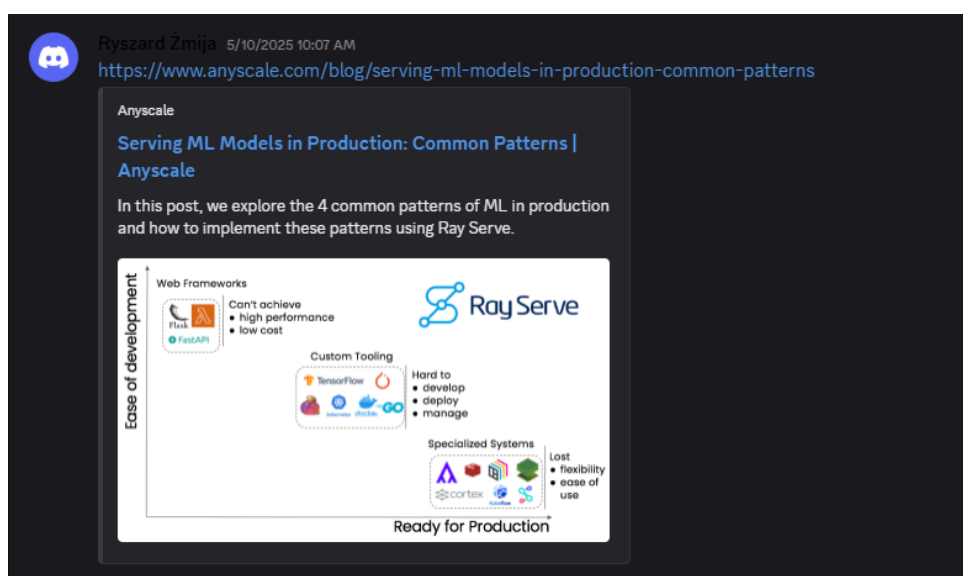


Figure 4.2: As none of us had experience in deploying machine learning models, it was crucial to keep everyone informed about industry best practices.

4.5.2. Task Management

Initially, we attempted to use GitHub Projects (Kanban boards). However, we found that the overhead of maintaining the board did not improve workflow efficiency. The process shifted to a more organic approach where tasks were assigned during voice chat meetings based on immediate project needs and individual availability.

4.5.3. AI Tool Usage

In the preparation of this bachelor's thesis, AI-powered language tools were utilized to enhance the clarity, coherence and overall quality of the written text. Additionally, AI tools were

used as a support during the initial research phase by assisting in the understanding of scientific literature and exploration of the problem domain. Information obtained in this manner was verified against original sources and referenced research papers. However, the content, ideas and conclusions presented herein are solely the author's own. Any errors or omissions remain the responsibility of the author.

4.6. Techniques and Practices Applied

We applied several software engineering practices to ensure code quality and ease of collaboration.

4.6.1. Development Workflow and Version Control

The source code was managed using Git and hosted on GitHub, which served as the central point of truth for the project. We adopted the GitHub Flow strategy, a streamlined branching model well-suited for our team size and deployment requirements.

In this approach, the main branch contains deployable code at all times, while new features and fixes are developed on separate feature branches. Each feature branch is created from main, developed independently, and then integrated back through Pull Requests (PRs).

Pull Requests served multiple purposes in the workflow:

- **Code review:** Team members reviewed each other's code before merging, attempting to catch potential issues early and ensure code quality.
- **Documentation:** PRs provided a record of what changed, why it changed, and how it was implemented.
- **Discussion:** PRs facilitated technical discussions about implementation approaches and design decisions.

This workflow enabled us to perform parallel development across different system components while at the same time facilitating code review and knowledge sharing, ensuring that all team members remained informed about changes to the codebase.

4.6.2. Testing and Validation

Given the distributed and non-deterministic nature of generative AI systems, traditional automated testing approaches proved challenging to implement within our time constraints. Instead, we adopted a pragmatic validation strategy that balanced thoroughness with development velocity.

We primarily relied on manual validation to verify implementations on feature branches before merging them into the main branch. Each of us tested their changes locally, ensuring that new functionality worked as expected in our development environment. During code reviews, we would pull feature branches of each other to validate functionality and catch issues that might not be apparent from reading the code alone.

To support parallel development between the frontend and backend, we relied on supporting tools such as Mockoon, Postman, and Swagger UI. Mockoon allowed the frontend to be developed independently during the early stages by providing realistic mock API responses,

eliminating blockers caused by incomplete backend functionality. In turn, Postman and Swagger facilitated backend testing, API inspection, and rapid iteration on endpoint definitions. This toolset ensured that both parts of the system could progress simultaneously without introducing integration delays.

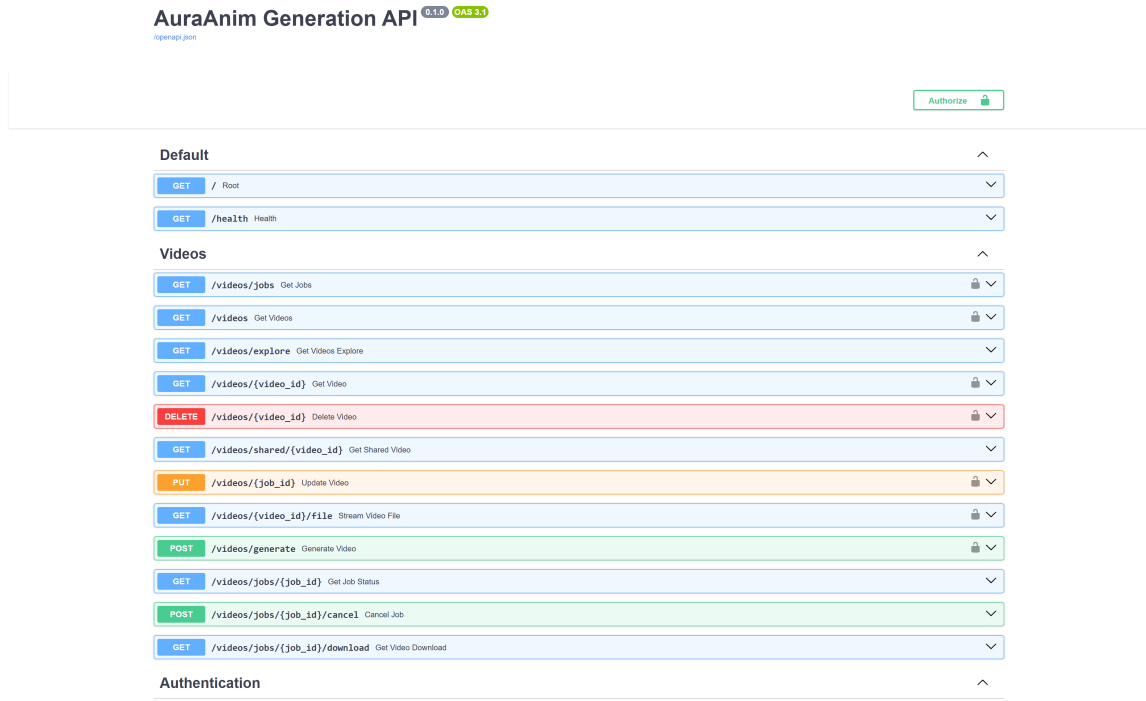


Figure 4.3: Screenshot of the API documentation view in Swagger UI.

For more complex features involving the integration of multiple system components, we used end-to-end testing scenarios that simulated realistic user workflows. This included testing the complete video generation pipeline from prompt submission through job execution to final video retrieval, helping us identify integration issues and performance bottlenecks that only manifested when the entire system operated together.

While we recognized the value of automated integration tests, the complexity of testing a system that involves GPU-accelerated model inference, distributed job scheduling, and non-deterministic generative outputs made this impractical given our timeline. We focused instead on delivering a functional system and ensuring that critical user flows were thoroughly validated through manual testing.

Chapter 5

Project results

5.1. Introduction

In this chapter, we present the results of our work on the text-to-video generation application. Over the course of approximately ten months, we designed, implemented, and integrated a complete system that transforms natural language descriptions into videos. Our goal, as established in Chapter 1, was to create an accessible platform that approximates the capabilities of commercial solutions like Sora and Veo, while operating within the constraints of consumer-grade hardware and open weights models.

We structured the system around three main components: a video generation pipeline built on AnimateDiff, FILM interpolation, and RealESRGAN upscaling - a scalable backend infrastructure powered by Ray Serve - and an intuitive web application frontend. Each component was designed to address specific challenges inherent to AI-powered video generation—from managing computationally intensive GPU workloads to providing real-time feedback during long-running generation processes.

This chapter serves two purposes. First, we provide a comprehensive walkthrough of the implemented system, demonstrating its features through screenshots and usage scenarios. We explain what users see and experience when interacting with the application, from account creation through video generation and library management, to content sharing on social media platforms. Second, we reflect on the project as a whole—evaluating our success against the original objectives, acknowledging limitations we encountered, and identifying directions for future development.

We begin with an overview of the implemented system components in Section 5.2, followed by a detailed application walkthrough in Section 5.3. Section ?? demonstrates key usage scenarios, while Section 5.4 summarizes practical performance characteristics. We then discuss limitations in Section 5.5 and potential future work in Section 5.6. Finally, Section 5.7 offers our concluding reflections on the project’s success and the lessons we learned throughout its development.

5.2. Overview of the Implemented System

5.2.1. Video Generation Pipeline

The modular, multi-stage video generation pipeline transforms text prompts into video sequences. Rather than attempting to generate high-resolution, high-frame-rate videos directly -

an approach that quickly exhausts available GPU memory - we adopted a strategy of generating base frames at reasonable resolution and fixed frame rate and enhancing them through multiple post-processing stages.

Our pipeline consists of five interconnected components deployed as independent Ray Serve actors. First, the Preprocessor analyzes the user's requested dimensions, frame rate and duration to calculate optimal scaling factors and determine which enhancement stages will be needed. Then Video Generator creates base frames at maximum resolution of 512x512 pixels and constant 8 FPS using AnimateDiff, which extends Stable Diffusion models with motion adapters to maintain temporal coherence. Users can choose from multiple base models, which gives them flexibility in visual style, along with LoRA modules that introduce specific type of camera movement.

When higher frame rate is requested, the Interpolator synthesizes intermediate frames between base sequence using FILM neural network. This approach proves dramatically more efficient than generating every frame through diffusion: interpolation usually adds 15-30 seconds to generation time, while generating all frames through AnimateDiff would require substantially longer. Similarly, when resolution exceeds 512 pixels in either dimension, the Upscaler applies frame upscaling using RealESRGAN instead of attempting direct high-resolution generation, which would consume even 10-20x more processing time and frequently fail due to memory exhaustion.

Finally, the Postprocessor handles the final transformations. It first trims the frame sequence to match the requested duration, removing any extra frames that were generated as interpolation buffers. If frame dimensions do not match the target resolution after upscaling, the Postprocessor applies center cropping. Lastly, it encodes frames to user's chosen format, making them ready to be saved in MinIO storage.

5.2.2. Backend Infrastructure

The backend infrastructure provides the foundation that connects user interactions with the video generation pipeline. We built a system that handles authentication, job management, and data persistence while maintaining responsiveness despite the computationally intensive nature of video generation.

API Server

At the core of our backend sits a FastAPI server that exposes RESTful endpoints for all client operations. The server handles user registration and authentication through JWT tokens, ensuring secure access to protected resources. When a user submits a video generation request, the API server validates the input parameters, creates a job record in the database, and forwards the request to the generation pipeline. Throughout the generation process, the server tracks job status and provides this information to clients through WebSocket connections.

We implemented ownership-based access control to ensure users can only view and manage their own videos and generation jobs. Each API request includes the user's identity extracted from the JWT token, which is validated against resource ownership before granting access. This approach provides data isolation between users without requiring complex role-based permission systems.

Distributed Processing with Ray Serve

For managing the computationally intensive video generation workload, we adopted Ray Serve as our distributed serving framework. Ray Serve allows us to deploy the generation pipeline as a collection of independent replicas, each capable of processing video generation requests. The framework automatically distributes incoming requests across available replicas, preventing resource contention whenever possible when multiple users submit jobs simultaneously.

The integration between the FastAPI server and Ray Serve follows a clean separation of concerns. The API server handles all user-facing operations—authentication, request validation, and job tracking—while Ray Serve manages the actual video generation computation. This separation allows us to scale these components independently: we can add more API server instances to handle increased user traffic, or deploy additional Ray Serve replicas on GPU nodes to increase generation throughput.

Database and Storage Systems

We use PostgreSQL as our relational database for storing structured data. The schema organizes information across four main tables: user accounts, generation jobs, job parameters, and job results. This structure allows us to efficiently query job history, retrieve generation parameters for reproducibility, and track the status of ongoing generation tasks. SQLAlchemy provides the ORM layer, offering a Pythonic interface to database operations while managing connection pooling to prevent exhaustion under heavy load.

For storing generated video files, we deployed MinIO as an S3-compatible object storage service. This dual-storage architecture separates concerns appropriately: PostgreSQL handles metadata queries efficiently, while MinIO manages the high-throughput demands of video file storage and retrieval. When a generation job completes, the pipeline writes the output file to MinIO and stores only the object reference in PostgreSQL. Users can then download their videos through URLs, which offload file serving from the API server directly to MinIO.

5.2.3. Web Application Frontend

To access the video generation capabilities of the system, users interact with the "AuraAnim" web application. The frontend was developed using the Next.js framework and designed with full bilingual support in Polish and English. Thanks to the integration of the `i18n` internationalization mechanism, additional languages can be easily introduced in the future without modifying the application's core structure. The homepage, as well as the remainder of the interface, is available to users after registering or logging into the application.

The visual design of the web interface is based on a neon, cyberpunk-inspired color palette combined with the Genos font, creating a modern and futuristic aesthetic that aligns with the theme of video generation.

The interface also incorporates numerous animations and transitions, including an animated background, smoothly expanding sections, and hover-triggered zoom effects. These elements enhance interactivity and provide a dynamic, visually engaging user experience.

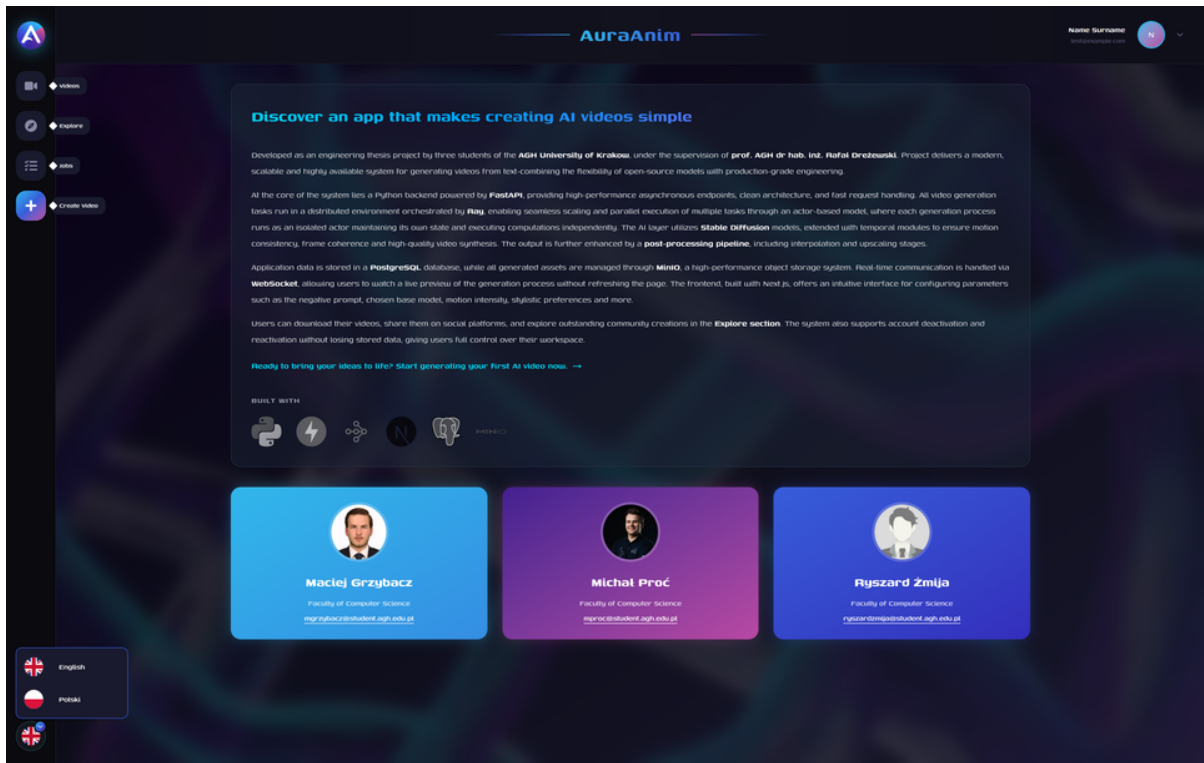


Figure 5.1: Homepage of the application in English.

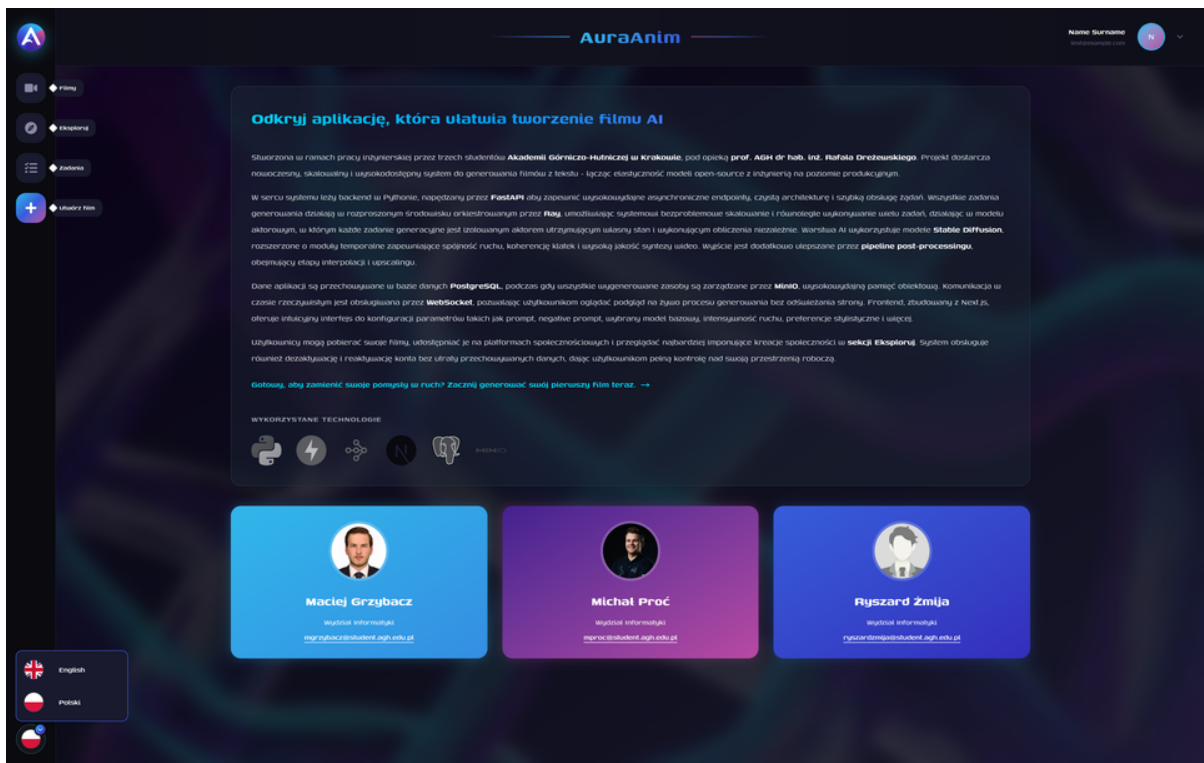


Figure 5.2: Homepage of the application in Polish.

Table 5.1: Color palette of the AuraAnim application

Color Sample	Name	Hex Code
	Sky	#31B7EA
	Ocean	#358EE3
	Royal	#375DDA
	Indigo	#342EBC
	Violet	#442090
	Magenta	#B949A3

5.3. Application Walkthrough

In addition to the homepage, the application includes a complete panel for generating and managing created videos, as well as a set of authentication views with a separate layout, visible to users who have not yet registered in the system.

The authentication views include:

- **/login** — the login view, requiring an email and password,
- **/register** — the registration view, requiring an email address, username, full name, and password with confirmation,
- **/forgot-password** — the view displayed after selecting the "Forgot password" option,
- **/reset-password** — the view for resetting a user's password,
- **/verify-email** — the view displayed after registration or reactivation, informing the user that email verification is required,
- **/reactivate** — the view for reactivating a previously deactivated account.

The user panel provides several views that allow users to generate new videos, browse their previous generations, and explore example results in the "Explore" section:

- **/videos** — a list of all videos generated by the user,
- **/videos/explore** — the "Explore" view, where the user can browse example videos generated by the model,
- **/videos/create** — instructions and forms for creating a new video,
- **/videos/:id** — the video detail view, where the user can watch the video in fullscreen, inspect generation parameters, share the output, and manage the video,
- **/jobs** — an overview and management panel for ongoing generation tasks, as well as a history of completed jobs,
- **/users/:id** — the user profile settings view, where the user can modify personal information (full name, nickname, email, password) and deactivate or delete the account.

In addition to the views listed above, the panel also provides the dedicated view **/videos/:id/shared**, where other users can preview videos shared via social media or a direct link.

5.3.1. User Registration and Authentication

The authentication views use a separate layout in which all other system functionalities remain hidden. These views cover all interaction paths available to an unauthenticated user, including logging into the system, registering a new account and reactivating an existing account, along with displaying confirmation messages sent to the user's email address. Authentication section provide also forms for resetting a password and defining a new one. All critical fields, such as email and password, include full validation (the password requirements include a minimum of 8 characters, at least one lowercase letter, one uppercase letter, and one digit).

The screenshots below present example authentication views. The remaining views share the same layout structure and differ only in the displayed information and the specific form they contain.

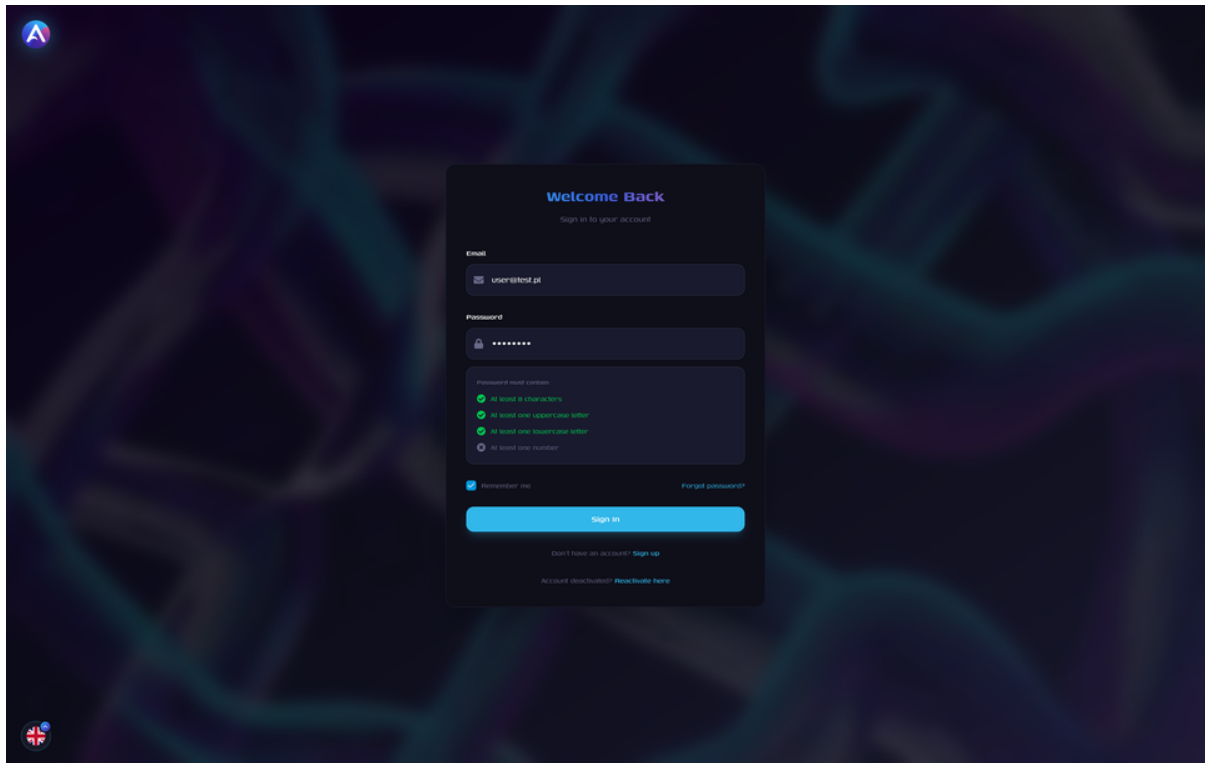
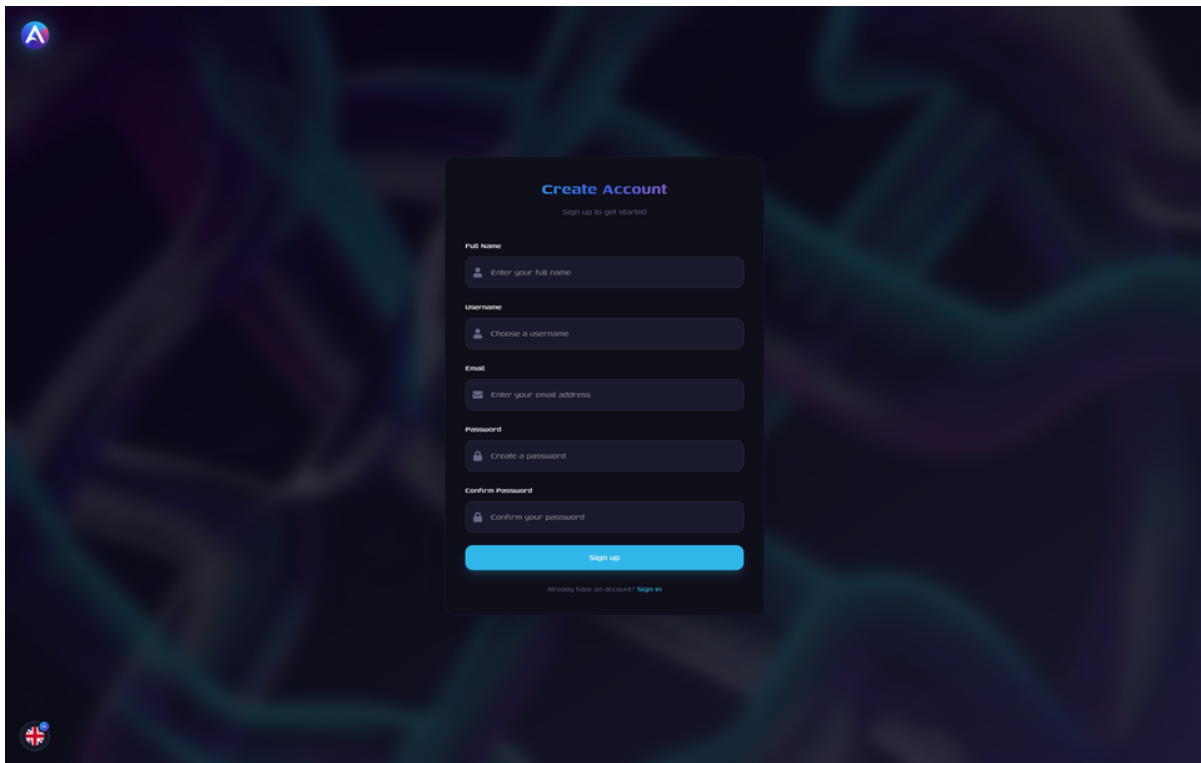
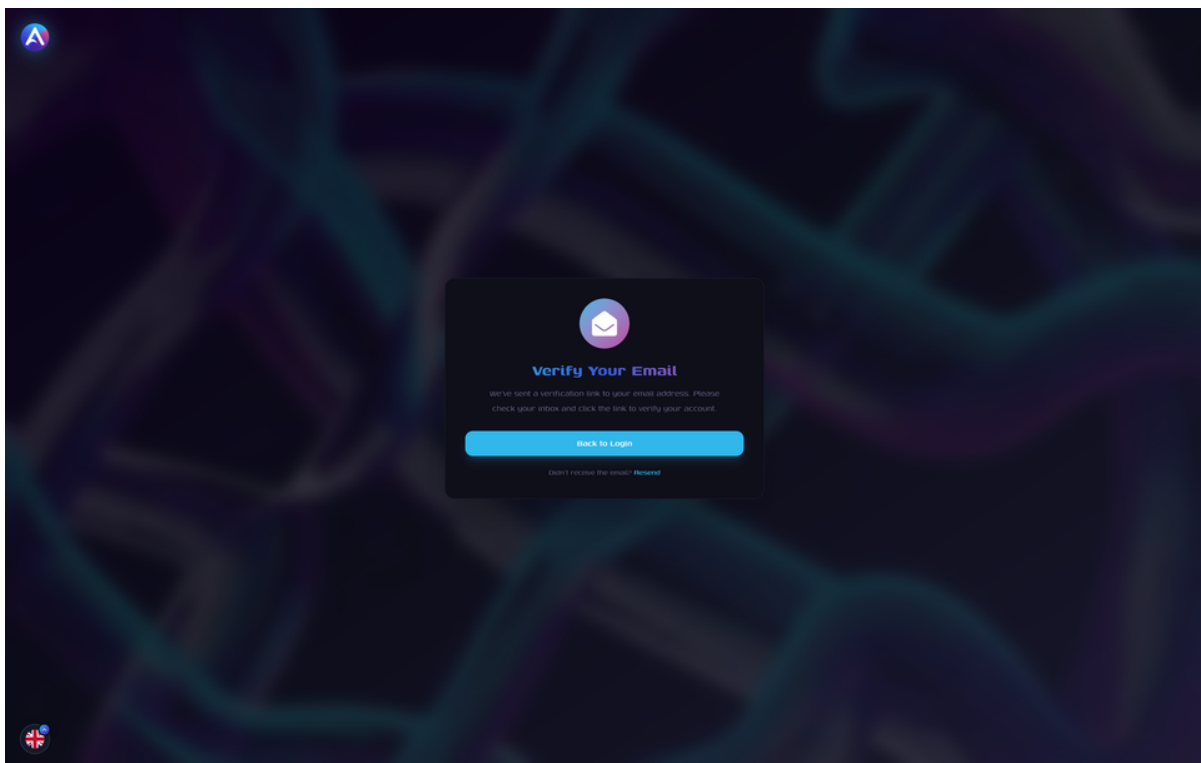


Figure 5.3: View of the login form with a password that does not meet the validation requirements.



The image shows a 'Create Account' registration form centered on a dark background with a subtle pattern. The form is a dark gray card with the title 'Create Account' and the subtitle 'Sign up to get started'. It contains five input fields: 'Full Name' (with a person icon), 'Username' (with a person icon), 'Email' (with an envelope icon), 'Password' (with a lock icon), and 'Confirm Password' (with a lock icon). Each field has a placeholder text: 'Enter your full name', 'Choose a username', 'Enter your email address', 'Create a password', and 'Confirm your password'. Below the fields is a prominent blue 'Sign up' button. At the bottom of the card, there is a link: 'Already have an account? Sign in'. The background features a small logo in the top left and a small flag icon in the bottom left.

Figure 5.4: View of the empty registration form in the system.



The image shows a 'Verify Your Email' notification card centered on the same dark background. The card has a title 'Verify Your Email' with an envelope icon above it. Below the title, it says: 'We've sent a verification link to your email address. Please check your inbox and click the link to verify your account.' There is a blue 'Back to Login' button. At the bottom, there is a link: 'Didn't receive the email? Resend'. The background features the same small logo in the top left and small flag icon in the bottom left.

Figure 5.5: View of the verification email notification displayed after account registration or reactivation.

5.3.2. Video Generation Interface

A user who wishes to generate a new video, upon opening the "Create new video" view, is presented with three expandable sections: **Instructions**, **Quick Generation** and **Full Generation**.

Instructions section provides detailed explanations of each parameter, describing their purpose and how to adjust them so that the model interprets the input correctly and produces the best possible result. This section also includes the option to select the generation model, along with information on the visual style offered by each available model.

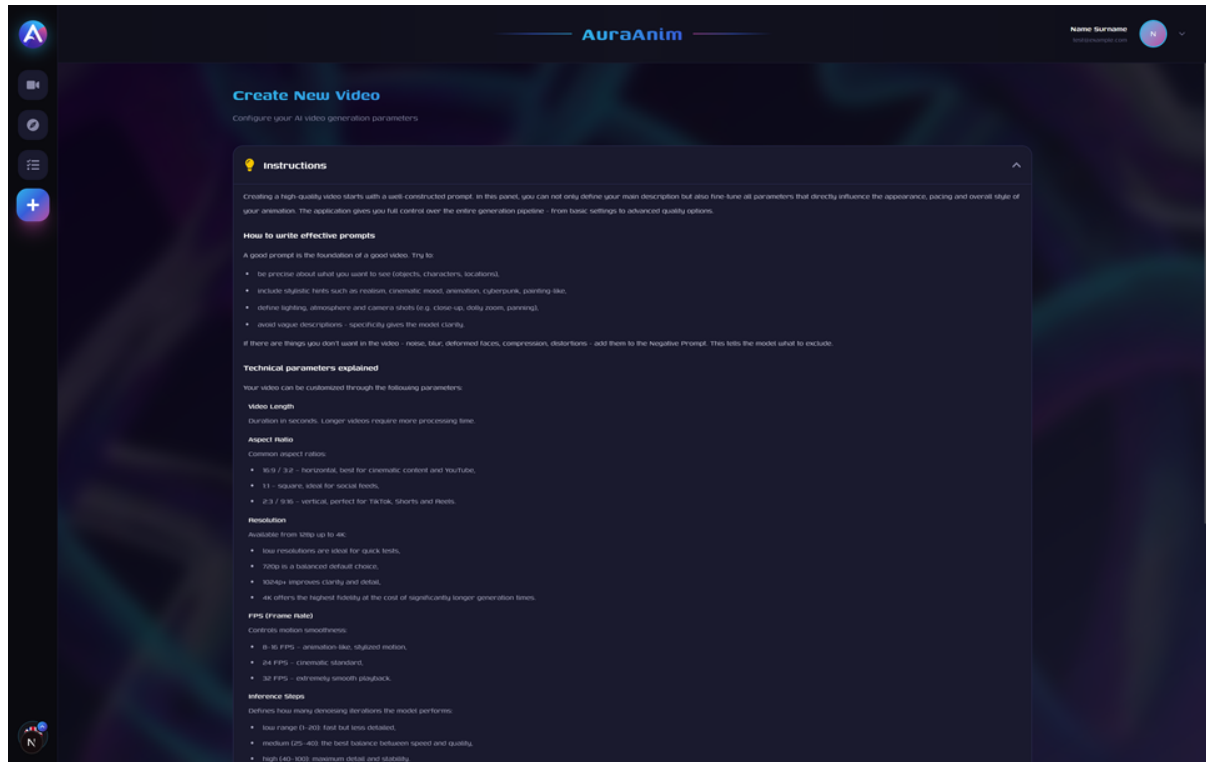


Figure 5.6: Video creation view with the expanded **Instructions** section.

Quick Generation is a simplified generation form in which the user provides only the prompt and the desired video duration. All other parameters are either omitted (e.g., negative prompt) or assigned default values, such as the base model and aspect ratio.

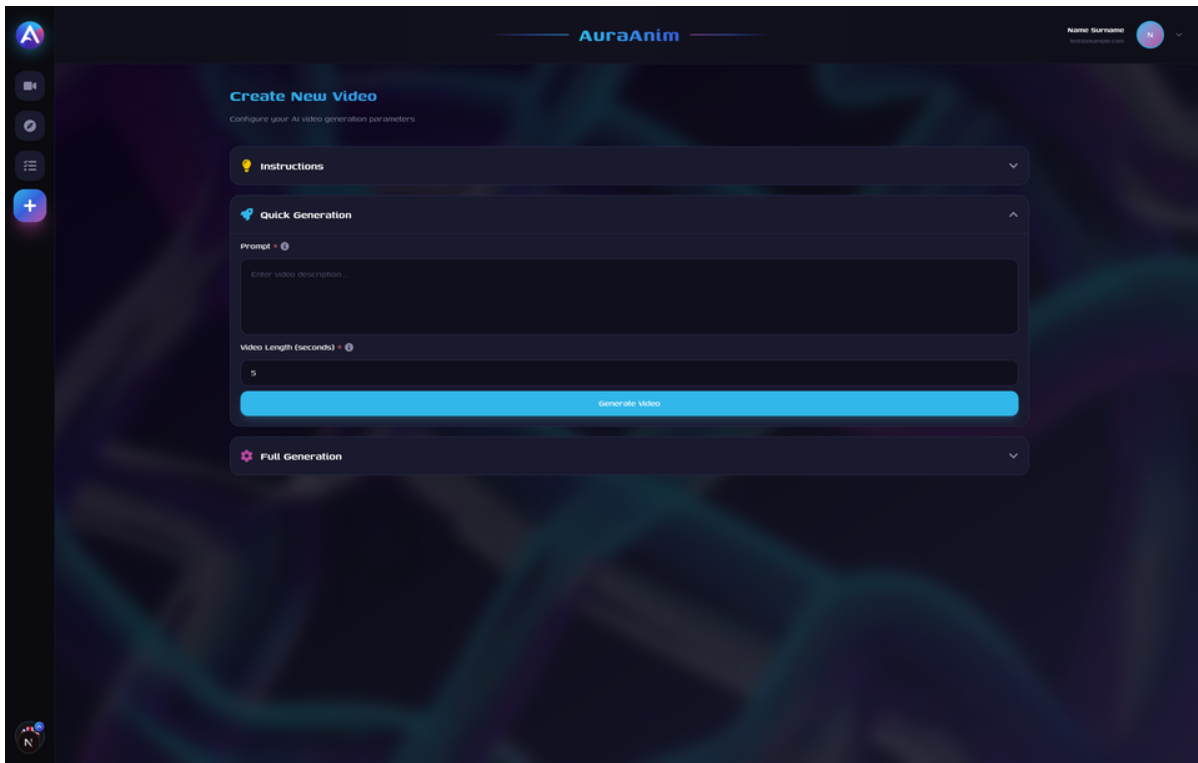


Figure 5.7: Video creation view with the expanded **Quick Generation** section.

Full Generation is designed for users who want full control over the process and intend to utilize the complete set of available parameters. This view allows the user to specify the prompt, negative prompt, video duration, aspect ratio, resolution, frames per second, base model, and inference steps. Each parameter includes a tooltip containing a brief explanation of its meaning and function.

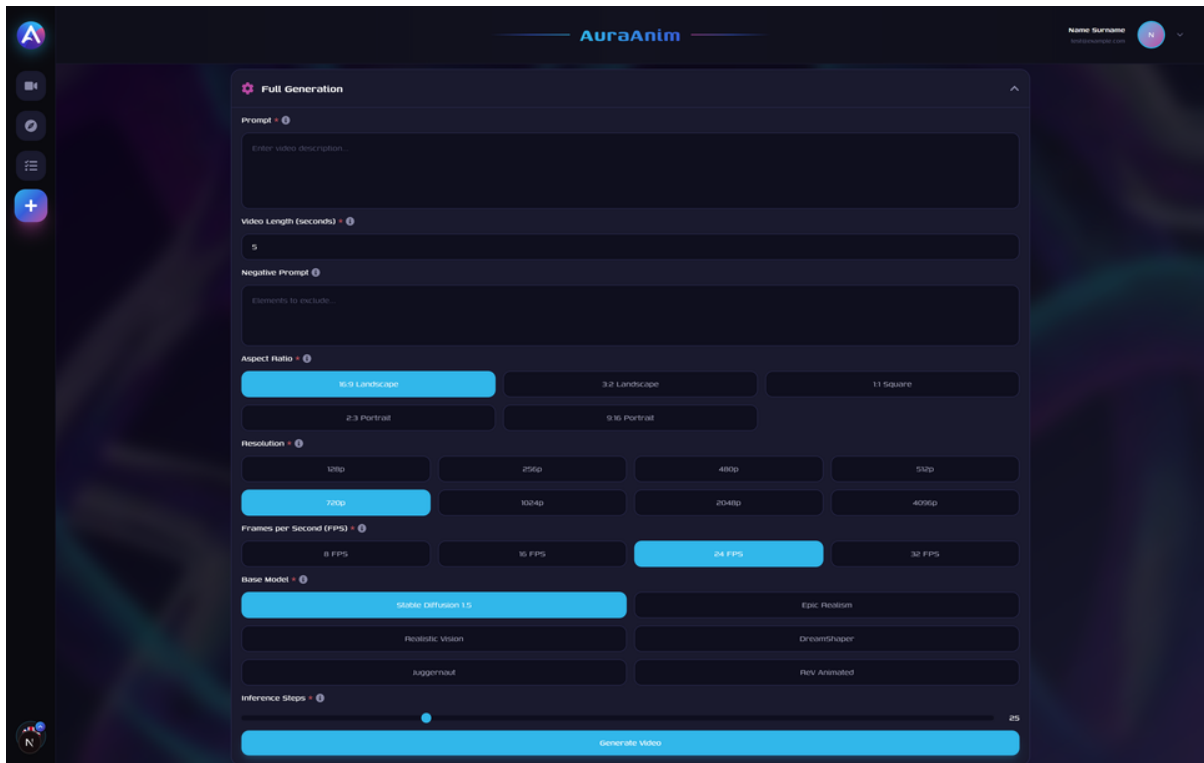
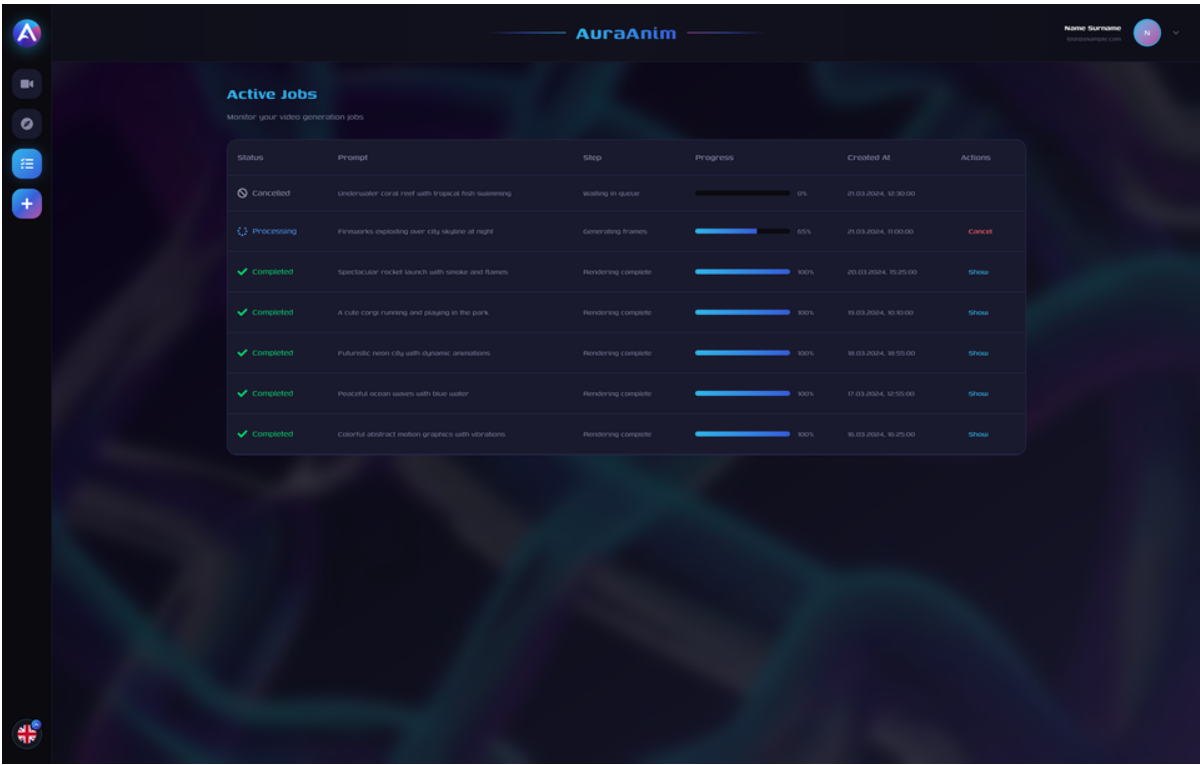


Figure 5.8: Video creation view with the expanded **Full Generation** section.

5.3.3. Generation Progress Monitoring

The user has two ways to monitor active and completed jobs. The first is the **Jobs** section in the user panel, where all initiated and completed jobs are listed with their status and basic information. For completed jobs, the application allows the user to navigate to the generated video. For jobs with the pending or processing status, the user may cancel the generation process (although the job entry does not disappear from the table). An example of this table is shown in Figure 5.9.

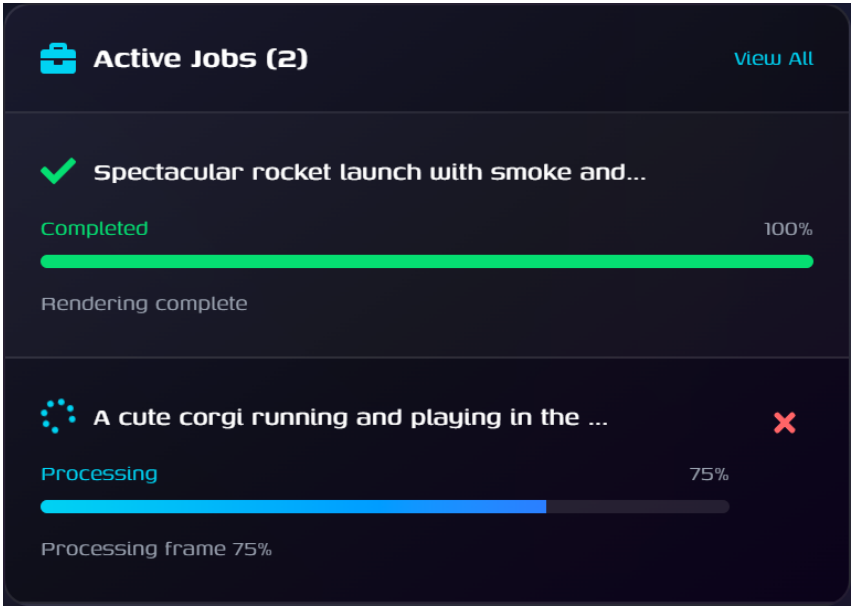
The second method is the job preview panel located in the bottom-right corner of the screen, shown in Figure 5.10. This panel retrieves all active jobs and displays them along with their status and completion percentage. Once a job is finished, it disappears from the list after a short moment. This mechanism enables real-time progress tracking through the use of websockets. The user may collapse the panel at any time so that it does not interfere with other interactions within the application.



The screenshot shows the AuraAnim web interface. At the top, there's a header with the AuraAnim logo and a user profile. Below the header, there's a sidebar with navigation icons. The main content area is titled "Active Jobs" and contains a table with the following data:

Status	Prompt	Step	Progress	Created At	Actions
Cancelled	Underwater coral reef with tropical fish swimming	Waiting in queue	0%	21.03.2024, 12:30:00	
Processing	Fireworks exploding over city skyline at night	Generating frames	60%	21.03.2024, 11:00:00	Cancel
Completed	Spectacular rocket launch with smoke and flames	Rendering complete	100%	20.03.2024, 15:25:00	Show
Completed	A cute corgi running and playing in the park	Rendering complete	100%	19.03.2024, 10:10:00	Show
Completed	Futuristic neon city with dynamic animations	Rendering complete	100%	18.03.2024, 16:05:00	Show
Completed	Peaceful ocean waves with blue water	Rendering complete	100%	17.03.2024, 10:50:00	Show
Completed	Colorful abstract motion graphics with vibrations	Rendering complete	100%	16.03.2024, 10:25:00	Show

Figure 5.9: View of the table with all jobs.



The screenshot shows a detailed view of active jobs. It features a header with a briefcase icon, the text "Active Jobs (2)", and a "View All" link. Below the header, there are two job entries:

- Completed Job:** "Spectacular rocket launch with smoke and...". It has a green checkmark icon, the status "Completed", a green progress bar at 100%, and the text "Rendering complete".
- Processing Job:** "A cute corgi running and playing in the ...". It has a blue gear icon, the status "Processing", a blue progress bar at 75%, and the text "Processing frame 75%". A red 'X' icon is visible in the top right corner of this entry.

Figure 5.10: View of active jobs updated in real time.

5.3.4. Video Gallery and Management

In the **Videos** tab, the user can browse all of their completed projects. List displays every finished generation along with its name (automatically derived from the prompt, although the user may later modify it) and basic information such as duration and resolution. For performance optimization, the view loads the media file only when it becomes visible on the screen - before that, a loading spinner is displayed instead of the video preview.

Within this view, the user can select any video to open its detailed profile, where the full preview and all available actions are presented.

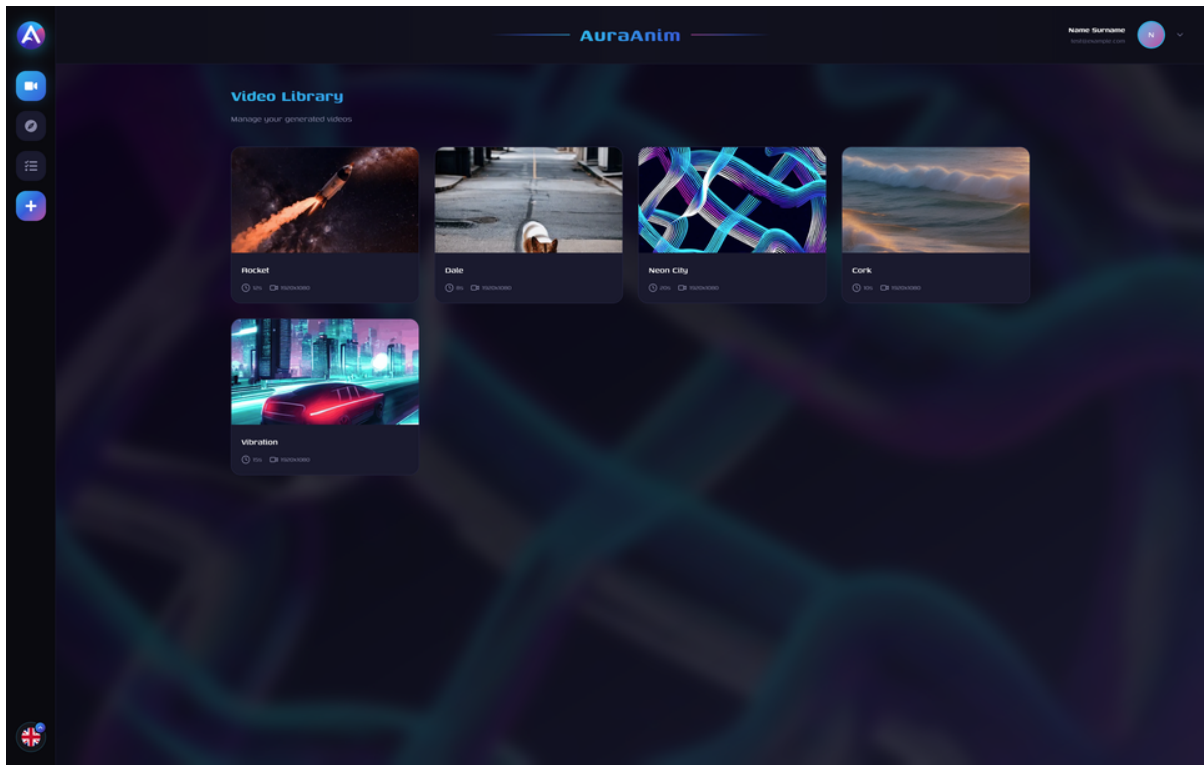


Figure 5.11: View of all user generations.

After opening a video profile, the user is presented with a full preview of the generated video, the creation date and key generation details. The interface displays parameters such as prompt and resolution, but application does not retain certain internal generation settings, such as negative prompt or number of inference steps.

The user also has access to a wide range of operations that can be performed on a video. These include downloading it, deleting it, renaming within the collection and sharing the video on social media or other web platforms.

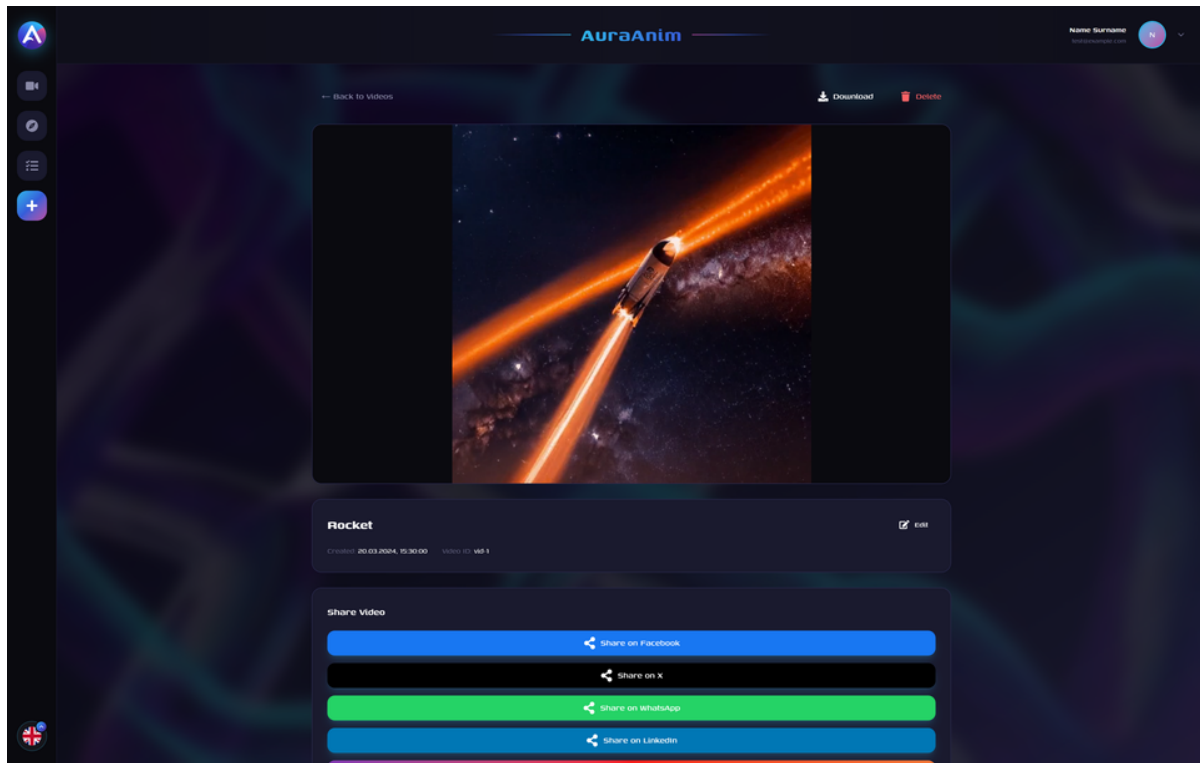


Figure 5.12: View of the full generation preview.

5.3.5. Social Media

The video profile allows users to share generated videos on social media platforms and other services directly from the application view. App supports opening external platforms via deep links, which redirect the user directly to the post creation interface (in the case of Facebook, X, WhatsApp, and LinkedIn). For Instagram, direct redirection to the post creation view from external applications is not supported due to platform limitations. In this case, the user is appropriately informed about the need to download the video and share it manually within the Instagram application.

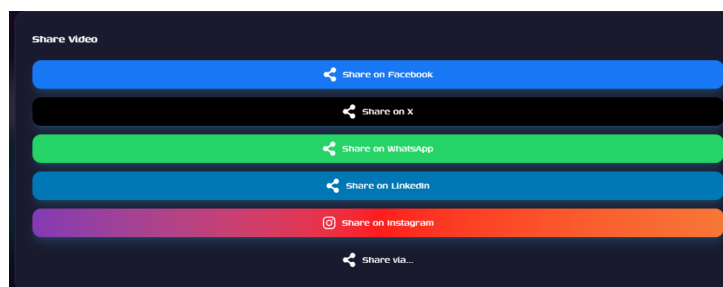


Figure 5.13: Panel displaying all available sharing options.

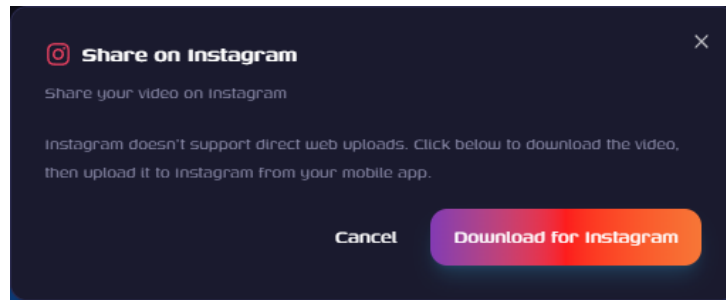


Figure 5.14: Modal informing the user that direct sharing to Instagram is not supported.

Additionally, the application integrates the **Web Share API**, which enables native sharing functionality provided by the user's operating system or browser. This mechanism allows users to share a video link or file using available system-level sharing options, such as messaging applications, email clients, or social media apps, without requiring platform-specific integrations.

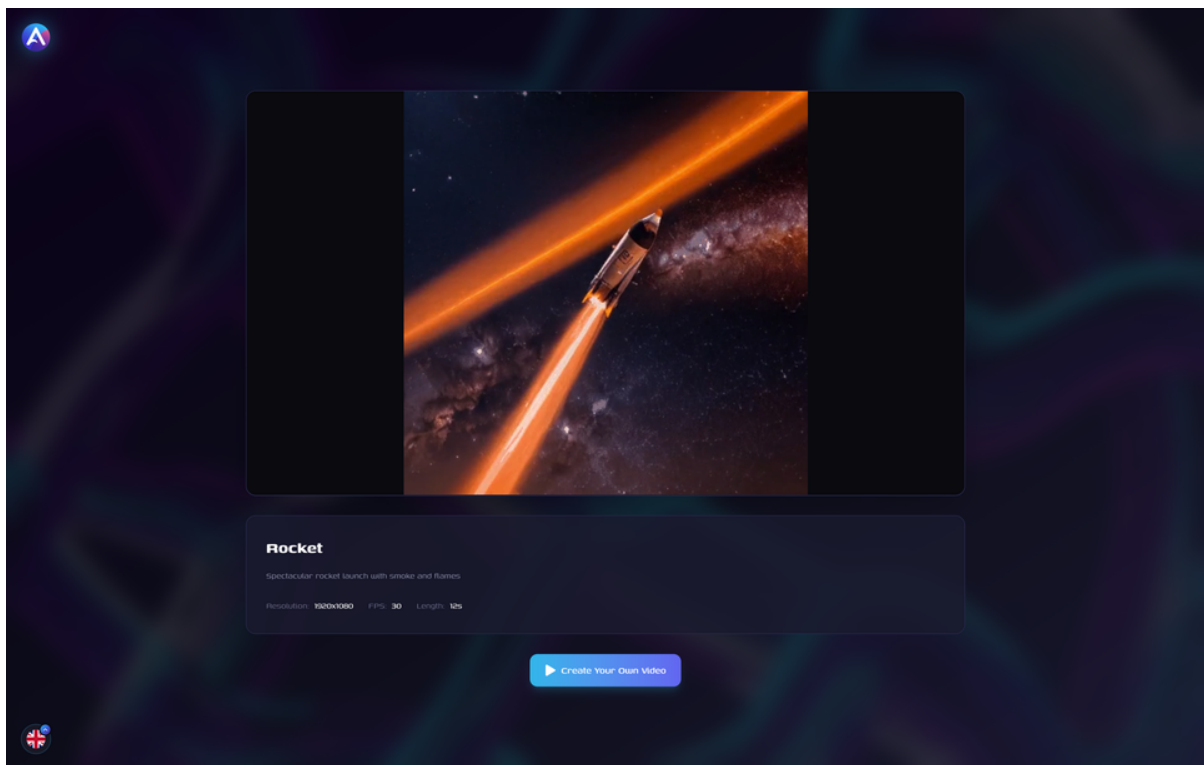


Figure 5.15: View of the shared generation preview.

5.3.6. Explore Section

The **Explore** section presents a view where users can browse example generations, download them or use the displayed prompts as inspiration. List of videos is the same for all users and is updated exclusively by system administrators.

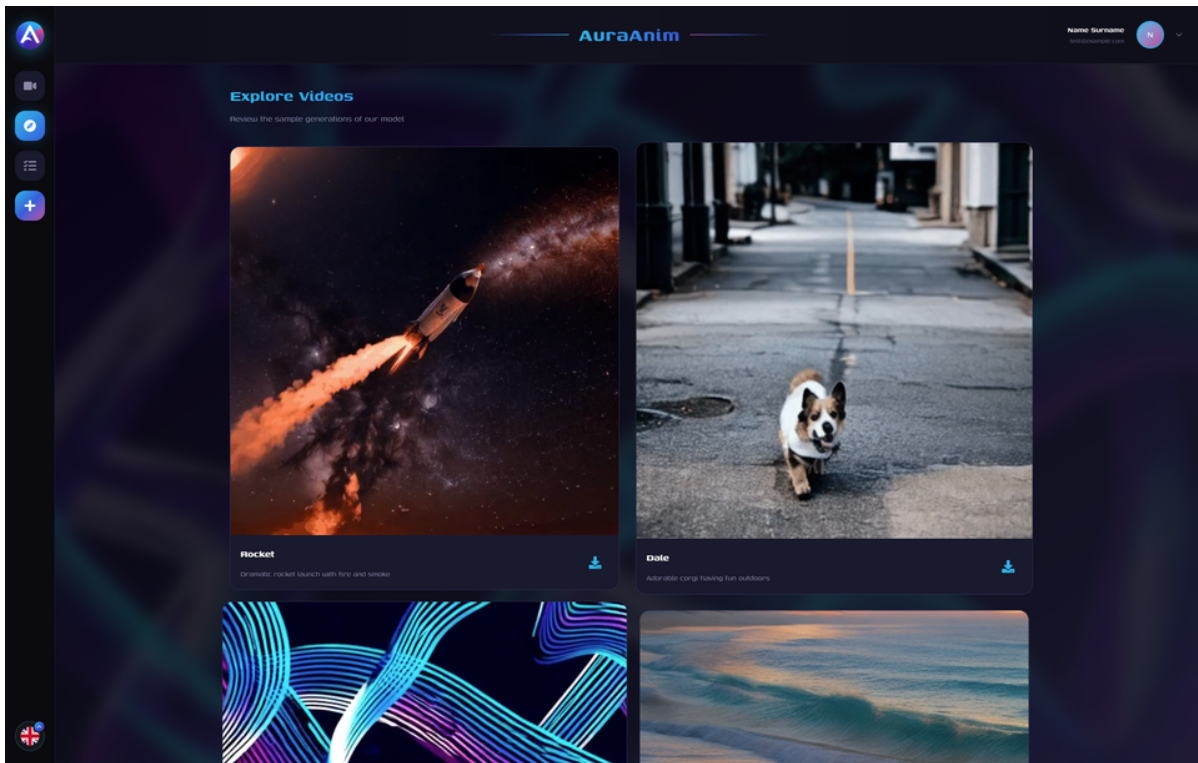


Figure 5.16: View of the **Explore** section with example application generations.

5.3.7. User settings

In the **User Settings** view, the user can modify basic account information, such as email address and personal details, as well as change the account password. The user may also deactivate the account (reactivation requires email confirmation before logging in again) or permanently delete the account.

In future development of the application, this view is intended to include privacy-related settings. At the current stage, all user accounts and their content are visible only to their respective owners.

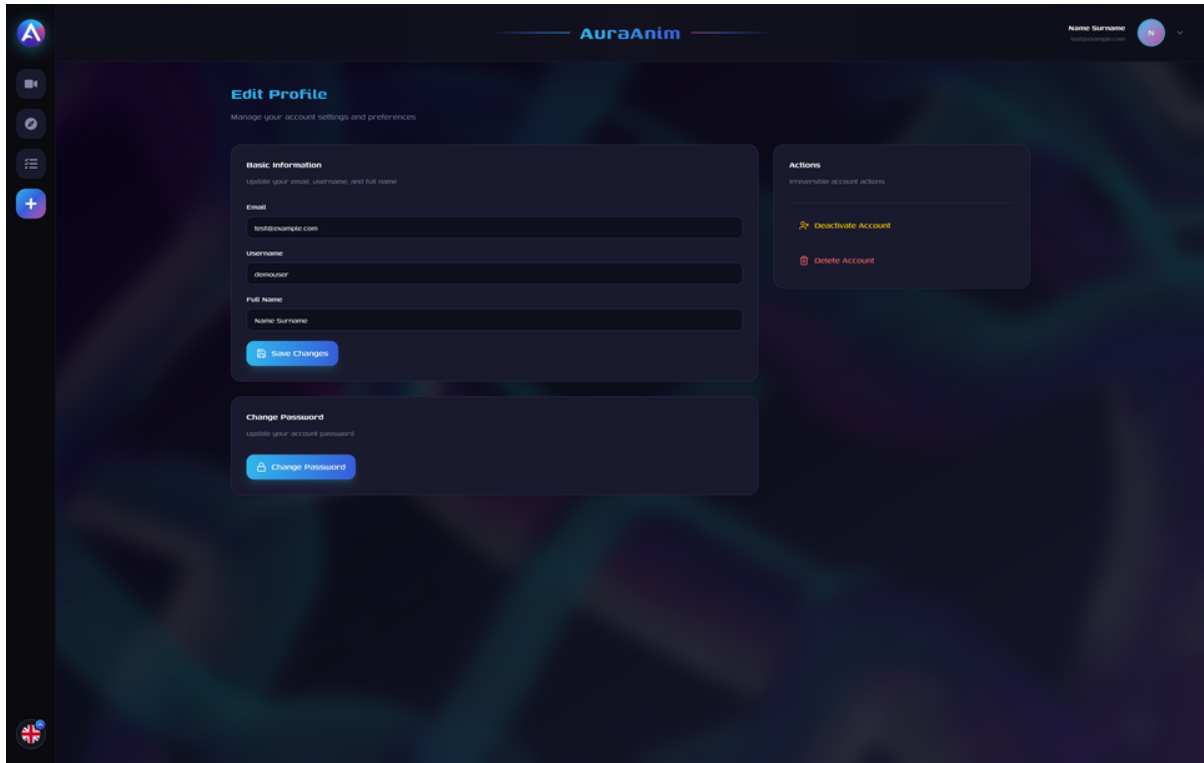


Figure 5.17: View of the **User Settings** page.

5.4. Performance Results Summary

We conducted performance benchmarks on a machine running Ubuntu 24.04, equipped with an NVIDIA RTX 4070 Super GPU with 12 GB of VRAM, 32 GB of DDR5 RAM, and an AMD Ryzen 9 7900 CPU.

At 512x512 resolution and constant 8 FPS, generation time takes less than 40 seconds for 2-second video, and at most 110 seconds for 6 seconds of video. At higher resolutions, the frames are generated at maximum 512x512 resolution and 8 FPS, and then upscaled and interpolated if necessary. This multi-stage architecture proves far more efficient than attempting direct generation at target parameters. For example, interpolation from 8 FPS to 24 FPS at 512x512 resolution can reduce generation time by factor of 10 for 6 seconds of video. Similarly, generating 1024x1024 videos through upscaling from 512x512 is over 25 times faster (for 5 seconds of video) than direct high-resolution generation, which fails with out-of-memory errors for videos longer than 5 seconds.

Those results validate our core architectural decisions. By introducing multiple specialized stages to our video generation pipeline, we achieved the system that generates videos in reasonable time constraints on consumer hardware.

5.5. Limitations and Known Issues

Despite achieving our core objectives, the system faces several limitations that are primarily stemming from hardware constraints.

5.5.1. Hardware Limitations

The most significant limitation affecting our system is GPU availability. While newer or larger models can produce higher-quality videos, we had to prioritize models that can operate within VRAM constraints - many modern open-source models designed specifically for video generation deliver better results than AnimateDiff, but their hardware requirements exceeded what our hardware could support (i.e. Wan2.2 requires over 22 GB of VRAM to run video generation in 720p).

Another constraint that comes with limited resources is maximum video length - generating videos longer than 6 seconds becomes increasingly impractical due to memory consumption that grows linearly with frame count. While the system can technically produce longer videos, generation time extends beyond what most users consider acceptable for interactive use (i.e. generating 10 seconds of video at 512x512 and 8 FPS takes about 20 minutes).

Hardware constraints also forced us to set maximum base generation resolution at 512x512 and base frame rate at constant 8 FPS - parameters chosen not because they represent optimal quality, but because they allow reliable generation without CUDA out-of-memory errors.

Lastly, our hardware configuration prevented us from fully testing the system scalability characteristics. Ray Serve's architecture supports deploying pipeline replicas across multiple GPU nodes, but we had access to only a single GPU. While we validated that the system can spawn concurrent pipeline instances and distribute requests appropriately, we could not measure how performance scales when adding additional GPU resources. We also could not measure how efficiently the system handles multiple, simultaneous generation requests.

5.5.2. Architectural Limitations

The visual quality and temporal coherence of videos generated by our system falls short of what commercial solutions can achieve. While Sora and Veo produce outputs with consistent physics, good handling of complex motions, and high overall visual quality, our reliance on AnimateDiff and open-weight Stable Diffusion models means we inherit their limitations: occasional temporal inconsistencies, artifacts in complex scenes and difficulties maintaining coherent object permanence across longer sequences.

The upscaling and interpolation stages, while drastically improving efficiency, introduce their own artifacts. RealESRGAN occasionally produces oversharpened textures or unrealistic details. FILM interpolation can struggle with fast motion or transitions.

The fixed 8 FPS base generation creates another constraint: output frame rates must be multiples of 8 - we cannot produce videos at arbitrary frame rates like 30 or 60 FPS.

5.6. Future Improvements

While the current system successfully demonstrates text-to-video generation on consumer-grade hardware, several promising directions could enhance its capabilities, performance and usability.

5.6.1. Pipeline Improvements

The rapid evolution of open-source video generation models presents continuous opportunities for quality improvements. If professional-grade GPUs with larger VRAM and computational power become available, the system could leverage significantly more capable models.

They could enable direct generation at high resolutions like 1080p at arbitrary frame rate, potentially eliminating the need for interpolating and upscaling stages entirely - what would lead to improved output quality and reduced artifacts introduced by post-processing.

Similarly, the interpolation and upscaling stages could benefit from more advanced models. Newer frame interpolation or upscaling neural networks might reduce the artifacts we currently observe, improving the overall output quality of the multi-stage pipeline.

5.6.2. Feature Extensions

The system currently focuses exclusively on text-to-video generation, but the underlying architecture could support additional generation models.

Image-to-video generation - where users provide a starting frame and model animates it - would leverage similar technical foundations while opening new creative possibilities. Audio generation and synchronization represents another natural extension. Integrating text-to-audio models to generate soundtracks or sound effects synchronized with video content would create a more complete multimedia generation experience.

The current web interface provides essential functionality but could be extended with features that improve the creative workflow. A prompt template library offering pre-composed prompts for common scenarios would help users achieve desired aesthetic more quickly. Style transfer capabilities, allowing users to reference existing videos or images as style guides could provide more intuitive control than text prompts alone. A more sophisticated gallery with tagging, search and organization features would help users manage larger collections of generated videos.

Collaborative features — shared libraries, commenting, reviewing, remixing each other's generations with modified prompts — could build community engagement and encourage creative exploration. Creation of a dedicated social media-like feed presenting generations created by other users, which could encourage more frequent engagement with the application.

5.6.3. Infrastructure Scaling

The current single-node deployment is only the initial configuration of what could become much larger distributed system. Ray Serve's architecture was chosen specifically to make horizontal scaling easier, and there are several cloud deployment strategies that could increase the system's capacity and reliability significantly.

Cloud GPU Provisioning

Services like Lambda Cloud, AWS, Google Cloud Platform, and Azure give on-demand access to high-performance GPU instances that would allow for substantial throughput improvements. Lambda Cloud provides cost-effective access to NVIDIA A100 and H100 GPUs, which have 40-80 GB of VRAM compared to our current 12 GB—this would enable direct generation at higher resolutions using models like Wan2.2 [wan_ai_wan22_i2v_2024]. AWS EC2 instances with NVIDIA A10G or A100 GPUs could be set up through Auto Scaling Groups, which would automatically spawn additional GPU workers when there is high demand and terminate them during idle periods to save on costs.

For production deployment, we could use Kubernetes together with the NVIDIA GPU Operator to orchestrate containerized pipeline replicas across a heterogeneous cluster. Ray has native Kubernetes integration through KubeRay that would allow seamless scaling of the generation

pipeline, with the Ray autoscaler creating additional worker pods based on how many jobs are waiting in the queue. This kind of approach would let the system handle hundreds of concurrent generation requests by distributing the workload across dozens of GPU nodes.

Cost Optimization Strategies

Cloud GPU instances have significant hourly costs, so efficient resource utilization is very important for economic viability. Spot instances on AWS or preemptible VMs on GCP offer 60-90% cost reductions when compared to on-demand pricing, but they require the system to handle potential interruptions in a graceful way. The pipeline's checkpoint capabilities could be extended to save intermediate generation state, which would allow interrupted jobs to resume on newly provisioned instances instead of having to restart from the beginning.

A tiered service model could help balance cost and user experience: users on free tier might have to wait in queue for shared GPU resources with longer wait times, while premium users would get access to dedicated GPU pools with guaranteed capacity. Also, implementing request batching—where multiple short video generation requests get processed together—could improve GPU utilization and reduce the per-video costs.

5.7. Conclusions and Reflections

5.7.1. Project Success Evaluation

When starting this thesis, we established a clear goal: to design and develop a web application that enables users to generate videos from textual descriptions through a multi-stage pipeline approach. After approximately 10 months of development, we can assess our success against this objective.

We successfully implemented a complete multi-stage video generation pipeline that operates within consumer-grade hardware constraints. The system integrates AnimateDiff for base frame generation, FILM for frame interpolation and RealESRGAN for upscaling into pipeline orchestrated through Ray Serve.

The containerized backend infrastructure handles the complexity of long-running GPU workloads effectively. The separation between API Server and generation workload was a good architectural decision, enabling independent scaling. PostgreSQL and MinIO clearly separate metadata and file storage, simplifying both implementation and debugging.

We delivered a functional web application that makes text-to-video generation accessible without requiring technical expertise. The interface provides two versions of the video generation view - simplified for those less experienced with generative AI, and fully customizable for those seeking maximum control. Advanced users can experiment with different base models, resolutions, frame rates and apply camera movement effects. The interface is bilingual, supporting both Polish and English languages.

5.7.2. Challenges Faced During Development

Throughout the development process, we faced several technical challenges that required creative problem-solving.

The most significant challenge was managing GPU memory constraints. Early attempts at direct high-resolution and high frame rate generation consistently failed with CUDA out-of-

memory errors, forcing us to rethink our approach. This led to the multi-stage architecture we adopted, which remediated those issues.

Integrating the FastAPI endpoints with Ray’s distributed execution model presented unexpected complications. The two frameworks handle asynchronous operations differently, and coordinating between the API server’s request handling and Ray’s distributed actors required understanding both systems deeply. Also, implementing proper error handling and cleanup when generation job failed or were cancelled required multiple iterations to get right.

Real-time progress tracking demanded solving the problem of communication between GPU processes and web clients. `AnimateDiff` doesn’t naturally expose fine-grained progress information, so we had to implement callbacks at strategic points.

5.7.3. Satisfaction with Outcomes

Reflecting on the completed system, we feel genuine satisfaction with what we accomplished while maintaining realistic perspective about its limitations.

We are proud that the system works reliably within its design parameters. Users can generate videos from text prompts through user-friendly interface, the pipeline handles generation efficiently, and outputs match user expectations given the constraints we mentioned. The system doesn’t crash under normal usage, handles concurrent requests appropriately, and provides clear feedback when operations fail. This reliability, often taken for granted, required substantial effort to achieve and represents meaningful engineering success.

We are particularly satisfied with the performance optimizations that made the system practical. The multi-stage approach enabling over 10x speedups compared to direct generation represents genuine insight into how to work within hardware constraints effectively.

However, we maintain realistic expectations about the system’s place in the broader landscape. Our outputs don’t match Sora or Veo quality, our videos are shorter than commercial solutions produce, and our hardware requirements remain non-trivial despite optimizations.

We are also aware of implementation gaps and technical debt. Some error handling could be more robust, the codebase could benefit from more comprehensive testing, and documentation could be more thorough. These gaps reflect the realities of development timelines and prioritization, but acknowledging them prevents overestimating what we delivered.

Looking back at the whole project, we think the most valuable part was not the final application itself, but what we learned along the way. Before starting, we had only theoretical knowledge about diffusion models from online resources—actually making them work on real hardware turned out to be completely different experience. We learned that GPU memory is always the bottleneck, that distributed systems are harder to debug than we expected, and that keeping frontend and backend in sync gets complicated when heavy computation runs in background. We also gained practical experience with technologies that are becoming increasingly important in the industry: containerization with Docker, distributed computing with Ray, modern web frameworks. These skills will be useful regardless of whether we continue working with generative AI or move to other areas.

In the end, this thesis showed us that building AI-powered applications is not just about choosing the right model. It requires understanding the entire stack, from GPU memory management to user interface design, and making many compromises along the way.

Bibliography

- [1] A. M. Turing. “Computing Machinery and Intelligence”. In: *Mind* 49 (1950), pp. 433–460.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 25. Curran Associates, Inc., 2012. URL: https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.
- [3] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. *Generative Adversarial Networks*. arXiv:1406.2661 [stat]. June 2014. DOI: [10.48550/arXiv.1406.2661](https://doi.org/10.48550/arXiv.1406.2661).
- [4] D. P. Kingma and M. Welling. *Auto-Encoding Variational Bayes*. arXiv:1312.6114 [stat] version: 10. May 2014. DOI: [10.48550/arXiv.1312.6114](https://doi.org/10.48550/arXiv.1312.6114).
- [5] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. arXiv:1503.03585 [cs]. Nov. 2015. DOI: [10.48550/arXiv.1503.03585](https://doi.org/10.48550/arXiv.1503.03585).
- [6] J. Ho, A. Jain, and P. Abbeel. *Denoising Diffusion Probabilistic Models*. arXiv:2006.11239 [cs]. Dec. 2020. DOI: [10.48550/arXiv.2006.11239](https://doi.org/10.48550/arXiv.2006.11239).
- [7] D. Bahdanau, K. Cho, and Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv:1409.0473 [cs] version: 3. Oct. 2014. DOI: [10.48550/arXiv.1409.0473](https://doi.org/10.48550/arXiv.1409.0473).
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. *Attention Is All You Need*. arXiv:1706.03762 [cs] version: 5. Dec. 2017. DOI: [10.48550/arXiv.1706.03762](https://doi.org/10.48550/arXiv.1706.03762).
- [9] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever. *Jukebox: A Generative Model for Music*. arXiv:2005.00341 [eess]. Apr. 2020. DOI: [10.48550/arXiv.2005.00341](https://doi.org/10.48550/arXiv.2005.00341).
- [10] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall. *DreamFusion: Text-to-3D using 2D Diffusion*. arXiv:2209.14988 [cs]. Sept. 2022. DOI: [10.48550/arXiv.2209.14988](https://doi.org/10.48550/arXiv.2209.14988).
- [11] A. v. d. Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu. *WaveNet: A Generative Model for Raw Audio*. arXiv:1609.03499 [cs]. Sept. 2016. DOI: [10.48550/arXiv.1609.03499](https://doi.org/10.48550/arXiv.1609.03499).
- [12] A. Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. arXiv:2103.00020 [cs]. Feb. 2021. DOI: [10.48550/arXiv.2103.00020](https://doi.org/10.48550/arXiv.2103.00020).
- [13] J. Ho et al. *Imagen Video: High Definition Video Generation with Diffusion Models*. arXiv:2210.02303 [cs]. Oct. 2022. DOI: [10.48550/arXiv.2210.02303](https://doi.org/10.48550/arXiv.2210.02303).

-
- [14] *Sora: Creating video from text*. en-US. URL: <https://openai.com/index/sora/>.
 - [15] *Veo*. en. URL: <https://deepmind.google/models/veo/>.
 - [16] W. Wang, H. Yang, Z. Tuo, H. He, J. Zhu, J. Fu, and J. Liu. *Swap Attention in Spatiotemporal Diffusions for Text-to-Video Generation*. arXiv:2305.10874 [cs]. Apr. 2024. DOI: [10.48550/arXiv.2305.10874](https://doi.org/10.48550/arXiv.2305.10874).
 - [17] J. Cho, F. D. Puspitasari, S. Zheng, J. Zheng, L.-H. Lee, T.-H. Kim, C. S. Hong, and C. Zhang. *Sora as an AGI World Model? A Complete Survey on Text-to-Video Generation*. en. arXiv:2403.05131 [cs]. June 2024. DOI: [10.48550/arXiv.2403.05131](https://doi.org/10.48550/arXiv.2403.05131).
 - [18] Y. Liu et al. *Sora: A Review on Background, Technology, Limitations, and Opportunities of Large Vision Models*. en. arXiv:2402.17177 [cs]. Apr. 2024. DOI: [10.48550/arXiv.2402.17177](https://doi.org/10.48550/arXiv.2402.17177).
 - [19] J. Pennington, R. Socher, and C. Manning. “Glove: Global Vectors for Word Representation”. en. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics, 2014, pp. 1532–1543. DOI: [10.3115/v1/D14-1162](https://doi.org/10.3115/v1/D14-1162).
 - [20] R. Kiros, Y. Zhu, R. Salakhutdinov, R. S. Zemel, A. Torralba, R. Urtasun, and S. Fidler. *Skip-Thought Vectors*. arXiv:1506.06726 [cs]. June 2015. DOI: [10.48550/arXiv.1506.06726](https://doi.org/10.48550/arXiv.1506.06726).
 - [21] Y. Pan, Z. Qiu, T. Yao, H. Li, and T. Mei. *To Create What You Tell: Generating Videos from Captions*. arXiv:1804.08264 [cs]. Apr. 2018. DOI: [10.48550/arXiv.1804.08264](https://doi.org/10.48550/arXiv.1804.08264).
 - [22] Y. Li, M. R. Min, D. Shen, D. Carlson, and L. Carin. *Video Generation From Text*. arXiv:1710.00421 [cs]. Oct. 2017. DOI: [10.48550/arXiv.1710.00421](https://doi.org/10.48550/arXiv.1710.00421).
 - [23] Y. Li, Z. Gan, Y. Shen, J. Liu, Y. Cheng, Y. Wu, L. Carin, D. Carlson, and J. Gao. *StoryGAN: A Sequential Conditional GAN for Story Visualization*. arXiv:1812.02784 [cs]. Apr. 2019. DOI: [10.48550/arXiv.1812.02784](https://doi.org/10.48550/arXiv.1812.02784).
 - [24] D. Kim, D. Joo, and J. Kim. *TiVGAN: Text to Image to Video Generation with Step-by-Step Evolutionary Generator*. arXiv:2009.02018 [cs]. June 2021. DOI: [10.48550/arXiv.2009.02018](https://doi.org/10.48550/arXiv.2009.02018).
 - [25] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv:1810.04805 [cs]. May 2019. DOI: [10.48550/arXiv.1810.04805](https://doi.org/10.48550/arXiv.1810.04805).
 - [26] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. en. arXiv:1910.10683 [cs]. Sept. 2023. DOI: [10.48550/arXiv.1910.10683](https://doi.org/10.48550/arXiv.1910.10683).
 - [27] R. Villegas, M. Babaeizadeh, P.-J. Kindermans, H. Moraldo, H. Zhang, M. T. Saffar, S. Castro, J. Kunze, and D. Erhan. *Phenaki: Variable Length Video Generation From Open Domain Textual Description*. arXiv:2210.02399 [cs]. Oct. 2022. DOI: [10.48550/arXiv.2210.02399](https://doi.org/10.48550/arXiv.2210.02399).
 - [28] C. Wu, L. Huang, Q. Zhang, B. Li, L. Ji, F. Yang, G. Sapiro, and N. Duan. *GODIVA: Generating Open-Domain Videos from Natural Descriptions*. arXiv:2104.14806 [cs]. Apr. 2021. DOI: [10.48550/arXiv.2104.14806](https://doi.org/10.48550/arXiv.2104.14806).
 - [29] W. Hong, M. Ding, W. Zheng, X. Liu, and J. Tang. *CogVideo: Large-scale Pretraining for Text-to-Video Generation via Transformers*. arXiv:2205.15868 [cs]. May 2022. DOI: [10.48550/arXiv.2205.15868](https://doi.org/10.48550/arXiv.2205.15868).

- [30] D. Kondratyuk et al. *VideoPoet: A Large Language Model for Zero-Shot Video Generation*. arXiv:2312.14125 [cs]. June 2024. DOI: [10.48550/arXiv.2312.14125](https://doi.org/10.48550/arXiv.2312.14125).
- [31] U. Singer et al. *Make-A-Video: Text-to-Video Generation without Text-Video Data*. arXiv:2209.14792 [cs]. Sept. 2022. DOI: [10.48550/arXiv.2209.14792](https://doi.org/10.48550/arXiv.2209.14792).
- [32] Y. Ma, Y. He, X. Cun, X. Wang, S. Chen, Y. Shan, X. Li, and Q. Chen. *Follow Your Pose: Pose-Guided Text-to-Video Generation using Pose-Free Videos*. arXiv:2304.01186 [cs]. Jan. 2024. DOI: [10.48550/arXiv.2304.01186](https://doi.org/10.48550/arXiv.2304.01186).
- [33] J. Z. Wu, Y. Ge, X. Wang, W. Lei, Y. Gu, Y. Shi, W. Hsu, Y. Shan, X. Qie, and M. Z. Shou. *Tune-A-Video: One-Shot Tuning of Image Diffusion Models for Text-to-Video Generation*. arXiv:2212.11565 [cs]. Mar. 2023. DOI: [10.48550/arXiv.2212.11565](https://doi.org/10.48550/arXiv.2212.11565).
- [34] Y. Guo, C. Yang, A. Rao, Z. Liang, Y. Wang, Y. Qiao, M. Agrawala, D. Lin, and B. Dai. *AnimateDiff: Animate Your Personalized Text-to-Image Diffusion Models without Specific Tuning*. arXiv:2307.04725 [cs]. Feb. 2024. DOI: [10.48550/arXiv.2307.04725](https://doi.org/10.48550/arXiv.2307.04725).
- [35] A. Blattmann, R. Rombach, H. Ling, T. Dockhorn, S. W. Kim, S. Fidler, and K. Kreis. *Align your Latents: High-Resolution Video Synthesis with Latent Diffusion Models*. arXiv:2304.08818 [cs]. Dec. 2023. DOI: [10.48550/arXiv.2304.08818](https://doi.org/10.48550/arXiv.2304.08818).
- [36] A. Blattmann et al. *Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets*. arXiv:2311.15127 [cs]. Nov. 2023. DOI: [10.48550/arXiv.2311.15127](https://doi.org/10.48550/arXiv.2311.15127).
- [37] H. Song, L. Dong, W.-N. Zhang, T. Liu, and F. Wei. *CLIP Models are Few-shot Learners: Empirical Studies on VQA and Visual Entailment*. arXiv:2203.07190 [cs]. Mar. 2022. DOI: [10.48550/arXiv.2203.07190](https://doi.org/10.48550/arXiv.2203.07190).
- [38] A. v. d. Oord, O. Vinyals, and K. Kavukcuoglu. *Neural Discrete Representation Learning*. arXiv:1711.00937 [cs]. May 2018. DOI: [10.48550/arXiv.1711.00937](https://doi.org/10.48550/arXiv.1711.00937).
- [39] A. Maharana, D. Hannan, and M. Bansal. *StoryDALL-E: Adapting Pretrained Text-to-Image Transformers for Story Continuation*. arXiv:2209.06192 [cs]. Sept. 2022. DOI: [10.48550/arXiv.2209.06192](https://doi.org/10.48550/arXiv.2209.06192).
- [40] Y. Jiang, S. Yang, T. L. Koh, W. Wu, C. C. Loy, and Z. Liu. *Text2Performer: Text-Driven Human Video Generation*. arXiv:2304.08483 [cs]. Apr. 2023. DOI: [10.48550/arXiv.2304.08483](https://doi.org/10.48550/arXiv.2304.08483).
- [41] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid. *ViViT: A Video Vision Transformer*. arXiv:2103.15691 [cs]. Nov. 2021. DOI: [10.48550/arXiv.2103.15691](https://doi.org/10.48550/arXiv.2103.15691).
- [42] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models*. en. arXiv:2112.10752 [cs]. Apr. 2022. DOI: [10.48550/arXiv.2112.10752](https://doi.org/10.48550/arXiv.2112.10752).
- [43] W. Peebles and S. Xie. *Scalable Diffusion Models with Transformers*. arXiv:2212.09748 [cs]. Mar. 2023. DOI: [10.48550/arXiv.2212.09748](https://doi.org/10.48550/arXiv.2212.09748).
- [44] J. Lv, Y. Huang, M. Yan, J. Huang, J. Liu, Y. Liu, Y. Wen, X. Chen, and S. Chen. *GPT4Motion: Scripting Physical Motions in Text-to-Video Generation via Blender-Oriented GPT Planning*. arXiv:2311.12631 [cs]. Apr. 2024. DOI: [10.48550/arXiv.2311.12631](https://doi.org/10.48550/arXiv.2311.12631).
- [45] H. Fei, S. Wu, W. Ji, H. Zhang, and T.-S. Chua. *Dysen-VDM: Empowering Dynamics-aware Text-to-Video Diffusion with LLMs*. arXiv:2308.13812 [cs]. Mar. 2024. DOI: [10.48550/arXiv.2308.13812](https://doi.org/10.48550/arXiv.2308.13812).

-
- [46] S. Yin et al. *NUWA-XL: Diffusion over Diffusion for eXtremely Long Video Generation*. arXiv:2303.12346 [cs]. Mar. 2023. DOI: [10.48550/arXiv.2303.12346](https://doi.org/10.48550/arXiv.2303.12346).
 - [47] Y. Hu, C. Luo, and Z. Chen. *Make It Move: Controllable Image-to-Video Generation with Text Descriptions*. arXiv:2112.02815 [cs]. Mar. 2022. DOI: [10.48550/arXiv.2112.02815](https://doi.org/10.48550/arXiv.2112.02815).
 - [48] A. Gupta, L. Yu, K. Sohn, X. Gu, M. Hahn, L. Fei-Fei, I. Essa, L. Jiang, and J. Lezama. *Photorealistic Video Generation with Diffusion Models*. arXiv:2312.06662 [cs]. Dec. 2023. DOI: [10.48550/arXiv.2312.06662](https://doi.org/10.48550/arXiv.2312.06662).
 - [49] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. *Neural Ordinary Differential Equations*. arXiv:1806.07366 [cs]. Dec. 2019. DOI: [10.48550/arXiv.1806.07366](https://doi.org/10.48550/arXiv.1806.07366).
 - [50] D. Ahn, D. Kim, G. Song, S. H. Kim, H. Lee, D. Kang, and J. Choi. *Story Visualization by Online Text Augmentation with Context Memory*. arXiv:2308.07575 [cs]. Aug. 2023. DOI: [10.48550/arXiv.2308.07575](https://doi.org/10.48550/arXiv.2308.07575).
 - [51] D. Kim, D. Joo, and J. Kim. “TiVGAN: Text to Image to Video Generation With Step-by-Step Evolutionary Generator”. In: *IEEE Access* 8 (2020), pp. 153113–153122. DOI: [10.1109/ACCESS.2020.3017881](https://doi.org/10.1109/ACCESS.2020.3017881).
 - [52] J. Lei, L. Wang, Y. Shen, D. Yu, T. L. Berg, and M. Bansal. *MART: Memory-Augmented Recurrent Transformer for Coherent Video Paragraph Captioning*. arXiv:2005.05402 [cs]. May 2020. DOI: [10.48550/arXiv.2005.05402](https://doi.org/10.48550/arXiv.2005.05402).
 - [53] G. Mittal, T. Marwah, and V. N. Balasubramanian. “Sync-DRAW: Automatic Video Generation using Deep Recurrent Attentive Architectures”. In: *Proceedings of the 25th ACM international conference on Multimedia*. arXiv:1611.10314 [cs]. Oct. 2017, pp. 1096–1104. DOI: [10.1145/3123266.3123309](https://doi.org/10.1145/3123266.3123309).
 - [54] R. Kumar, J. Sotelo, K. Kumar, A. d. Brebisson, and Y. Bengio. *ObamaNet: Photo-realistic lip-sync from text*. arXiv:1801.01442 [cs]. Dec. 2017. DOI: [10.48550/arXiv.1801.01442](https://doi.org/10.48550/arXiv.1801.01442).
 - [55] F. Chollet. *Xception: Deep Learning with Depthwise Separable Convolutions*. arXiv:1610.02357 [cs]. Apr. 2017. DOI: [10.48550/arXiv.1610.02357](https://doi.org/10.48550/arXiv.1610.02357).
 - [56] S. Ge, S. Nah, G. Liu, T. Poon, A. Tao, B. Catanzaro, D. Jacobs, J.-B. Huang, M.-Y. Liu, and Y. Balaji. *Preserve Your Own Correlation: A Noise Prior for Video Diffusion Models*. arXiv:2305.10474 [cs]. Mar. 2024. DOI: [10.48550/arXiv.2305.10474](https://doi.org/10.48550/arXiv.2305.10474).
 - [57] J. An, S. Zhang, H. Yang, S. Gupta, J.-B. Huang, J. Luo, and X. Yin. *Latent-Shift: Latent Diffusion with Temporal Shift for Efficient Text-to-Video Generation*. arXiv:2304.08477 [cs]. Apr. 2023. DOI: [10.48550/arXiv.2304.08477](https://doi.org/10.48550/arXiv.2304.08477).
 - [58] H. Chen, Y. Zhang, X. Cun, M. Xia, X. Wang, C. Weng, and Y. Shan. *VideoCrafter2: Overcoming Data Limitations for High-Quality Video Diffusion Models*. en. Jan. 2024. URL: <https://arxiv.org/abs/2401.09047v1>.
 - [59] H. Chen, X. Wang, G. Zeng, Y. Zhang, Y. Zhou, F. Han, Y. Wu, and W. Zhu. *Video-Dreamer: Customized Multi-Subject Text-to-Video Generation with Disen-Mix Finetuning on Language-Video Foundation Models*. arXiv:2311.00990 [cs]. Apr. 2025. DOI: [10.48550/arXiv.2311.00990](https://doi.org/10.48550/arXiv.2311.00990).
 - [60] A. Blattmann et al. *Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets*. en. arXiv:2311.15127 [cs]. Nov. 2023. DOI: [10.48550/arXiv.2311.15127](https://doi.org/10.48550/arXiv.2311.15127).

- [61] O. Bar-Tal et al. *Lumiere: A Space-Time Diffusion Model for Video Generation*. arXiv:2401.12945 [cs]. Feb. 2024. DOI: [10.48550/arXiv.2401.12945](https://doi.org/10.48550/arXiv.2401.12945).
- [62] B. Qin, W. Ye, Q. Yu, S. Tang, and Y. Zhuang. *Dancing Avatar: Pose and Text-Guided Human Motion Videos Synthesis with Image Diffusion Model*. arXiv:2308.07749 [cs]. Aug. 2023. DOI: [10.48550/arXiv.2308.07749](https://doi.org/10.48550/arXiv.2308.07749).
- [63] Y. Lu, L. Zhu, H. Fan, and Y. Yang. *FlowZero: Zero-Shot Text-to-Video Synthesis with LLM-Driven Dynamic Scene Syntax*. arXiv:2311.15813 [cs]. Nov. 2023. DOI: [10.48550/arXiv.2311.15813](https://doi.org/10.48550/arXiv.2311.15813).
- [64] H. Huang, Y. Feng, C. Shi, L. Xu, J. Yu, and S. Yang. *Free-Bloom: Zero-Shot Text-to-Video Generator with LLM Director and LDM Animator*. arXiv:2309.14494 [cs]. Sept. 2023. DOI: [10.48550/arXiv.2309.14494](https://doi.org/10.48550/arXiv.2309.14494).
- [65] J. Wang, F. Zhang, X. Li, V. Y. F. Tan, T. Pang, C. Du, A. Sun, and Z. Yang. *Error Analyses of Auto-Regressive Video Diffusion Models: A Unified Framework*. arXiv:2503.10704 [cs]. Mar. 2025. DOI: [10.48550/arXiv.2503.10704](https://doi.org/10.48550/arXiv.2503.10704).
- [66] F. Reda, J. Kontkanen, E. Tabellion, D. Sun, C. Pantofaru, and B. Curless. *FILM: Frame Interpolation for Large Motion*. arXiv:2202.04901 [cs]. July 2022. DOI: [10.48550/arXiv.2202.04901](https://doi.org/10.48550/arXiv.2202.04901).
- [67] X. Wang, L. Xie, C. Dong, and Y. Shan. *Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data*. arXiv:2107.10833 [eess]. Aug. 2021. DOI: [10.48550/arXiv.2107.10833](https://doi.org/10.48550/arXiv.2107.10833).
- [68] X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, C. C. Loy, Y. Qiao, and X. Tang. *ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks*. arXiv:1809.00219 [cs]. Sept. 2018. DOI: [10.48550/arXiv.1809.00219](https://doi.org/10.48550/arXiv.1809.00219).

List of Figures

1.1	Text-to-video generation example by Imagen Video	10
1.2	CLIP contrastive learning approach	14
1.3	Architecture of the Latent Diffusion Model	17
1.4	Pseudo-3D convolutional and attention layers from Make-A-Video	20
1.5	AnimateDiff inference architecture with Motion Module integration	23
1.6	Real-ESRGAN generator architecture for image super-resolution	26
2.1	Mapping between user test scenarios and functional requirements	45
3.1	High-level system architecture.	47
3.2	Application logo displayed in the dashboard.	48
3.3	Video generation pipeline architecture.	49
3.4	Total generation time across the complete pipeline for various video lengths and resolutions at 8 FPS.	62
3.5	Generation time comparison for 16 FPS. The interpolation approach achieves nearly 10x speedup for longer videos.	63
3.6	Generation time comparison for 24 FPS. The interpolation approach achieves nearly 20x speedup for longer videos.	63
3.7	Generation time comparison for 1024x1024 output. The upscaling approach achieves 15-30x speedup for longer videos.	64
4.1	Sharing deep learning research findings on Discord	71
4.2	Sharing model deployment best practices	71
4.3	Screenshot of the API documentation view in Swagger UI.	73
5.1	Homepage of the application in English.	77
5.2	Homepage of the application in Polish.	77
5.3	View of the login form with a password that does not meet the validation requirements.	79
5.4	View of the empty registration form in the system.	80
5.5	View of the verification email notification displayed after account registration or reactivation.	80
5.6	Video creation view with the expanded Instructions section.	81
5.7	Video creation view with the expanded Quick Generation section.	82
5.8	Video creation view with the expanded Full Generation section.	83
5.9	View of the table with all jobs.	84
5.10	View of active jobs updated in real time.	84
5.11	View of all user generations.	85
5.12	View of the full generation preview.	86

5.13	Panel displaying all available sharing options.	86
5.14	Modal informing the user that direct sharing to Instagram is not supported. . . .	87
5.15	View of the shared generation preview.	87
5.16	View of the Explore section with example application generations.	88
5.17	View of the User Settings page.	89

List of Tables

5.1	Color palette of the AuraAnim application	78
-----	---	----